

ASHESI UNIVERSITY

DUOSIGN: A GLOSS-BASED APPROACH TO AUTOMATED ASL GENERATION

APPLIED PROJECT

B.Sc. Computer Science

Nana Kwaku Amoako

2026

ASHESI UNIVERSITY

DUOSIGN: A GLOSS-BASED APPROACH TO AUTOMATED ASL GENERATION

APPLIED PROJECT

APPLIED PROJECT submitted to the Department of Computer Science & Information Systems, Ashesi University in partial fulfilment of the requirements for the award of Bachelor of Science degree in Computer Science.

Nana Kwaku Amoako

2026

DECLARATION

I hereby declare that this APPLIED PROJECT is the result of my own original work and that no part of it has been presented for another degree in this university or elsewhere.

Candidate's Signature:

.....

Candidate's Name:

.....

Date:

.....

I hereby declare that preparation and presentation of this APPLIED PROJECT was supervised in accordance with the guidelines on supervision of APPLIED PROJECT laid down by Ashesi University.

Supervisor's Signature:

.....

Supervisor's Name:

.....

Date:

.....

Acknowledgements

First and foremost, I give all glory to God—for the idea, the wisdom, the grace, the vision, the strength, and the intuition to carry this project from concept to completion.

To Kwabena Bamfo, thank you for believing in me and in this idea from the very beginning. Your confidence in me was fuel for this journey.

To my friends and family, thank you for being my steady support system through the writing, building, testing, and refining of this work. Your encouragement never went unnoticed.

To Vera Anthonio CS' 26, thank you for that timely conversation when I felt lost and was searching for direction for my final thesis. God used you in that moment, and I remain deeply grateful.

To the Accessibility Department at Ashesi—Nick, Mustapha, and Francis—thank you for believing in this vision and supporting it wholeheartedly from day one.

To my peer reviewers, thank you for your careful eyes and thoughtful reflections. Your feedback strengthened my writing and sharpened the flow of this project.

To everyone who tested the system and shared honest feedback, thank you. Your input directly improved DuoSign and made it more useful in real-world contexts.

To Elikem Gale-Zoyiku CS' 25 and Victor Quagraine CS' 25, thank you for showing up for me in a difficult moment and helping me better understand the software and technical foundations of this work.

I also acknowledge Dongxu Li and collaborators for creating the WLASL dataset and for granting access to the requested data used in this project. Their work provided critical infrastructure for this research.

To my parents, thank you for your sacrifices, for funding my education, and for supporting my dreams and ambitions with unwavering love.

Finally, to every person whose name is not written here but whose prayers, advice, kindness, and encouragement carried me along this path: thank you. This work is, in many ways, yours too.

Abstract

Communication tools for deaf people often work in one direction: they help hearing people understand sign language, but they do not reliably return information in sign language for deaf users. DuoSign was built to address this gap. It is a web-based system that takes English input and presents the output as animated American Sign Language (ASL), without requiring specialised hardware.

DuoSign follows a three-stage workflow: it converts English text into ASL gloss, maps each gloss to sign motion data derived from real signing videos, and renders the result through a 3D signing avatar. The system also includes fallback strategies such as synonym matching and fingerspelling to reduce missing-word failures. Because the motion data is grounded in human signing video, the produced animations are generally more natural than purely notation-driven output.

A key contribution of this work is portability. The same pipeline can be reused for other sign languages when video resources are available, including low-resource settings where large annotated corpora are limited. In this sense, DuoSign is both a working ASL generation tool and a practical baseline for communities that want to bootstrap their own sign-language accessibility systems.

Keywords

Keywords: text-to-sign generation, sign language technology, accessibility, gloss translation, pose estimation, avatar animation, American Sign Language (ASL), computer vision, natural language processing (NLP), assistive technology.

Table of Contents

DECLARATION i

Acknowledgements ii

Abstract iii

Keywords iv

Table of Contents v

List of Figures xi

List of Tables xiv

List of Abbreviations and Technical Terms xiv

1 Introduction 1

1.1 Background 2

1.2 Problem Definition & Context 3

1.2.1 Directionality Gap: Who Gets Served? 4

1.2.2 Intelligibility-Replicability Tradeoff 5

1.3 Objectives and Significance 6

1.3.1 Contribution to Sustainable Development Goals 7

1.4 Review of Literature and Related Works 7

1.4.1 History of Sign Language Technology 8

1.4.2 Sign Language Linguistics: Why This Is Harder Than It Looks . . . 8

1.4.3 Glosses: Necessary Evil or Fundamental Limitation? 9

1.4.4 Skeletal Poses as Representation 10

1.4.5 Data Scarcity and the Low-Resource Challenge 11

1.4.6 Related Works: Recognition vs. Generation 13

1.4.6.1 Recognition Systems: The Dominant Direction 13
1.4.6.2 Generation Systems: Four Decades of Avatars 14
1.4.6.3 Where DuoSign Fits 14
1.4.6.4 Analysis of Related Works 15

2 Requirements **16**

2.1 Requirements Elicitation 16
2.1.1 Literature Review 16
2.1.2 Secondary Data from Existing User Studies 17
2.1.3 Accessibility Standards 17
2.1.4 Limitations and Future Evaluation 17
2.2 Key Stakeholders 17
2.3 Use Case Analysis 19
2.3.1 End-Users 19
2.3.2 System Workflows 22
2.3.2.1 Core Translation Workflow 22
2.3.2.2 Video Export Workflow 23
2.3.2.3 History and Replay Workflow 23
2.3.3 Behavioural Model 23
2.4 System Requirements 26
2.4.1 Functional Requirements 27
2.4.2 Non-Functional Requirements 28
2.4.3 In-Scope and Out-of-Scope Requirements 31
2.4.4 Requirements Traceability Matrix 31
2.4.5 Acceptance Criteria for Critical Requirements 31
2.5 DuoSign: Proposed System 33
2.6 Ethics 34

3 Architecture and Design **36**

3.1 System Architecture 36

3.1.1 Frontend Architecture . . . 37
3.1.1.1 Feature-Sliced Design (FSD) . . . 37
3.1.2 Backend Architecture . . . 37
3.1.2.1 FastAPI NLP and Export Service . . . 37
3.1.3 Detailed Architecture Using the C4 Model . . . 38
3.1.3.1 System Context Diagram (Level 1) . . . 38
3.1.3.2 Container Diagram (Level 2) . . . 40
3.1.3.3 Component Diagram (Level 3) . . . 42
3.2 System Design . . . 47
3.2.1 Design Principles . . . 47
3.2.2 Video-Driven Avatar Playback and Pose-Based Skeleton Fallback . 49
3.2.3 Data Model . . . 50
3.2.4 Interaction Diagrams . . . 53
3.2.4.1 Full Translation Pipeline Activity . . . 53
3.2.4.2 Voice Input Flow . . . 54
3.2.4.3 Guest-to-Registered User Conversion Flow . . . 56
3.2.4.4 Video Export Flow . . . 56
3.2.5 User Interface Design . . . 57
3.2.6 Security Design . . . 59
3.3 Deployment Architecture . . . 60
3.3.1 Hosting Stack . . . 62
3.3.2 Scalability Considerations . . . 63
3.4 Summary . . . 63

4 Implementation **65**

4.1 Software, Tools, and Technologies . . . 65
4.2 Software Development Approach . . . 65
4.3 Implementation Details . . . 66
4.3.1 Project File Structure . . . 66
4.3.2 Client State and Asset Storage . . . 66

4.3.3 Text-to-Gloss Translation Engine 68
4.3.3.1 Text Normalisation 68
4.3.3.2 Grammatical Transformation 68
4.3.3.3 Lexicon Validation and Fallback 69
4.3.3.4 Algorithm: Text-to-Gloss Conversion Pipeline 69
4.3.4 WLASL Pose Extraction Pipeline 71
4.3.4.1 Pipeline Architecture 71
4.3.4.2 Performance Results 74
4.3.4.3 Quality-Based Pose Selection 74
4.3.5 Pose Serialisation and Runtime Loading 75
4.3.6 Avatar Rendering and Animation 76
4.3.6.1 Kalidokit Limitations in Practice 76
4.3.7 Video Export Pipeline 77
4.3.7.1 Client-Side Canvas Capture 77
4.3.7.2 Server-Side FFmpeg Conversion 77
4.3.7.3 User Experience 78
4.3.8 User Interfaces and Functionalities 78
4.3.8.1 Translation Interface 79
4.3.8.2 Avatar Customisation 79
4.3.8.3 Translation History 81
4.3.8.4 Settings Interface 82
4.4 Integration of Components 82

5 Testing and Results **83**

5.1 Introduction 83
5.2 Component Testing 83
5.3 Accessibility Compliance 84
5.4 User Study Design 85
5.4.1 Ethical Approval and Participants 85
5.4.2 Instruments 86

5.4.3 Technical Performance . . . 86
5.4.4 System Usability Scale . . . 86
5.4.5 Expert Sign Review . . . 87
5.4.6 Avatar Quality . . . 87
5.5 User Acceptance . . . 88
5.6 User Feedback . . . 88
5.6.1 Positive Feedback . . . 88
5.6.2 Constructive Feedback . . . 88
5.6.3 Practical Use Contexts Identified . . . 89
5.7 Discussion . . . 89
5.7.1 Hypothesis Evaluation . . . 89
5.7.2 Limitations . . . 90
5.8 Analysis of Test Results . . . 90

6 Conclusions and Recommendations 91
6.1 Limitations and Challenges . . . 91
6.2 Future Work . . . 92
6.3 Requirements Compliance and Objective Attainment . . . 93
6.4 Conclusion . . . 94

References 95

Appendices 100
A AI Use Declaration . . . 100
A.1 Declaration Form . . . 100
A.2 AI Tools and Usage Summary . . . 100
B Project Structure . . . 101
B.1 Repository Layout and AI Contribution Per File . . . 101
C System Comparison Table . . . 104
D Related-Works Comparison Matrix . . . 106
E User Study Instruments . . . 108

E.1 Post-Session Questionnaire (SUS and Supplementary Items) 108
F Application Interface Screenshots 110
G Kalidokit Retargeting Capability Matrix 116
H Kalidokit and VRM implementation findings 117
I Project Management 118
I.1 Development Methodology 118
J External Links and Resources 120

List of Figures

1.1 The vicious cycle of structural barriers to sign language tool development in Africa. Arrows represent causal dependencies; the dashed line indicates DuoSign's intended intervention point. 12

2.1 Use-Case Diagram of the DuoSign System 20
2.2 Activity Diagram: Core Translation and Playback Workflow 24
2.3 Activity Diagram: Video Export Workflow 25
2.4 Activity Diagram: Translation History and Replay Workflow 26
2.5 State Machine Diagram: Avatar Animation Playback Player 26

3.1 System context diagram of the DuoSign sign language translation system. . 41
3.2 Container diagram of the DuoSign sign language translation system. 43
45figure.caption.15
47figure.caption.16
3.5 DuoSign's animation architecture: sign videos drive real-time VRM avatar playback via MediaPipe Tasks Vision and Kalidokit (primary path), while precomputed `.pose` files provide the pose-based skeleton fallback overlay when VRM quaternion output diverges from source landmarks (FR005, FR008, FR009). 51
3.6 Entity relationship diagram for the DuoSign production database schema (FR013–FR015, NFR006, NFR017). 53
3.7 Activity diagram of the full translation pipeline from text input to avatar animation. 55
3.8 Activity diagram of the dual-path voice input flow (Web Speech API and Groq Whisper paths). 56
3.9 Activity diagram of the guest-to-registered user conversion flow. 57

3.10 Activity diagram of the video export flow from canvas capture to MP4 download. 58

3.11 Deployment architecture for DuoSign. The Next.js frontend is hosted on Vercel and calls the FastAPI backend on Render for translation, transcription, and export endpoints. Media assets are delivered through Supabase Storage CDN directly to the browser, persistent data is stored in Supabase PostgreSQL, and Groq API provides Whisper transcription and optional LLM quality-refinement calls. 61

4.1 Project structure diagram for DuoSign, showing the high-level repository layout across frontend, backend, media asset storage (`bucket/`), and supporting configuration files used during development and deployment. 67

4.2 Text-to-gloss conversion pipeline in DuoSign. Input text is normalised, transformed by the rule-based ASL grammar engine, and confidence-scored against the vocabulary. Low-confidence or complex inputs trigger an optional LLM quality-refinement path, and results are returned through the streaming API as rule-based and optional quality-improved events. 71

4.3 Export video modal UI showing the three-phase recording experience. Left: *preparing* phase while the VRM model loads into the hidden canvas. Centre: *composing* phase during avatar animation playback. Right: *packaging* phase during server-side FFmpeg conversion, with progress bar at 100% before download begins. 78

4.4 DuoSign translation interface on desktop (≥1024 px). The `InputPanel` (left column) contains the text input area, microphone button, submit control, and character counter. The `AvatarPanel` (right column) displays the Three.js VRM canvas with playback controls, display mode selectors, and export button. Active gloss tokens are highlighted in the chip row below the avatar. 79

4.5 DuoSign translation interface on mobile (<1024 px). Panels stack vertically with the avatar canvas occupying the upper portion of the viewport and the input panel fixed at the bottom, maintaining touch-target sizes at a minimum of 44×44 px. 80

4.6 Translation history page. Entries are grouped by date (Today, Yesterday, named dates) and display input type icon, original text, timestamp, and metadata badges (gloss count, input method, exported flag). The expanded entry reveals the gloss token chip sequence and four action buttons: **Replay**, **Export** (reopens `ExportVideoModal`), **Edit** (pre-fills translate input), and **Delete**. The filter bar (top) supports date range, input type, search, and exported-only views. 81

4.7 Settings page with sidebar navigation across three sections: **Account**, **Translation**, and **Developer**. Settings are persisted to `localStorage` via `SettingsProvider`; unsaved changes are tracked with a dirty-state indicator and a Save/Discard action bar. 82

Appendix Figures

1 DuoSign landing page. 110

2 User login page. 110

3 User sign-up page. 111

4 Main translation workspace with avatar output. 111

5 Full-body avatar display mode. 112

6 3D world view of the avatar environment. 112

7 Skeleton view mode for pose playback. 113

8 Translation history page. 113

9 Application settings view. 114

10 Export modal for saving generated output. 114

11 Mobile-responsive interface view. 115

12 Development timeline Gantt chart. 120

List of Tables

2.1 In-scope and out-of-scope requirement boundary for this project 32

2.2 Requirements traceability matrix 32

3.1 DuoSign production hosting stack. 62

4.1 WLASL Pose Extraction Pipeline Performance 74

5.1 Automated component and unit test coverage summary. 83

5.2 Colour contrast checks against WCAG 2.1 AA. 85

5.3 WCAG 2.1 AA accessibility summary for DuoSign. 85

5.4 Technical performance summary. 86

5.5 Overall SUS results. 87

5.6 Ratings from returned post-session surveys (1–5 scale). 87

1 AI tools used in DuoSign development. 100

2 Project files with one-line descriptions and AI contribution type. 101

3 Comparison of DuoSign with existing sign language production systems. . . 105

4 Comparison of representative sign-language systems. 107

5 DuoSign post-session questionnaire and SUS items. 108

6 ASL parameter coverage under the current Kalidokit-VRM retargeting pipeline. 116

7 Practical findings from the Kalidokit and VRM implementation used in DuoSign. 117

8 Summary of DuoSign backend API endpoints. 118

9 DuoSign development timeline. 119

10 DuoSign external resources. 120

List of Abbreviations and Technical Terms

API Application Programming Interface

ASL American Sign Language

BLEU Bilingual Evaluation Understudy (automatic translation-quality metric)

BSL British Sign Language

CCC Cross-Modal Contrastive Creator

CSL Chinese Sign Language

DGS Deutsche Gebärdensprache (German Sign Language)

EMG Electromyography

FFmpeg Fast Forward MPEG multimedia framework

FPS Frames Per Second

FSD Feature-Sliced Design

GAN Generative Adversarial Network

GhSL Ghanaian Sign Language

GPU Graphics Processing Unit

HCI Human–Computer Interaction

IMU Inertial Measurement Unit

IRB Institutional Review Board

JWT JSON Web Token

KeSL Kenyan Sign Language

KSL Korean Sign Language

LLM Large Language Model

MP4 MPEG-4 Part 14 video container format

NGT Nederlandse Gebarentaal (Sign Language of the Netherlands)

NLP Natural Language Processing

NSL Nigerian Sign Language

OWASP Open Worldwide Application Security Project

RWTH-PHOENIX-Weather Benchmark sign-language dataset derived from German weather broadcasts

SiGML Signing Gesture Markup Language

SML Signing Markup Language

SSE Server-Sent Events

SUS System Usability Scale

VRM 3D humanoid avatar format used for virtual character animation

WLASL Word-Level American Sign Language dataset

XML Extensible Markup Language

Chapter 1: Introduction

Communication is central to human connection; it enables individuals to interact, build relationships, and participate in society. People communicate through five primary and often overlapping modalities: verbal speech, non-verbal cues, written text, visual representations, and active listening [25]. While these modes enable effective interaction for much of the global population, they are not equally accessible to everyone. For an estimated 430 million people worldwide who are deaf or hard-of-hearing [29], sign languages serve as a primary means of communication, offering rich linguistic structures expressed through body movement and facial expression [28, 10].

Over the past decade, human–computer interaction (HCI) has been reshaped by machine learning and computer vision, shifting from purely keyboard-and-mouse input toward more natural modalities such as vision-guided, voice-enabled, and gesture-based interaction. In assistive technology, sign language recognition systems now achieve high accuracy using RGB[1] cameras, depth sensors, and wearable IMU/EMG devices, translating sequences of signs into text or speech and thereby mediating communication between deaf and hearing users [39, 20, 18, 36, 38, 13]. Yet, these systems still face persistent challenges in robustness "in the wild," signer and domain generalization, and coverage across diverse sign languages, limiting their deployment beyond controlled settings [39, 9, 7].

In parallel, a line of work has focused on representing sign languages in machine-readable form. Early descriptive and markup languages—such as HamNoSys and its XML[2]-based derivative SiGML[3], as well as related avatar-driven systems like eSIGN and SignSynth—encode handshape, location, movement, and non-manual markers for synthesis by virtual humans [3, 2, 12, 15]. These frameworks, along with corpus-driven pipelines that map glosses to pose sequences and then to rendered video [26, 9, 11] have been crucial in standardizing how signs are described and animated. However, user studies consistently show that current pose-based

[1] RGB = Red, Green, Blue (standard color channels in digital images).
[2] XML = Extensible Markup Language.
[3] SiGML = Signing Gesture Markup Language.

and GAN[4]-based generation still struggle with fast hand motion, co-articulation, and facial grammar, yielding synthetic videos that are only partially intelligible to fluent signers [9, 35]. Recent end-to-end and gloss-free sign language production models—such as Cross-Modal Contrastive Creator (CCC) and multilingual generators like Signs as Tokens—demonstrate impressive progress, but they also underline ongoing difficulties in semantic–motion alignment, naturalness, and scalability [42, 44, 17, 43].

1.1 Background

Early sign language technologies, such as fingerspelling-to-text systems, have demonstrated promise but typically target narrow sub-tasks. For example, smartwatch-based systems that take advantage of Inertial Measurement Unit (IMU) sensors have proven to recognise ASL[5] alphabet sequences or short sentences from wrist motion alone, and EMG-based approaches classify a limited set of handshapes or isolated signs with high accuracy [14]. These systems are valuable proofs of concept, but they do not deliver the end-to-end text-to-sign capabilities that DuoSign aims to provide.

Beyond fingerspelling, prior work has explored a spectrum of sensing and synthesis strategies. Wearable devices based on IMUs, EMG, or hybrid armbands have been used for continuous ASL and CSL recognition and even end-to-end sign-to-speech translation [14, 39]. Vision-based systems such as OpenPose and MediaPipe leverage pose estimation to recognise full-body Korean Sign Language (KSL) or Chinese Sign Language (CSL) motion from RGB video and have taken a leap further in detecting non-manual signals like facial expression to boost accuracy [6, 30, 9].

On the generation side, in the early and mid-2000s, avatar animation pipelines such as eSIGN and WebSign were developed to show realistic signing avatars that were structured & linguistically informed representations that made use of markup language (e.g., SML[6], SiGML) [22, 3]. More recent gloss-based pipelines and neural sign-generation models extend this idea by composing skeleton sequences from lexicons and rendering them with

[4]GAN = Generative Adversarial Network.
[5]ASL = American Sign Language.
[6]SML = Signing Markup Language.

neural or rule-based avatars [26, 35].

While African nations host diverse Deaf communities with rich sign language traditions, including Ghanaian Sign Language (GhSL), Kenyan Sign Language (KeSL), Nigerian Sign Language (NSL), and many others, these languages remain severely underrepresented in assistive technology research. The primary barrier is data scarcity: unlike German Sign Language (DGS) Corpus, as utilized in Amit Moryossef's "An Open-Source Gloss-Based Baseline for Spoken to Signed Language Translation", which benefits from decades of corpus development, large-scale datasets, and government and institutional support, African Sign Languages lack the extensive annotated video corpora, lexical databases, and motion-capture resources needed to train modern neural generation systems. This creates a cycle in which data scarcity impedes tool development [37], because reproducing comparable corpora, annotation pipelines, and signer datasets requires substantial funding, and this further marginalises hard-of-hearing people in these communities [9, 24, 4].

DuoSign is designed to sit precisely in this gap by providing a modular, web-based, openly available English-to-ASL generation pipeline that others can inspect, extend, and benchmark against, without rebuilding the entire stack from scratch.

1.2 Problem Definition & Context

Despite rapid advances in sign-language technology, the overwhelming majority of research targets the recognition direction—converting signs into text for hearing users. The reverse direction, generating sign language from text, remains significantly underexplored in academic literature. Recent surveys of sign-language processing highlight that most benchmark efforts and public datasets concentrate on sign-to-text translation, leaving text-to-sign pipelines scarcely explored [44]. This imbalance creates a communicative asymmetry: deaf individuals can express themselves through systems that interpret their signs, yet the replies they receive are rendered only as plain written language, which is not their primary linguistic modality.

This asymmetry has a tangible human cost. Deaf individuals are frequently excluded from spontaneous, informal communication—not just formal settings—because the tools

designed to support inclusive communication have not addressed the generation side of the problem. Receiving a response in plain English, when a person's first language is sign language—which can take many forms such as American (ASL), British (BSL), or Ghanaian (GhSL)—is linguistically and culturally reductive. Sign languages are not simplified versions of spoken languages; they have distinct grammar, spatial structure, and non-manual markers that carry grammatical meaning [26]. Stripping all of that from a response produces something functional but incomplete.

1.2.1 Directionality Gap: Who Gets Served?

Recognition has been the dominant direction in sign language technology. Surveys and system overviews foreground pipelines that detect and classify signed input—using vision-based hand tracking, glove sensors, or wearable IMU/EMG devices [25]. The same orientation appears in wearable sensor work. DeepSLR uses two armbands with IMU and multi-channel sEMG to recognize continuous Chinese Sign Language sentences and convert them into speech in real time. SignSpeaker deploys a smartwatch to recognize American Sign Language sentences and output spoken English through a paired smartphone, achieving a word error rate of around 1% on its sentence set. Other systems use smartwatches and consumer wearables to detect ASL letters or short words from wrist motion, or EMG bands to classify a fixed set of ASL hand motions and alphabet letters with high accuracy. Across these projects, the information flow is overwhelmingly from sign-to-text or sign-to-speech: they are designed to help hearing people understand signed input. This directionality gap is central to DuoSign's motivation.

Recent efforts have begun to address the generation gap. Moryossef et al.'s open-source baseline demonstrates reproducible text-to-gloss-to-pose translation for Swiss German, Swiss French, and Swiss Italian sign languages [26], and the research.sign.mt platform provides accessible resources and tools for sign language processing. These contributions represent meaningful progress toward democratizing sign language technology.

Despite this rich ecosystem, a stark asymmetry persists: most deployed systems focus on recognition (sign-to-text) rather than generation (text-to-sign). Even state-of-the-

art generators are often closed-source, difficult to reproduce, and rarely tailored to African local sign languages [26, 35].

From a user perspective, this directionality has concrete implications. In many real-world interactions—a deaf student reading course materials online, a deaf professional navigating a web-based HR[7] system, or a deaf person accessing a digital government service—the available tools make it easier to convert signing into written or spoken language than to present information back to the user in their primary signed language.

1.2.2 Intelligibility-Replicability Tradeoff

The second problem concerns how existing generation approaches trade off intelligibility, naturalness, and replicability—especially in lower-resource settings.

Early avatar systems such as eSIGN rely on linguistic notation and rule-based synthesis. Authors describe signs using HamNoSys and SiGML, encoding handshape, location, palm orientation, and movement, and a sign-language editing environment then composes these elements into avatar motion [15]. This representation is compact and language-agnostic at the notation level, and it supports flexible rendering within the manually authored domain. However, creating and maintaining a large lexicon in HamNoSys requires specialist expertise; extending coverage to new domains or sign languages is labour-intensive; and generating smooth co-articulation between consecutive signs is non-trivial. The eSIGN project explicitly restricts itself to limited, pre-scripted content and notes that fully automatic translation from arbitrary text into sign is not feasible with their approach.

Corpus-driven pipelines move generation closer to real signing. How2Sign introduces an 80-hour multimodal ASL corpus with RGB, depth, 2D keypoints for more than six million frames, and a 3D pose subset recorded in the CMU[8] Panoptic Studio [9]. Duarte et al. use these modalities to train pose-estimation and GAN-based video synthesis models, then evaluate whether ASL users can understand the resulting synthetic signing. Their user study is instructive: while participants can often identify the broad topic of a synthetic video, the skeleton overlays and GAN-generated clips are only partially intelligible at the sentence

[7] HR = Human Resources.
[8] CMU = Carnegie Mellon University.

level, with fast hand motion, co-articulation, and facial grammar cited as major sources of confusion. They conclude that current pose- and GAN-based generation does not yet support fully reliable comprehension for fluent signers.

Recent neural text-to-sign models such as CCC and Signs as Tokens underscore both the promise and the limitations of fully learned generation [42, 44]. Both papers highlight ongoing challenges: semantic-motion alignment remains fragile; high-quality generation depends on large, carefully curated corpora; and extending the approach to new sign languages is constrained by the availability and quality of 3D motion annotations.

Taken together, these works reveal a consistent "ceiling." Notation-driven avatars are linguistically explicit and relatively light-weight, but they require expensive lexicon building and are often perceived as stiff or robotic by fluent signers. Pose- and GAN-based systems grounded in real signing can achieve more natural motion, but current models still produce only partially intelligible output and are tightly coupled to large, language-specific corpora. Even where code is released, pipelines are deeply entwined with particular datasets, transcription conventions, and avatars, making them difficult to reproduce or adapt for lower-resource sign languages.

1.3 Objectives and Significance

In addressing the communication barriers faced by deaf and hard-of-hearing users, the goal is to design and build a system with the following aims:

1. Develop functionality that supports accessible, sign-first communication in common digital contexts.

2. Implement an algorithm that translates English text into American Sign Language (ASL) gloss.

3. Design friendly VRM avatars that perform sign language using natural, expressive motion.

4. Prototype a lightweight browser-based reader-assistant workflow that captures text from webpages (including user-highlighted passages, pasted text, and available video

captions such as YouTube subtitles) and renders sign-language output through a signing avatar or skeleton.

1.3.1 Contribution to Sustainable Development Goals

This project aligns with three United Nations Sustainable Development Goals (SDGs):

- **SDG 4 (Quality Education):** About 80% of deaf individuals worldwide face educational barriers due to limited access to sign-language resources [29]. DuoSign reduces this gap by enabling on-demand translation of learning materials into ASL. A deaf student in Ghana or Philadelphia can access textbook content in their primary language without relying on interpreter availability.

- **SDG 10 (Reduced Inequalities):** Hearing users often benefit from speech recognition and translation tools in their native modalities, while deaf users are commonly expected to rely on written text. DuoSign helps close this asymmetry by prioritizing text-to-sign translation rather than placing the adaptation burden on deaf users.

- **SDG 8 (Decent Work and Economic Growth):** Many workplace exclusions for deaf professionals stem from communication barriers rather than capability gaps. A browser-based tool that translates workplace emails, policies, and documentation into sign language makes inclusive employment more practical, even for organizations with limited technical budgets.

1.4 Review of Literature and Related Works

This literature review covers the foundations most relevant to DuoSign: the history of sign language technology, sign language linguistics, glosses as an intermediate representation, skeletal-pose modelling, low-resource data constraints, and related work in sign recognition and generation.

1.4.1 History of Sign Language Technology

Sign language technology has historically prioritized recognition over generation, leaving a major access gap for deaf users. Early systems in the 1980s focused on notation-driven avatars—formal systems like HamNoSys that broke signs into handshapes, locations, and movements, then tried to "play them back" through animated characters. Projects like eSIGN (2002–2008) demonstrated that this could technically work for narrow domains (government e-services in BSL, DGS, and NGT)[9], but the avatars looked robotic, required specialist linguistic expertise to author content, and couldn't automatically translate arbitrary text into signing[15]. The 2010s brought motion capture approaches, where researchers recorded real signers with optical tracking to drive avatars. These looked more natural—co-articulation and timing came for free—but building even a 1,000-sign lexicon required months of studio time and tens of thousands of dollars[8]. The 2018–present era introduced neural generation pipelines: text-to-gloss-to-pose-to-video systems trained on parallel corpora like RWTH-PHOENIX-Weather[10] and How2Sign [35, 9]. These systems hit decent automatic metrics (BLEU[11] 0.6) but struggled with comprehension—user studies showed deaf signers could identify topics but not full sentences, citing missing facial grammar and choppy motion[9]. The common thread: recognition (sign-to-text) has always gotten more attention than generation (text-to-sign), leaving deaf users with tools that help hearing people understand them but do not help them consume information in their primary language.

1.4.2 Sign Language Linguistics: Why This Is Harder Than It Looks

Sign languages are not manual versions of spoken languages. Contrary to a common but misguided belief, ASL is not "English on the hands"; it has its own grammar, spatial syntax, and phonological system realized through handshape, location, movement, palm orientation, and non-manual markers such as facial expressions, head position, and eyebrow

[9]BSL = British Sign Language; DGS = Deutsche Gebärdensprache (German Sign Language); NGT = Nederlandse Gebarentaal (Sign Language of the Netherlands).

[10]RWTH-PHOENIX-Weather is a benchmark sign-language dataset derived from German weather forecast broadcasts and commonly used in sign-language translation research.

[11]BLEU = Bilingual Evaluation Understudy, an automatic metric used to evaluate machine-generated text against reference translations.

configuration [31]. Three things make this computationally difficult.

First, simultaneity: spoken-language phonemes happen sequentially (one sound after another), but sign-language parameters happen at the same time—a sign like MOTHER in ASL combines a "5" handshape, chin location, tapping movement, and neutral facial expression all at once. Most NLP models, including transformers, are built for sequences and struggle with this [4].

Second, spatial syntax: sign languages use 3D signing space to encode grammatical relations—verbs inflect for subject and object by changing start/end locations, pronouns are realized by pointing to where referents were established earlier in the discourse (for example, signing "I" means pointing to the speaker), and entire clauses can be spatially indexed. Text-to-gloss translation cannot fully capture this because glosses are linear [26].

Third, non-manual markers (NMMs): raised eyebrows mark yes/no questions, furrowed brows mark wh-questions (information-seeking questions such as who, what, where, when, why, and how), and headshakes mark negation. These are not just emotional expressions—they are grammatical. Most sign-language datasets do not annotate NMMs at the frame level, so current generation systems either omit them or copy them blindly from source videos without checking whether they match sentence structure [21].

1.4.3 Glosses: Necessary Evil or Fundamental Limitation?

Most papers have argued against the use of glosses, questioning their effectiveness and appropriateness for generating sign-language videos. However, the core issue is that glosses are lossy. Glosses are word-level labels for signs, written in capital letters (e.g., MOTHER, GO, WHERE). They serve as an intermediate representation in many sign-language systems because they are easier to work with than raw video: you can train a neural machine-translation model to map English text to ASL glosses, then look up each gloss in a dictionary to retrieve a pose sequence. The problem is that glosses are lossy. They do not encode spatial syntax (which hand location corresponds to which discourse referent), non-manual markers (is this a question or a statement?), or classifier constructions (how multiple objects are arranged in space) [26]. Linguists have also pointed out that glossing conventions vary

across research groups, making datasets harder to combine and reducing inter-annotator consistency [26]. Recent work has tried to bypass glosses entirely—models like CCC and Signs as Tokens learn direct text-to-pose mappings using discrete motion tokenization [42, 44]—but they still struggle with semantic alignment: how do you map abstract concepts like "obligation" or "conditionality" to specific signing patterns when your training data only provides noisy sentence-level supervision? For now, glosses remain pragmatically useful but linguistically incomplete, and DuoSign uses them as a stepping stone while acknowledging their limitations.

1.4.4 Skeletal Poses as Representation

Skeletal pose sequences—2D or 3D coordinates of body joints over time—have become the de facto representation for sign language in computer vision. Tools like OpenPose and MediaPipe extract pose keypoints from RGB video in real time, providing a structured, lower-dimensional input for recognition and generation models [9]. For recognition, pose-based approaches often outperform raw-pixel models because they discard irrelevant background information and focus on body configuration, and they are computationally lighter to process. They primarily use temporal body-joint coordinates rather than full image tensors. For generation, pose sequences serve as the "skeleton" that drives avatar animation—either by directly manipulating a 3D humanoid model (inverse kinematics) or by conditioning a GAN to synthesize photorealistic video from skeletal input [35]. The catch is that pose estimation degrades in the wild: hands become unreliable during fast motion or occlusion, facial landmarks are coarse and miss subtle mouth morphemes, and 2D projections lose depth information needed to distinguish signs like `THINK` (hand at forehead) vs. `KNOW` (hand at temple) [9]. The How2Sign dataset addressed this by recording signers with depth cameras and motion capture for a subset of data, but that level of infrastructure is not available for most sign languages and is difficult to reproduce for most systems. DuoSign uses MediaPipe Tasks (formerly MediaPipe Holistic) for pose extraction because it is free, runs in the browser, and works on standard webcam footage—trading off some precision for accessibility.

1.4.5 Data Scarcity and the Low-Resource Challenge

For many African sign languages, the volume of available data is unknown as much as it is scarce. Part of this traces back to the diversity of African spoken languages—and sign languages follow the same fault lines. Just as spoken languages across the continent diverge in vocabulary and grammar, so do their signed counterparts. The sign language situation in West Africa alone has been described as "as diverse as it is undocumented" [26].

This creates a structural cycle rather than a single isolated problem. Low documentation and fragmented standards make corpus creation difficult; without corpora, tool development remains slow and expensive; without usable tools, deaf communities stay digitally marginalized; and that marginalization weakens the institutional and funding support needed to build new datasets.

Figure 1.1 visualizes this cycle and marks DuoSign's intended intervention point.

This problem is reinforced by linguistic fragmentation. A sign in one language can mean something entirely different in another, and the literature describes these as "false friends"—signs that share form but diverge in meaning across sign languages [26]. A gesture that means `lonely` in ASL, for instance, is routinely misread as "be quiet" because it visually resembles a common hushing gesture [26]. The same dynamic applies across national sign languages: a sign in BSL may carry a different or even opposite meaning in Korean Sign Language, not because either community made an error, but because each language evolved independently within its own cultural context. This makes dataset transfer and model reuse harder, which increases the cost of building new resources from scratch.

The problem is also structurally uneven across regions. In Europe and a few better-resourced contexts, a small set of language, e.g. German Sign Language (DGS), benefit from large annotated corpora, detailed gloss transcriptions, and institutional backing. The RWTH-PHOENIX-Weather dataset (8,257 DGS sentences from weather broadcasts) and How2Sign (80+ hours of ASL instructional videos) represent years of coordinated effort involving linguists, native signers, and motion capture studios [26, 9]. Meanwhile, most of the world's 200–300 sign languages have no computational datasets at all, including virtually

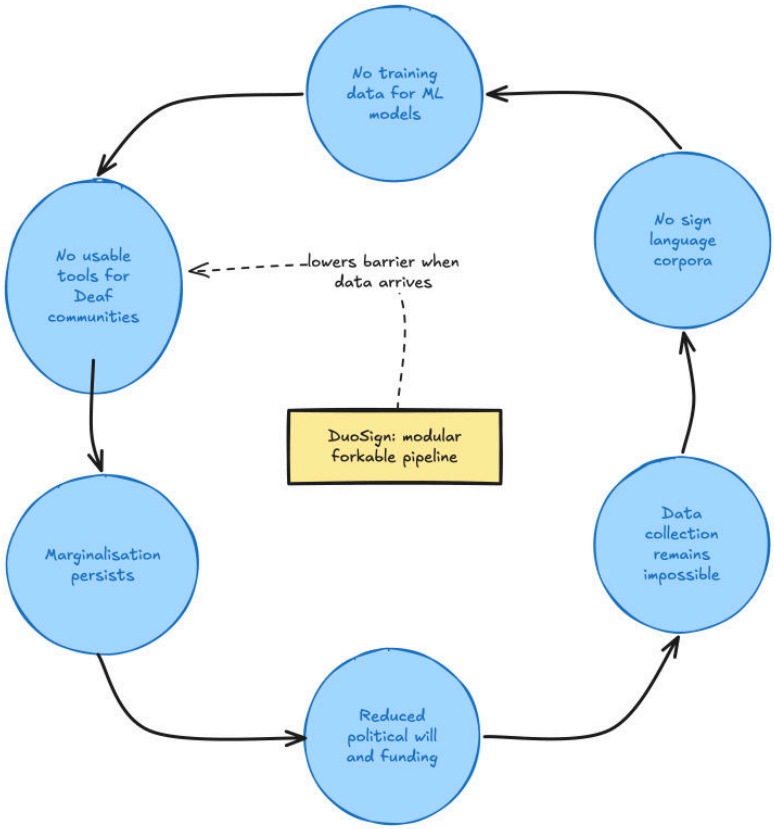


Figure 1.1: The vicious cycle of structural barriers to sign language tool development in Africa. Arrows represent causal dependencies; the dashed line indicates DuoSign's intended intervention point.

all African, South Asian, and Southeast Asian sign languages.

Ghanaian Sign Language (GhSL), used by roughly 10,000 deaf Ghanaians—though community estimates place the broader deaf population at around 500,000—has no publicly available corpus, no standardized glossing conventions, and minimal documentation beyond print dictionaries [29]. The same is true for Kenyan Sign Language (KeSL), Nigerian Sign Language (NSL), and dozens of others. Building a corpus requires institutional partnerships with Deaf community organizations, trained linguists, native signers willing to be recorded, ethical review protocols, and long-term maintenance infrastructure. In low-resource settings, these prerequisites are difficult to assemble at once, so the absence of data persists and downstream tool development remains stalled.

Taken together, these barriers limit tool development, reinforce digital marginalization, and suppress the institutional support needed for new data collection. DuoSign does not solve those structural constraints on its own—it cannot create African sign language corpora out of thin air—but it targets a practical leverage point: lowering implementation cost when data does become available. By providing a reproducible, modular pipeline, African researchers can adapt an existing system instead of rebuilding the full stack from scratch.

1.4.6 Related Works: Recognition vs. Generation

1.4.6.1 Recognition Systems: The Dominant Direction

Most sign language research still targets recognition (sign-to-text), not generation (text-to-sign). Vision-based systems use RGB cameras and pose estimation (OpenPose, MediaPipe) to recognize isolated signs or continuous sentences [25]. Wearable systems use IMU sensors and EMG armbands to classify signs from wrist motion and muscle activity. For example, SignSpeaker on a smartwatch reports 99% accuracy on continuous ASL sentences, and a Myo-based setup distinguishes 80 ASL signs at 96% accuracy [18, 14]. These systems help hearing users understand signers, but they do little to help deaf users consume information in their primary language. This recognition-first pattern is partly driven by funding priorities (speech technology has a clearer commercial market), easier evaluation (word error rate is straightforward), and a persistent assumption that spoken language is

the "default." For many deaf users, however, the urgent direction is text-to-sign: reading textbooks, navigating workplace portals, or accessing online video content without full caption support.

1.4.6.2 Generation Systems: Four Decades of Avatars

Text-to-sign generation has evolved across roughly three phases. In the notation-driven era (1980s–2010), systems like HamNoSys encoded signs as symbolic parameters and animated avatars by "playing back" those symbols. The eSIGN project used this model for government e-services in BSL, DGS, and NGT, but outputs often looked robotic, required specialist authoring expertise, and did not generalize to arbitrary text input [15]. In the motion-capture phase (2010–2018), researchers recorded real signers with optical tracking to drive 3D models, which improved naturalness but made lexicon growth expensive and slow (even a 1,000-sign lexicon could take months) [8]. The current neural phase (2018–present) uses text-to-gloss-to-pose-to-video pipelines trained on corpora such as RWTH-PHOENIX and How2Sign [35, 9]. These systems can report reasonable BLEU scores (around 0.6), yet user studies still show limited sentence-level comprehension, often due to weak facial grammar and uneven motion quality [9]. More recent gloss-free approaches (CCC, Signs as Tokens) learn motion codes directly from text and improve some automatic metrics, but non-manual markers remain a persistent gap [42, 44]. In short, technical performance has improved, but linguistic completeness remains the harder problem.

1.4.6.3 Where DuoSign Fits

DuoSign occupies a middle ground between research-grade prototypes and practically deployable accessibility tools. It is designed as a browser-based, openly inspectable baseline that prioritises sign generation, accessibility, and reproducibility over benchmark-driven complexity.

Most importantly, whereas systems such as SignSpeaker primarily optimise recognition, DuoSign prioritises the reverse direction: presenting information back to deaf users in a signed modality rather than only translating their input for hearing audiences.

The current system has clear limitations: it does not yet provide full grammatical facial-expression modelling, it has limited support for complex two-handed spatial constructions, and it inherits vocabulary constraints from the 2,000-word-class WLASL corpus (11,980 video clips in the processed subset). Even so, DuoSign is intentionally designed as a modular and reproducible plug-and-play baseline for African deployment contexts where motion-capture infrastructure and large curated corpora are often unavailable. The long-term objective is not to outperform data-intensive benchmark models, but to provide a practical foundation that can be incrementally localised—most importantly toward Ghanaian Sign Language (GhSL)—by researchers and community developers without rebuilding the entire stack.

1.4.6.4 Analysis of Related Works

A comparative matrix of representative sign-language systems is provided in Appendix D (Table 4). The key pattern is consistent: highly natural systems typically require expensive data or compute, while easily deployable systems tend to compromise naturalness or linguistic coverage. DuoSign is positioned as a reproducible middle ground focused on text-to-sign generation for practical deployment.

Key Observations

- **Directionality gap:** Recognition systems still dominate deployment, while generation systems remain less common and less accessible.
- **Infrastructure tradeoff:** The most natural outputs generally come from motion capture or GPU-heavy pipelines, which are hard to reproduce in low-resource settings.
- **Replicability constraint:** Open implementations are increasing, but many are tightly coupled to specific corpora and annotation schemes.
- **DuoSign's position:** DuoSign prioritizes reproducibility and portability, accepting moderate quality constraints to enable practical text-to-sign access on consumer hardware.

Chapter 2: Requirements

This chapter focuses on the stakeholder identification, requirements gathering, and analysis phase of DuoSign. It examines what target users need, how they expect to interact with the system, and where existing sign language technologies still fall short. Using use-case analysis, the chapter establishes the foundation for a system architecture and design that address these needs and support the project's accessibility goals.

2.1 Requirements Elicitation

Sommerville [34] identifies three main techniques for requirements elicitation: talking to stakeholders directly, observing existing systems in use, and reading existing documents and literature. For DuoSign, the third approach was used, drawing on published research, dataset documentation, and established accessibility standards.

2.1.1 Literature Review

Requirements were gathered by reviewing prior work on text-to-sign language generation. We examined eSIGN [15], Text2Sign [35], and Sign.MT [26][1] to identify what already works, where current systems fall short, and which user needs remain unmet. Four gaps stood out: many systems relied on synthetic or notation-driven animation instead of real video; no single system combined voice input, video export, and browser-based rendering; and existing systems were not designed for an African deployment context: they depend on large language-specific corpora, stable high-bandwidth infrastructure, and compute resources that are often unavailable in low-resource settings. These gaps directly informed what DuoSign set out to implement.

[1] MT = Machine Translation.

2.1.2 Secondary Data from Existing User Studies

No direct interviews with deaf users were conducted for this project. Instead, user requirements were derived from published studies and dataset documentation. The WLASL dataset [23] includes recordings from real deaf signers, so its signs reflect real usage. ASL-LEX [33][2] documents how signs are used in everyday ASL and informed the synonym lookup strategy in the gloss lexicon. In both datasets, the deaf community contributed to the source research. This project therefore builds on that input indirectly.

2.1.3 Accessibility Standards

Non-functional accessibility requirements came from the Web Content Accessibility Guidelines (WCAG[3]) 2.1 Level AA[4], which specifies colour contrast ratios, minimum touch target sizes, and keyboard navigability. These guidelines represent decades of research into what disabled users actually need. Applying them to the interface directly meant there was no need to rediscover those requirements from scratch through fresh user testing.

2.1.4 Limitations and Future Evaluation

The main limitation of this approach is the lack of direct consultation with deaf or hard-of-hearing users. No interviews, surveys, or usability tests were run. As a result, some choices, such as the default avatar, preferred playback speed, and level of visual detail, are based on literature-backed assumptions rather than direct user testing. A dedicated usability study with deaf participants is the next step.

2.2 Key Stakeholders

DuoSign is engineered to target a broad and diverse user base spanning deaf communities, educational institutions, and the research ecosystem. Among the many potential

[2]ASL-LEX = American Sign Language Lexicon Video Dataset.
[3]WCAG = Web Content Accessibility Guidelines.
[4]AA = Mid-tier conformance level in WCAG 2.1 (above A, below AAA).

stakeholders, the following groups were identified as key informants for requirements elicitation:

- **Deaf and Hard-of-Hearing Individuals:** The primary beneficiaries of the system. They need access to written content rendered in their primary visual language rather than plain text. DuoSign allows them to receive information in ASL rather than having to adapt to English grammar for every interaction. For authenticated users, the ability to export signed animations as downloadable MP4 videos further enables offline access and sharing of translated content.
- **Hearing Individuals and Content Creators:** People who interact with deaf individuals and want to communicate in a more linguistically respectful form than plain text, without requiring sign language training. This group also includes content creators, such as educators, social media producers, and accessibility officers, who want to produce signed video directly from written material without commissioning separate interpreter recordings.
- **ASL Learners and Students:** People learning ASL who need an interactive, on-demand tool for understanding how English maps to ASL grammar. Mainstream language platforms support many spoken languages, but equivalent large-scale support for ASL is still limited. DuoSign addresses this with gloss token breakdowns, skeleton visualisation, and adjustable playback speed so learners can study signs at their own pace.
- **Public Institutions and Accessibility Offices:** Schools, hospitals, universities, and government offices that need scalable tools for making public-facing written content accessible to deaf communities without relying entirely on the availability of human interpreters. The system's REST[5] API and video export capability enable institutions to integrate sign language generation into existing workflows programmatically.
- **Researchers and Developers:** Researchers who need a reproducible, open baseline to benchmark new translation approaches against, and developers who need an API for

[5]REST = Representational State Transfer.

adding sign language output to existing applications without building the full pipeline themselves. DuoSign exposes RESTful endpoints for translation, pose data retrieval, audio transcription, and vocabulary lookup, enabling programmatic integration and downstream experimentation.

2.3 Use Case Analysis

Use-case diagrams model the behaviour of a system and help to understand and capture its requirements. They describe the high-level functions and scope of a system and illustrate how actors interact with the system rather than how the system operates internally [19]. DuoSign's components are modelled with a single use-case diagram to provide an overview of how actors use and interact with the system. See Figure 2.1.

2.3.1 End-Users

The system's end-users are the individuals who directly interact with and use the system regularly. The user roles and their interactions are as follows:

Guest User

- Input English text (up to 500 characters) and receive a corresponding ASL avatar animation.
- Use voice input via microphone as an alternative to typed text, with clear visual feedback indicating listening, idle, and error states.
- Guest users may translate up to five words or gloss tokens before being prompted to log in or create an account.
- View real-time sign rendering with the gloss token breakdown displayed alongside the avatar.
- Switch between display modes (3D avatar, skeleton overlay, or hidden) to examine pose data at different levels of abstraction.

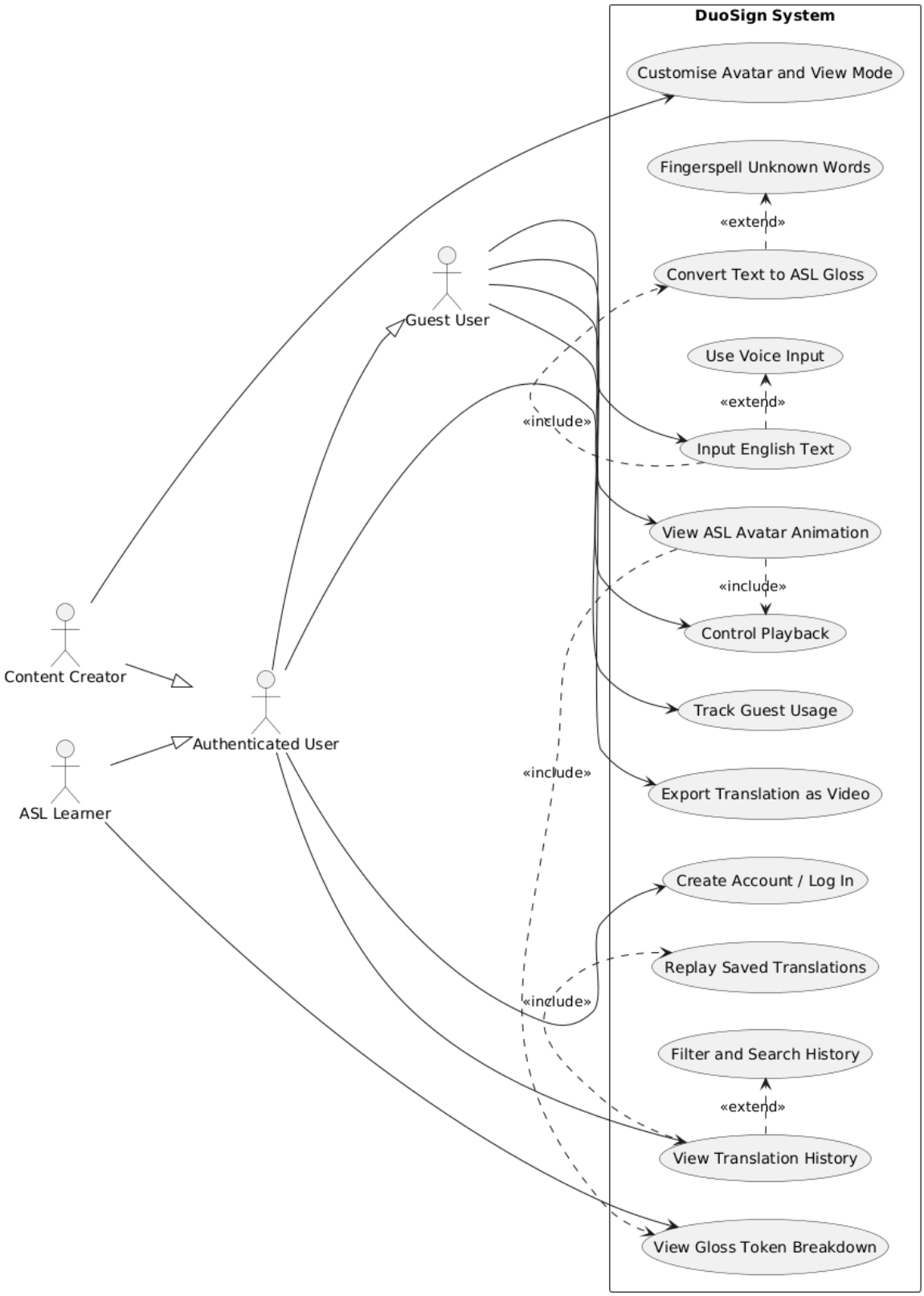


Figure 2.1: Use-Case Diagram of the DuoSign System

- Select from multiple avatar models and choose between close-up, full-body, and 3D world view modes.
- Control playback with pause, resume, previous/next token navigation, replay, and adjustable speed (0.5×, 1×, 1.5×, 2×).
- Access the system from any modern browser without installing additional software or creating an account, subject to a configurable per-session translation limit.
- Guest users cannot export MP4 videos and cannot access translation history actions (search, filter, replay from history, re-edit, or delete saved entries).

Authenticated User

- All capabilities available to Guest Users, without a translation limit.
- Export any translation, whether current or from history, as a downloadable MP4 video of the avatar performing the sign sequence.
- Create an account and log in to unlock persistent access once the guest usage threshold is reached, without blocking translation during the registration prompt.
- View previously translated phrases in a searchable, filterable translation history, grouped by date and annotated with input type (typed, voiced).
- Replay, re-edit, or delete any saved history entry.

Content Creator

- Translate written English content into ASL avatar animations directly from the web application.
- Export signed animations as MP4 video files for embedding in presentations, social media posts, instructional materials, or websites, without commissioning professional interpreter video production.
- Customise avatar appearance and view mode to produce output that fits the visual style of their content.

- Verify accessibility of written content by checking its signed rendering before publication.

ASL Learner

- View gloss token breakdowns alongside avatar animations to understand the English-to-ASL grammatical mapping sign by sign.
- Switch to skeleton visualisation mode to observe raw pose landmarks without avatar rendering, aiding understanding of body and hand positioning.
- Replay saved translations at reduced speed (0.5×) to study individual signs in detail.
- Navigate token-by-token through a signed sequence using previous and next controls to isolate specific signs.
- Build a personal study library using the translation history, returning to previously translated phrases for repeated practice.

2.3.2 System Workflows

The use-case diagram establishes *what* actors can do, while activity diagrams show *how* those interactions unfold in practice. Activity diagrams capture the full workflow lifecycle, including decision points, parallel paths, and failure branches, at a level of detail that use cases alone do not provide [19]. The three activity diagrams below model DuoSign's principal workflows: core translation and playback, video export, and history management.

2.3.2.1 Core Translation Workflow

Figure 2.2 models the end-to-end translation flow from the moment a user provides input to the point at which the avatar animation completes and the result is persisted to history.[6] The diagram covers both typed and voice input paths, the two-phase streaming response in which the rule-based result is emitted immediately followed by an optional LLM quality-improvement event, the three-way animation source resolution order (WLASL

[6]For a high-resolution version of this diagram, click here.

video, pose binary, or fingerspelling fallback), and the configurable guest usage threshold that triggers a non-blocking registration prompt without interrupting the translation in progress.

2.3.2.2 Video Export Workflow

Figure 2.3 models the export pipeline available to authenticated users. A browser-compatibility fork determines the capture mode: real-time WebM recording via the `MediaRecorder` API on supported browsers, or a JPEG[7] frame-array fallback for environments such as Safari that do not implement the `MediaRecorder` interface. Both paths converge at the backend, where FFmpeg encodes the captured data into a downloadable H.264 MP4 file at 1280×720 resolution.

2.3.2.3 History and Replay Workflow

Figure 2.4 models the lifecycle of a translation history entry from creation to deletion.[8] Entries are appended to browser-local storage on every successful translation, subject to a 100-entry FIFO eviction policy. The diagram also shows the four actions an authenticated user may perform on any saved entry: replay, export, re-edit, and delete. The replay mechanism uses URL[9] parameters to open the translation page with pre-filled text and autoplay enabled. This keeps the history and translation pages linked without requiring shared server-side session state.

2.3.3 Behavioural Model

Activity diagrams model workflows across actors, while state machine diagrams model the lifecycle of a single system object. Harel's statechart formalism [16], standardised in UML[10] 2.5.1, fits event-driven components with explicit transition guards, which matches the behaviour of a media playback controller. Figure 2.5 models the DuoSign avatar animation player, including its four states (*Idle*, *Playing*, *Paused*, *Complete*), the guard conditions

[7] JPEG = Joint Photographic Experts Group image format.
[8] For a high-resolution version of the translation history diagram, click here.
[9] URL = Uniform Resource Locator.
[10] UML = Unified Modeling Language.

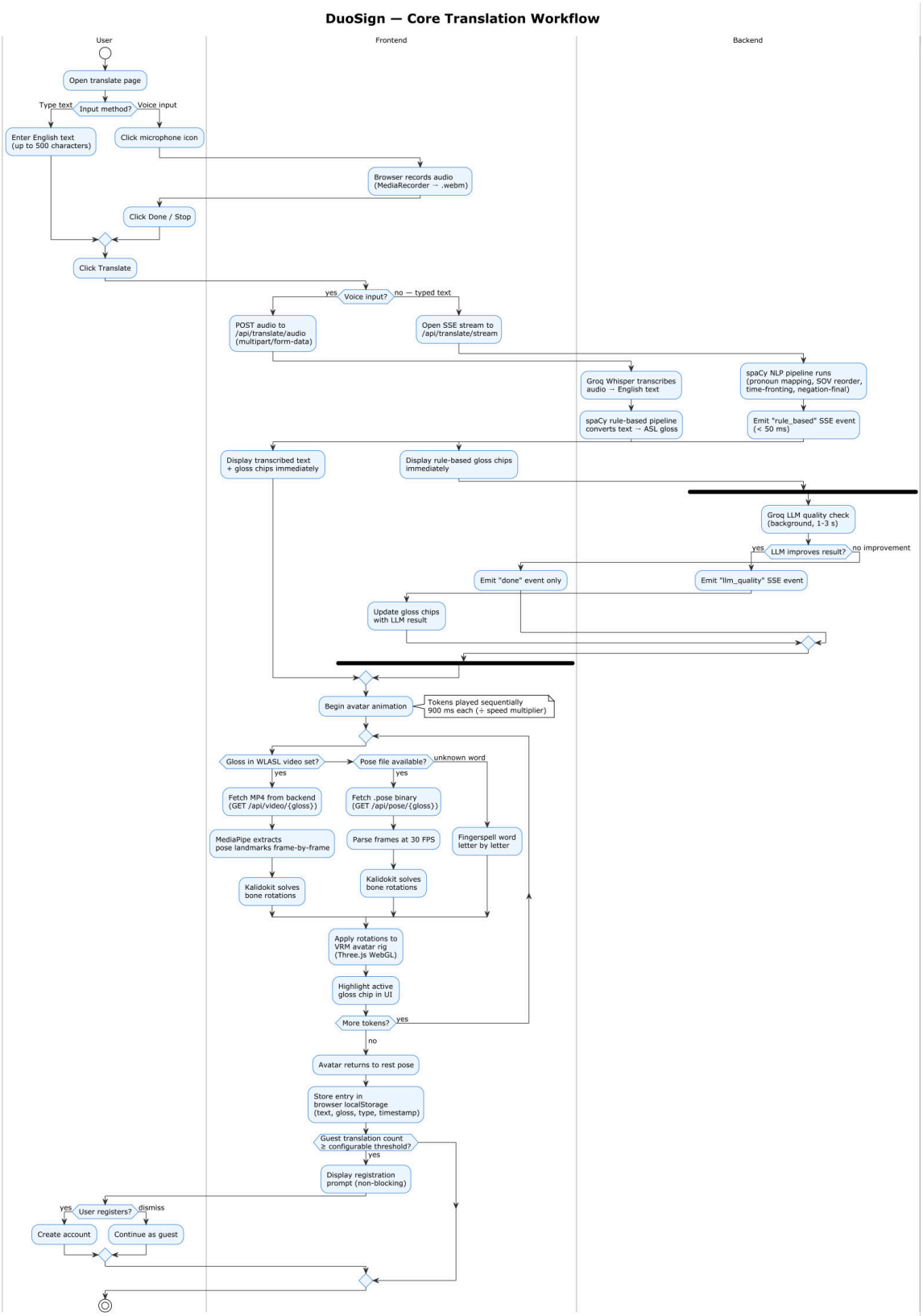


Figure 2.2: Activity Diagram: Core Translation and Playback Workflow

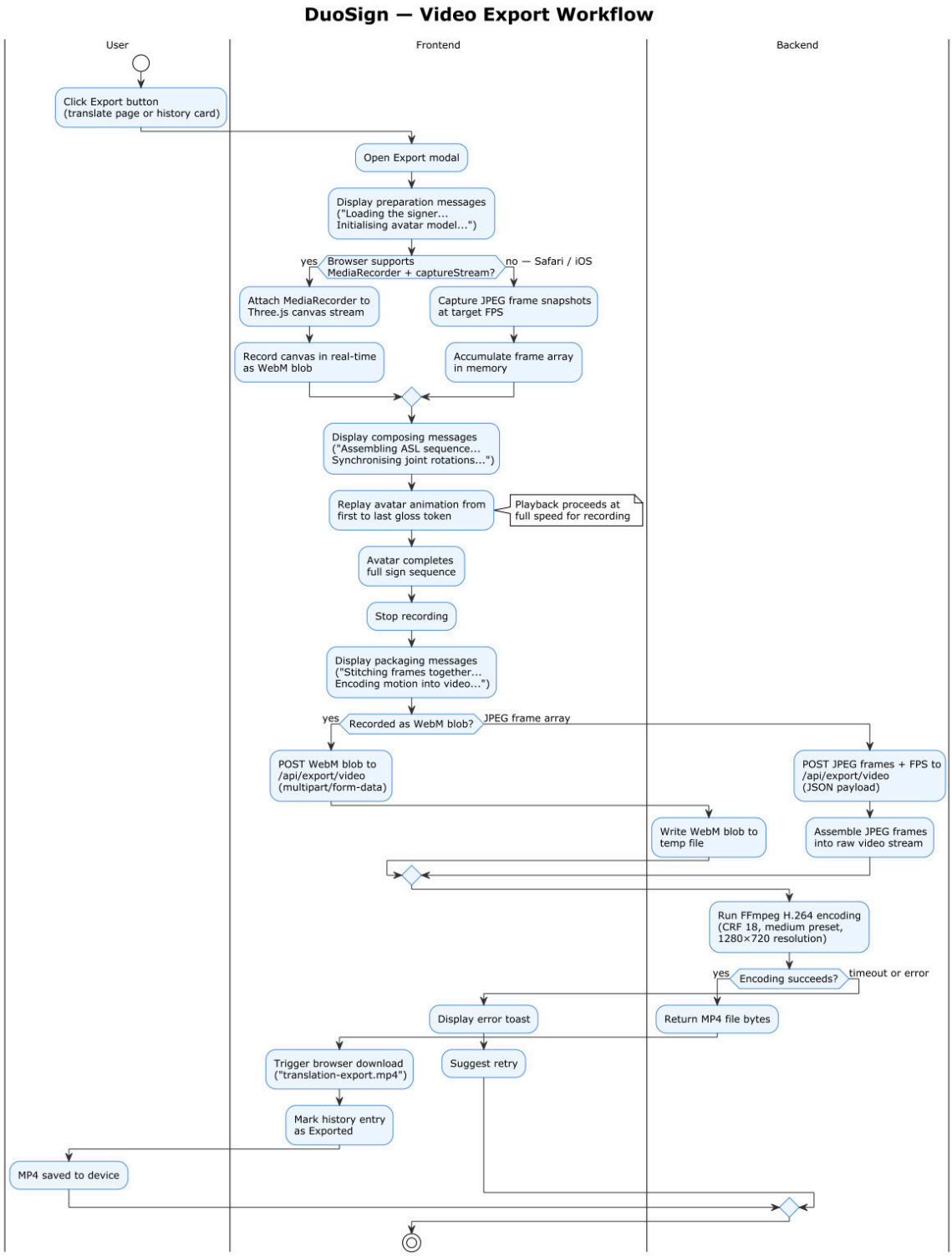


Figure 2.3: Activity Diagram: Video Export Workflow

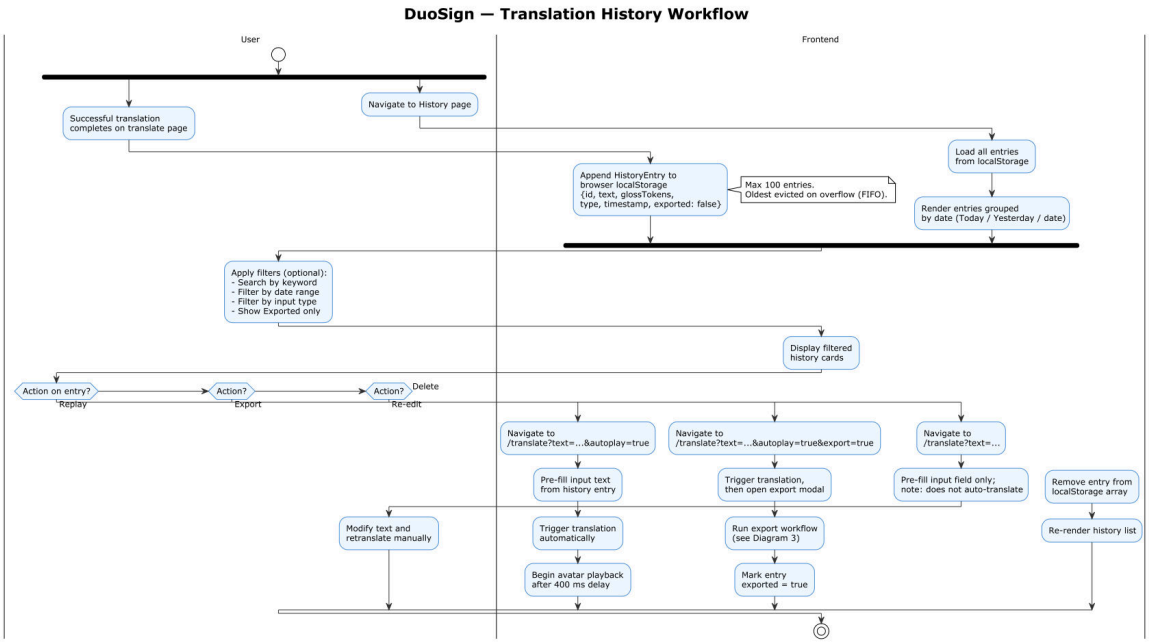


Figure 2.4: Activity Diagram: Translation History and Replay Workflow

for each transition, and the speed multiplier that sets per-token delay.[11]

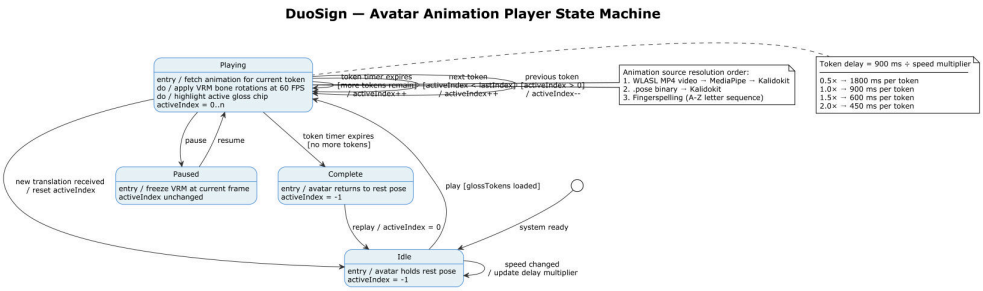


Figure 2.5: State Machine Diagram: Avatar Animation Playback Player

2.4 System Requirements

The 1990 IEEE[12] Standard Glossary of Software Engineering Terminology defines a requirement as (1) a condition or capability needed by a user to solve a problem or achieve an objective; (2) a condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification, or other formally imposed document; and (3) a documented representation of a condition or capability as in (1) or (2) [1]. From our engagement with stakeholders and analysis of the problem space, we gathered requirements

[11] For a high-resolution version of the state diagram, click here.
[12] IEEE = Institute of Electrical and Electronics Engineers.

that were meticulously ranked by order of priority for implementation.

2.4.1 Functional Requirements

[FR001] – The system shall accept English text input of up to 500 characters and produce a corresponding ASL avatar animation.

[FR002] – The system shall support voice input via two complementary paths: browser-based speech recognition using the Web Speech API for real-time transcription, and server-side transcription using the Groq Whisper API for uploaded audio files, supporting WebM, MP3, WAV, M4A, OGG, and MP4 formats up to 25 MB[13].

[FR003] – The system shall convert English text into ASL gloss notation using a spaCy-based rule pipeline that handles pronoun substitution, time-fronting, subject-object-verb reordering, and negation-final constructs, with an LLM fallback for edge cases the rule pipeline cannot resolve.

[FR004] – The system shall expose translation results through a streaming Server-Sent Events (SSE) endpoint that emits an immediate rule-based result, followed by an optional LLM refinement event, so the frontend can update results progressively without blocking the user.

[FR005] – The system shall render sign animations using a VRM-format 3D avatar driven by pose data extracted from real WLASL sign language videos via MediaPipe Tasks and mapped to bone rotations using Kalidokit.

[FR006] – The system shall fall back to fingerspelling for any word not found in the gloss lexicon or its synonyms by expanding the word character by character, ensuring all input text produces a visual output.

[FR007] – The system shall display real-time gloss token breakdowns alongside the avatar animation to show the English-to-ASL grammatical mapping at the word level.

[FR008] – The system shall provide playback controls for the sign animation sequence, including play, pause, replay, previous token, next token, and a cyclic speed selector with 0.5×, 1×, 1.5×, and 2× playback rates.

[13] MP3 = MPEG Audio Layer III; WAV = Waveform Audio File Format; M4A = MPEG-4 Audio; OGG = Ogg container format.

[FR009] – The system shall support multiple avatar display modes: 3D VRM avatar and skeleton pose overlay.

[FR010] – The system shall support multiple camera view modes (close-up interpreter, full body, and 3D world), switchable at any time during playback.

[FR011] – The system shall offer multiple VRM avatar models and allow users to switch between avatar appearances without interrupting the sign sequence.

[FR012] – The system shall let authenticated users record the animated avatar canvas and export the sign sequence as a downloadable MP4 file, using MediaRecorder with WebM output where supported and a JPEG frame-array fallback for incompatible browsers, followed by server-side FFmpeg conversion to H.264 MP4.

[FR013] – The system shall allow users to create accounts and log in after reaching a configurable guest usage threshold, without blocking the translation in progress during the registration prompt.

[FR014] – Registered users shall be able to view, search, filter, replay, export, re-edit, and delete previously translated phrases from a persistent translation history grouped by date and filterable by input type and date range; guest users shall not have access to these history-management actions.

[FR015] – The system shall track guest usage through a session-based browser identifier and prompt registration at a configurable threshold without interrupting the translation in progress.

[FR016] – The backend shall expose RESTful API endpoints for translation (rule-based, streaming, and audio), gloss lexicon lookup and search, pose and video-file serving, video export conversion, and health checks.

[FR017] – The system shall normalise pose landmarks extracted from WLASL videos to correct for inconsistent signer framing across the dataset.

2.4.2 Non-Functional Requirements

[NFR001] – *Usability:* All core actions shall be reachable within two steps from the main screen. During playback, the word currently being signed shall be highlighted so users can

follow along without losing their place. The system shall keep users informed of what is happening at every stage, including during long-running operations such as video export.

[NFR002] – *Responsiveness:* The system shall return an initial translation result within 300 milliseconds under normal network conditions. The avatar animation shall begin within one second of the translation being received. The user shall not be blocked or prevented from interacting with the system while a background quality-improvement step is running. Video downloads shall be ready within a time proportional to the length of the signed sequence.

[NFR003] – *Accessibility:* The interface shall follow established web accessibility guidelines, including sufficient colour contrast, adequately sized touch targets for mobile use, full keyboard navigation, and a readable text summary of the signed output for users who cannot view the animation. The avatar display shall include an option to show only the skeletal pose, and an option to hide the animation entirely so users who only need the text output are not forced to load rendering components.

[NFR004] – *Voice Input Feedback:* The microphone feature shall clearly communicate to the user whether it is actively listening, waiting, or has encountered a problem, so users always know what the microphone is doing.

[NFR005] – *Performance:* The system shall handle multiple simultaneous users without a noticeable drop in translation speed. The avatar animation shall play back smoothly without stuttering or frame drops. Exported videos shall be produced at a quality suitable for sharing or embedding in other materials.

[NFR006] – *Security:* User account passwords shall be stored in a secure, irreversible form. Login sessions shall be managed using tokens that cannot be read or modified by browser scripts or third-party page content. The system shall enforce limits on how frequently any single user can make requests, to protect against misuse. Audio and image data submitted for processing shall be discarded by the server as soon as the request is complete.

[NFR007] – *Scalability:* Each major component of the system (the translation engine, animation renderer, and sign vocabulary) shall be replaceable independently without requiring changes to the rest of the system. The server-side API shall act as the single point of integration for future client surfaces, such as a mobile application or browser extension.

[NFR008] – *Maintainability:* The server-side code shall separate concerns into distinct layers so that changes to one layer do not require changes to the others. The client-side code shall be organised by feature so that individual parts of the interface can be modified without affecting unrelated parts.

[NFR009] – *Reliability:* The system shall always produce a visual output for any input text, even when the exact sign is not in the vocabulary. If the primary translation pathway fails, the system shall automatically attempt a simpler fallback before showing an error to the user. If the preferred animation source for a sign is unavailable, the system shall try alternative sources before spelling the word out letter by letter.

[NFR010] – *Deployability:* All components of the system shall be deployable using free hosting plans so that the project can be demonstrated without incurring any infrastructure costs.

[NFR011] – *Browser Compatibility:* The system shall work correctly on all major modern web browsers. Any feature that is not supported in a particular browser shall have a working alternative, so that no user loses access to core functionality based on their choice of browser.

[NFR012] – *Portability:* The system shall run entirely within the user's web browser. No software installation, browser plugin, or device-specific hardware shall be required to access any feature of the system.

[NFR013] – *Learnability:* A first-time user shall be able to perform a complete translation and view the resulting animation without reading any documentation or on-screen instructions.

[NFR014] – *Error Recoverability:* When any part of the system encounters a problem, it shall display a plain, non-technical message and give the user a clear way to try again, without requiring a full page reload or loss of previously entered text.

[NFR015] – *Transparency:* The system shall never present a translation as a black box. The sequence of sign tokens derived from the input shall always be shown alongside the animation so users can read, understand, and verify what the avatar is signing.

[NFR016] – *Offline Resilience:* The core translation feature shall continue to function if external AI[14] services are temporarily unavailable, by falling back to the built-in rule-based

[14] AI = Artificial Intelligence.

translation pathway without displaying an error to the user.

[NFR017] – *Privacy by Design:* The system shall collect only the minimum data required for its operation. No user input, audio, or image data shall be stored on the server after the corresponding request has been completed.

[NFR018] – *Extensibility:* The system architecture shall not make assumptions specific to American Sign Language. The sign vocabulary, translation rules, and avatar motion data shall each be replaceable to support future adaptation to other sign languages, including Ghanaian Sign Language.

[NFR019] – *Content Authenticity:* All avatar animations shall be derived from recordings of real human signers. The system shall not use computer-generated or fabricated sign motions in place of recordings from real signers.

[NFR020] – *Cultural Appropriateness:* The default avatar shall suit the cultural context of the users this system was built for. Users shall be able to choose from avatars with different skin tones and clothing styles so that people from different backgrounds can find one that feels like them.

2.4.3 In-Scope and Out-of-Scope Requirements

Table 2.1 defines the implementation boundary for this project iteration. Items marked out-of-scope are recognised requirements but were intentionally deferred.

2.4.4 Requirements Traceability Matrix

Table 2.2 maps high-priority requirements to implementation evidence in this dissertation and system artifacts.

2.4.5 Acceptance Criteria for Critical Requirements

The following measurable acceptance criteria are used to verify the most critical requirements.

- **AC1 (FR001):** Given English input of 500 characters or fewer, the system returns a

Table 2.1: In-scope and out-of-scope requirement boundary for this project

Item	Status	Description	Reason / Timing
FR001–FR017	In Scope	Core translation pipeline, rendering, playback, export, history, guest and authenticated flows, and backend API surface.	Delivered in this project.
NFR001–NFR020	In Scope (Targeted)	Usability, responsiveness, accessibility, reliability, security, extensibility, and deployment constraints defined for this build.	Addressed as design and implementation targets.
Browser Extension	Out of Scope	Chrome extension integration for translating selected text outside the main web application.	Deferred to future work due to timeline constraints.
GhSL Localisation	Out of Scope	Ghanaian Sign Language support (lexicon, motion corpus, and rule adaptation).	Deferred pending dedicated language resources and validation.
Direct User Study	Out of Scope (Current Iteration)	Interviews and usability testing with deaf and hard-of-hearing participants.	Deferred to post-implementation evaluation phase.

Table 2.2: Requirements traceability matrix

Req ID	Description	Priority	Implemented?	Evidence (section/figure/API)
FR001	Accept text input and produce ASL avatar animation.	High	Yes	Use-case and core workflow (Sections 2.3, 2.3.2); Fig. 2.2; translation API endpoints (rule + streaming).
FR002	Dual-path voice input (Web Speech + server transcription).	High	Yes	End-user use cases and workflow narrative; audio transcription endpoint in backend API set (FR016).
FR004	Streaming SSE result with rule output and optional LLM refinement.	High	Yes	Core translation workflow description; Fig. 2.2; streaming translation endpoint.
FR005	Real-video-grounded avatar animation pipeline.	High	Yes	Proposed system description (Section 2.5); comparison table (Table 3).
FR008	Playback controls (pause/replay/token nav/speed).	High	Yes	End-user capabilities and playback state behaviour; Fig. 2.5.
FR012	Authenticated MP4 export with browser fallback modes.	High	Yes	Video export workflow; Fig. 2.3; export conversion endpoint (FR016).
FR014	Persistent history with replay/export/re-edit/delete for authenticated users.	High	Yes	History and replay workflow; Fig. 2.4; authenticated user use cases.
FR016	RESTful API for translation, lookup, serving, export, and health.	High	Yes	Functional requirement definition; system integration narrative in proposed system section.
NFR002	Responsive translation and playback startup behaviour.	High	Partially	Defined in NFR list; evidenced by two-phase response workflow; formal benchmark suite planned as future validation.
NFR003	Accessibility compliance (contrast, keyboard, touch targets, text support).	High	Partially	Elicited from WCAG in Section 2.1; reflected in end-user interface requirements; full audit scheduled for evaluation stage.
NFR006	Security controls for credentials, sessions, rate limiting, data retention.	High	Yes	Security requirement definition; authenticated flow and API boundary design in requirements and workflow sections.
NFR009	Reliable fallback behaviour for unknown words and source failures.	High	Yes	FR006 fallback logic; core translation workflow decision branches; history of output continuity constraints.

non-empty gloss sequence and starts avatar playback.

- **AC2 (FR002):** A user can trigger translation through either live microphone input or uploaded audio, and both paths produce transcribed text sent to the translation pipeline.
- **AC3 (FR004):** For streaming translation, the frontend receives an initial rule-based event before any optional LLM refinement event.
- **AC4 (FR006):** When a token is missing from the glossary and synonym set, the system outputs a fingerspelled sequence instead of an empty result.
- **AC5 (FR008):** Playback controls allow pause/resume, replay, previous/next token navigation, and speed switching across 0.5×, 1×, 1.5×, and 2×.
- **AC6 (FR012):** An authenticated user can export a played sequence and download an H.264 MP4 file for that sequence.
- **AC7 (FR014):** An authenticated user can search, filter, replay, re-edit, export, and delete saved history items; guest users cannot access these actions.
- **AC8 (NFR002):** Avatar playback begins within one second after translation output is received by the client.
- **AC9 (NFR006):** Passwords are never stored in plain text, authenticated sessions use protected tokens, and request rate limits are enforced at the API boundary.
- **AC10 (NFR009):** For any valid user input, the system either signs a known token or falls back to an alternative path (synonym/fingerspelling), preventing empty visual output.

2.5 DuoSign: Proposed System

From the system requirements gathered, the proposed system is a browser-based text-to-sign pipeline that accepts typed or spoken English, converts it to ASL gloss, and presents the result through an animated avatar. The detailed architectural rationale, technology choices, and deployment design are discussed in Chapter 3 (Section 3.1).

At the requirements stage, the key point is not the individual framework choices but the system shape: DuoSign separates language processing, animation, and media delivery so that each part can evolve independently. This modularity supports maintainability, extensibility, and future localisation to other signed languages while keeping the user-facing workflow simple. An extended comparison with representative prior systems is provided in Appendix C (Table 3).

2.6 Ethics

The design and implementation of DuoSign raise several ethical considerations that have been carefully addressed throughout the project:

Data Privacy. User data should not be retained beyond the immediate request scope unless the user has explicitly chosen to store it. Privacy is therefore treated as a design principle that constrains system behaviour rather than as a purely technical afterthought.

Responsible Use of Sign Language Datasets. The WLASL dataset [23] was used in accordance with its licence and citation requirements. The dataset is acknowledged as belonging to the linguistic community that produced it, and its use in this project respects the intellectual contribution of deaf signers and sign language researchers.

Avoiding Misrepresentation. The system uses real sign language videos as the source for avatar animation rather than synthetically generated motion, grounding output in authentic signing. Limitations in gloss accuracy and non-manual marker representation are documented transparently rather than concealed. The system does not claim to produce grammatically perfect ASL output, and the displayed gloss token breakdown makes the translation's intermediate representation visible to the user rather than presenting output as a black box.

Not Replacing Human Interpreters. DuoSign is designed as a supplementary accessibility tool for contexts where no interpreter is available. It is not a replacement for qualified sign language professionals in critical settings such as healthcare or legal proceedings. This distinction is made explicit in the system's documentation and user interface.

Cultural Sensitivity. The system does not imply that ASL is interchangeable with Ghanaian

Sign Language or other regional sign languages. The modular architecture is designed to support future localisation into GhSL as a documented future work direction, acknowledging the linguistic distinctiveness of each sign language.

Chapter 3: Architecture and Design

This chapter presents the architecture and design of DuoSign, the English-to-American Sign Language (ASL) translation system built for this project. It describes the structure of the software, the major components, how they connect, and the reasoning behind each decision. Every choice made here traces back to a functional or non-functional requirement established in Chapter 2.

3.1 System Architecture

DuoSign is built around a **modular pipeline architecture** that breaks the system into independently deployable services, each responsible for a single domain: natural language processing, pose extraction, avatar animation, or user interface rendering. Services communicate only through documented REST endpoints or file-based exchanges, with no shared internal state across boundaries.

This structure fits DuoSign for concrete reasons. The translation process is sequential: English text becomes ASL gloss notation, gloss tokens map to animation data, and animation data drives the avatar. Moryossef et al. describe this text-to-gloss-to-pose-to-video flow as the standard decomposition for spoken-to-signed language systems [26], and DuoSign follows that structure. Beyond ordering, the system spans human-computer interaction, computational linguistics, computer vision, and real-time 3D graphics. Separating the Python NLP backend from the JavaScript rendering engine lets each side use tools suited to its own workload. A researcher who wants to replace the rule-based NLP module with a neural model only needs to match the API contract, and the avatar pipeline remains unchanged. This directly supports NFR007 (scalability) and NFR018 (extensibility).

One core design decision in DuoSign is splitting compute-heavy media work from linguistic processing. Natural language processing and audio transcription run server-side on FastAPI, where Python's NLP stack is strongest. Video decoding, pose landmark extraction, and avatar rendering run client-side in the browser, where MediaPipe WebAssembly[1] and

[1]WebAssembly = A low-level binary instruction format that enables near-native performance in web

Three.js WebGL[2] use the user's hardware directly. This avoids streaming raw video between client and server, reduces latency, and keeps backend load low.

3.1.1 Frontend Architecture

3.1.1.1 Feature-Sliced Design (FSD)

The frontend uses Feature-Sliced Design (FSD), which organizes code by domain responsibility rather than technical role. Instead of placing all React components in one folder and all hooks in another, FSD defines five layers: `app`, `entities`, `features`, `widgets`, and `shared`. Each layer can import only from lower layers. This rule prevents circular dependencies by design and makes feature-level changes safer and easier [40].

FSD was selected over a conventional `components/hooks/pages` layout for three DuoSign-specific reasons. First, the application spans distinct domains, including avatar animation, voice input, translation history, and NLP integration. Each domain is kept in its own `features/` slice with separate `model/` and `ui/` folders, so one feature can evolve without destabilizing others. Second, the widget layer assembles `AvatarPanel`, `InputPanel`, and `NavigationBar` into stable page-level blocks that routes can consume directly. Third, this dependency structure maps cleanly to Next.js 14 App Router, where route files stay thin and focus on composition. This structure supports FR001 (translation interface), FR009 (history view), and FR013 (settings view).

3.1.2 Backend Architecture

3.1.2.1 FastAPI NLP and Export Service

The backend is a Python FastAPI service with three core responsibilities: text-to-gloss NLP processing, server-side audio transcription through the Groq Whisper API, and video export encoding through FFmpeg. FastAPI was chosen over Flask and Django for its native `async/await` concurrency model, automatic OpenAPI[3] documentation, and direct fit

browsers.

[2]WebGL = Web Graphics Library, a browser API for hardware-accelerated 2D/3D rendering.

[3]OpenAPI = A standard specification format for describing REST APIs.

with Python NLP tooling. The service exposes focused endpoints: `POST /api/translate`, `POST /api/translate/fast`, `POST /api/translate/stream` (SSE[4]), `POST /api/translate/audi` `GET /api/vocabulary`, `GET /api/pose/{gloss}`, `GET /api/video/{gloss}`, `POST /api/export/video`, and `GET /api/health`. This backend/frontend boundary supports FR003 (text-to-gloss), FR002 (voice transcription), and FR012 (video export).

3.1.3 Detailed Architecture Using the C4 Model

This chapter uses the C4[5] model to present DuoSign's architecture. C4 is an abstraction-first approach that reflects how engineers reason about real systems [5]. It defines four levels of detail: Context (Level 1), Containers (Level 2), Components (Level 3), and Code (Level 4). DuoSign uses the first three levels, which are sufficient to communicate architecture and design decisions without dropping to line-level implementation.

3.1.3.1 System Context Diagram (Level 1)

The system context diagram is the starting point. At this highest level, DuoSign is shown as one system with its external actors and connected services. Figure 3.1 shows these relationships.

- **Deaf and Hard-of-Hearing User:** Represents end users whose primary language is ASL. These users submit English text (typed or pasted) and receive sign language animations rendered by a 3D avatar, enabling them to consume written content in their preferred visual language.
- **Hearing User:** Represents hearing individuals who interact with deaf colleagues, students, or community members. These users type responses into DuoSign and receive signed avatar output, allowing them to communicate in a linguistically respectful form without requiring sign language training.

[4] SSE = Server-Sent Events, a unidirectional HTTP streaming mechanism for incremental server updates.

[5] C4 = Context, Containers, Components, and Code; a hierarchical model for communicating software architecture.

- **ASL Learner:** Represents students and self-directed learners studying American Sign Language. These users type English sentences and receive both the ASL gloss breakdown and the corresponding avatar animation, providing immediate visual feedback on how English grammar maps to ASL structure.

- **Content Creator:** Represents educators, accessibility officers, and social media producers who use DuoSign's video export feature to generate signed MP4 content from written text, without requiring professional interpreter video production.

- **Developer and Researcher:** Represents technical users who interact with DuoSign programmatically. These users call the translation API directly, swap the NLP module for their own models following the documented API contract, extend the lexicon with new sign data, and benchmark translation quality against the rule-based baseline.

- **DuoSign:** Represents the core software system under consideration, responsible for converting English text into ASL gloss notation, mapping gloss tokens to pose animation data, rendering sign language animations through a 3D avatar in real time, recording and exporting signed video, and serving media assets on demand.

- **Web Speech API (Browser-native):** External browser-native speech recognition system used by DuoSign's primary voice input path. Transcription occurs entirely within the browser without transmitting audio to any server; the browser returns a text string that is fed directly into the translation pipeline.

- **Groq API (Server-side):** External AI inference platform hosting the `whisper-large-v3-turbo` model used for DuoSign's secondary voice input path. The FastAPI backend forwards raw audio blobs received from the client to this service for transcription and discards the audio after the resulting text is returned. This path is used when the Web Speech API is unavailable or when higher multilingual accuracy is required.

- **Groq LLM (Translation Fallback):** External LLM inference endpoint used as the text-to-gloss fallback when the rule-based NLP pipeline produces a confidence score

below the configured threshold. Invoked only for complex or idiomatic sentences that the spaCy-based pipeline cannot handle adequately.

- **FFmpeg (Bundled via imageio-ffmpeg):** Video encoding library used to convert canvas-captured WebM recordings or JPEG frame arrays into H.264[6] MP4 files. Bundled with the FastAPI backend via the `imageio-ffmpeg` package, which includes a self-contained FFmpeg binary requiring no system-level installation.

- **Supabase (Storage and Database):** External managed platform used for both storage and database services. Supabase Storage hosts sign language reference videos, precomputed pose files, and avatar models as publicly accessible objects delivered over CDN; the frontend fetches them directly over HTTPS. Supabase PostgreSQL stores user accounts, session tokens, and server-side translation history.

3.1.3.2 Container Diagram (Level 2)

Next is the container diagram, which zooms into DuoSign and shows the high-level technical building blocks. In the C4 model, a container is a separately runnable or deployable unit that executes code or stores data. See Figure 3.2. This diagram shows each container as a box with its internal details abstracted away. The containers are:

- **Next.js Frontend (Web Application):** A React-based web application built with Next.js that serves as the primary user-facing interface. It handles text input, dual-path voice input, avatar rendering using Three.js and the VRM format, client-side pose playback via `.pose` file parsing, video-driven landmarking via MediaPipe Tasks Vision (WebAssembly), canvas recording and MP4 export, and translation history management via `localStorage`. Next.js is chosen over plain React for its server-side rendering capabilities and native WebAssembly support, both required for running MediaPipe in the browser. The frontend communicates with the backend through HTTPS REST calls and fetches media assets directly from the asset bucket.

[6] H.264 = Advanced Video Coding (AVC), a widely used video compression standard.

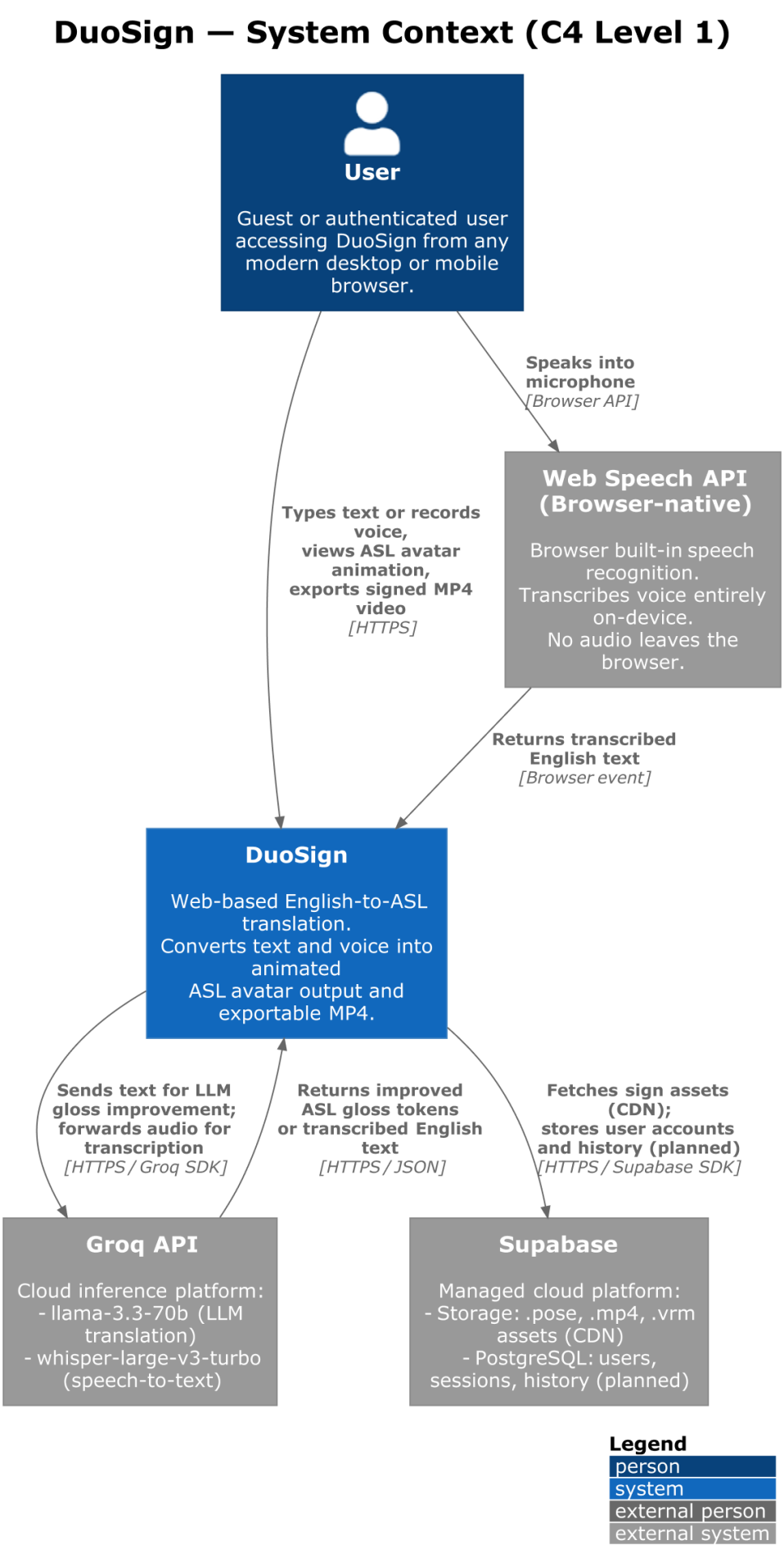


Figure 3.1: System context diagram of the DuoSign sign language translation system.

- **FastAPI Backend (API Application):** A Python-based API application responsible for text-to-gloss NLP processing, voice transcription, vocabulary serving, media asset delivery, and video export conversion. It employs FastAPI for its native async support, automatic OpenAPI documentation, and Python's NLP ecosystem. The backend reads gloss data from `glosses.tsv`, delegates audio to the Groq Whisper API, and invokes FFmpeg for MP4 conversion. In the current prototype, sign media assets are served directly from a local `bucket/` directory; in deployment, asset requests are proxied to Supabase Storage.

- **Supabase Storage (Media Asset Store):** An S3-compatible object store hosting sign language reference videos (`.mp4`), precomputed pose landmark files (`.pose`), and VRM avatar model files. In the current prototype, this is a local `bucket/` directory. In production, Supabase Storage is used: it provides 100 GB of free storage, a built-in global CDN, and a public URL per asset, without requiring a separate storage platform alongside the database.

- **Supabase PostgreSQL (Relational Database):** A managed PostgreSQL instance hosted on Supabase that stores user accounts, session tokens, guest usage counters, and translation history for registered users.

3.1.3.3 Component Diagram (Level 3)

Finally, each container is further decomposed to identify and show the internal components and how they interact with each other. See Figures 3.3 and 3.4.

Next.js Frontend Components. The frontend container is decomposed into the following components, organised by FSD layer:

- **TranslatePage / HistoryPage / SettingsPage (App Layer):** Thin Next.js App Router page files that orchestrate widget composition without containing business logic. Each addresses a corresponding section of requirements: `TranslatePage` satisfies FR001, FR002, FR007–FR011; `HistoryPage` satisfies FR014; `SettingsPage` addresses avatar and preference customisation.

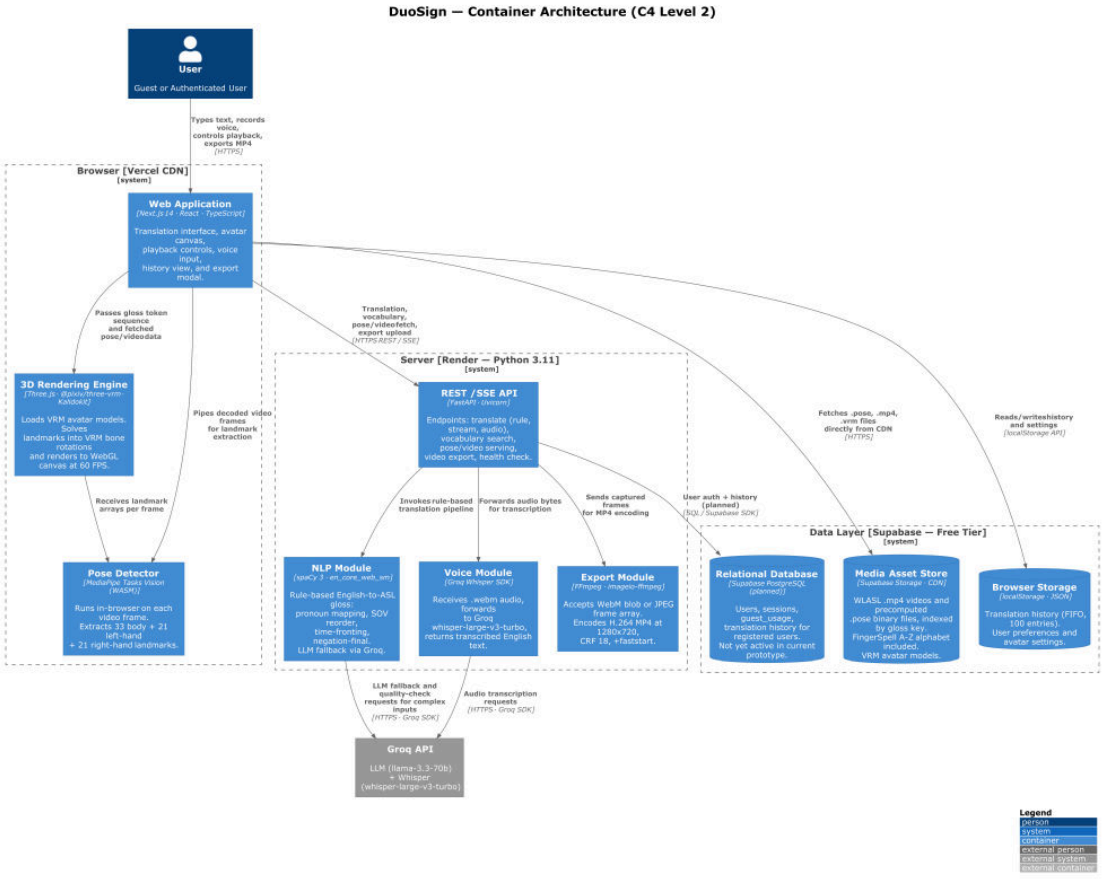


Figure 3.2: Container diagram of the DuoSign sign language translation system.

- **AvatarPanel / InputPanel / NavigationBar (Widget Layer):** Composite UI blocks that assemble feature slices into page-level sections. `AvatarPanel` integrates the VRM canvas, playback controls, avatar switcher, view mode selector, skeleton toggle, and export button into one cohesive surface.

- **useTranslate (Feature: translate-text):** The primary translation hook. Calls `POST /api/translate/stream` for SSE-streamed progressive results, falling back to `POST /api/translate/fast` if streaming fails. Handles IX-marker substitution (IX-1 → I, IX-2 → YOU), abort control for cancellation, and debug metadata (method, confidence, processing time). Satisfies FR003, FR004.

- **useVideoEngine (Feature: animate-avatar):** The primary animation driver in the default `avatar` render mode. Fetches sign videos from Supabase Storage CDN, plays each through a hidden <video> element, pipes decoded frames through MediaPipe Tasks Vision (`HolisticLandmarker` running as WebAssembly), and drives the VRM rig via Kalidokit. When no source video is found for a gloss token, delegates to `usePosePlayer` as the fallback. Satisfies FR005, FR008.

- **usePosePlayer (Feature: animate-avatar):** Animation driver for the `skeleton` render mode and the pose-based fallback path. Fetches `.pose` binary files from `GET /api/pose/{gloss}`, parses the precomputed landmark frames in JavaScript, and drives the VRM rig via Kalidokit frame by frame. Maintains an in-memory `Map` cache keyed by gloss name to avoid refetching. Falls back to fingerspelling for unknown glosses. Also serves as the fallback called by `useVideoEngine` when a source video cannot be retrieved. Satisfies FR005, FR008.

- **useCanvasRecorder (Feature: animate-avatar):** Manages canvas-to-MP4 recording. Selects between the `captureStream/MediaRecorder` path (Chrome, Firefox, Edge) and a JPEG frame-sampling path (Safari/iOS), uploads the result to `POST /api/export/video`, and triggers a file download. Satisfies FR012.

- **ExportVideoModal (Feature: animate-avatar):** React component managing the export user experience: a three-phase status display (preparing, composing, packaging),

cycling status messages, and a progress bar from 0% (start) to 100% (download ready).

- **AvatarSwitcher / SkeletonCanvas (Feature: animate-avatar):** `AvatarSwitcher` allows selecting between four VRM models; `SkeletonCanvas` provides a 2D Canvas fallback rendering raw pose landmarks for inspection. Satisfies FR009, FR011.
- **usePlayback (Feature: animate-avatar):** Manages animation sequence state: idle, playing, paused, complete. Provides play, pause, replay, previous/next token, and speed cycling (0.5×–2×). Satisfies FR008.
- **useHistory (Shared):** `localStorage`-based translation history with FIFO eviction at 100 entries. Supports filtering by type, date range, and search query; tracks export flags. Satisfies FR014.
- **KalidokitSolver / vrmRigger (Shared utilities):** Receive normalised landmarks and return VRM-compatible bone rotation quaternions and facial blendshapes. Kalidokit has not received active maintenance since 2022, but it remains the only open-source library that bridges MediaPipe landmark output to VRM bone rotations. No maintained alternative exists for this specific task; the library is stable for the landmark solving requirements of this project.

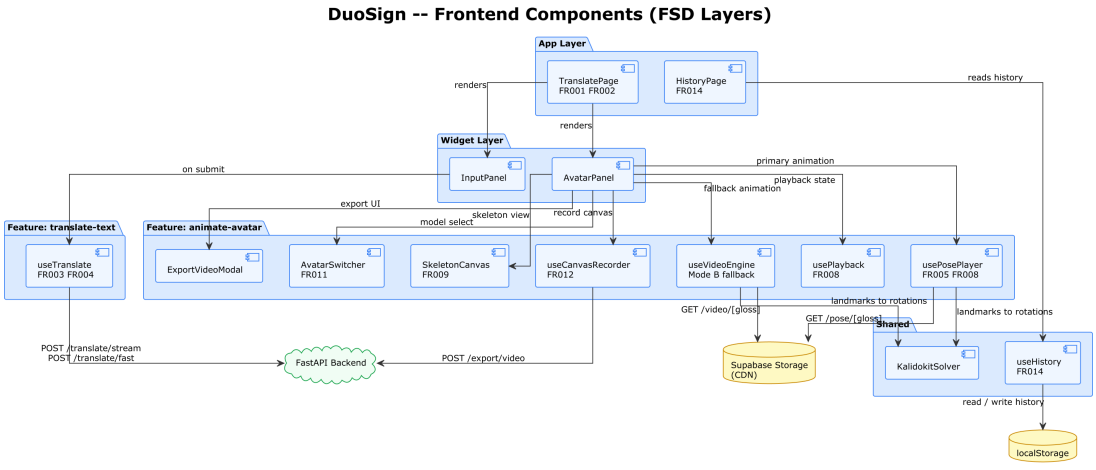


Figure 3.3: Component diagram of the Next.js frontend container.[7]

FastAPI Backend Components. The backend container is decomposed into the following components:

- **TranslateRouter (API Router):** Handles `POST /api/translate`, `POST /api/translate/fast`, and `POST /api/translate/stream` requests. Validates input text (1–500 characters), invokes the GlossEngine, and returns gloss tokens with method and confidence metadata. The streaming endpoint returns an SSE channel with an immediate `rule_based` event and an optional `llm_quality` improvement event.

- **AudioRouter (API Router):** Handles `POST /api/translate/audio`. Accepts multipart audio uploads, forwards the file to the Groq Whisper API (`whisper-large-v3-turbo`), and feeds the transcription into the GlossEngine. Supports BCP-47 language hints. Satisfies FR002.

- **VocabularyRouter (API Router):** Handles `GET /api/vocabulary` and `GET /api/vocabulary/s` Loads the gloss index from `glosses.tsv` and exposes prefix-based search for client-side autocomplete and lexicon checking.

- **MediaRouter (API Router):** Handles `GET /api/pose/{gloss}` and `GET /api/video/{gloss}`, serving binary `.pose` and `.mp4` assets from the bucket directory with HTTP cache headers (`Cache-Control: max-age=86400`).

- **ExportRouter (API Router):** Handles `POST /api/export/video`. Accepts either a WebM blob or a JSON array of base64 JPEG frames, invokes FFmpeg via `imageio-ffmpeg` for H.264 MP4 encoding, and returns the binary MP4 response. Satisfies FR012.

- **GlossEngine (Core Service):** The rule-based NLP pipeline using spaCy for tokenisation, lemmatisation, and Part-of-Speech tagging, followed by ASL grammar transformations: temporal topicalisation, subject-object-verb reordering, article and copula removal, and negation repositioning. Pronoun IX-marker[8] substitution maps I/you/he to IX-1/IX-2/IX-3. When confidence falls below 0.8, the engine fires an async LLM quality-check using the Groq API for improved output. Satisfies FR003, FR004.

[8]IX = Index marker in sign-language gloss notation, used to represent pointing pronouns and referential indexing in signing space.

- **FingerspellEngine (Core Service):** Called when a gloss token has no matching pose asset in the vocabulary. Decomposes the word into individual ASL alphabet letters and produces a sequence of letter gloss tokens (e.g., "Netflix" → N-E-T-F-L-I-X). Satisfies FR006.

- **Authentication and Persistence Layer:** The backend includes user registration, JWT-based authentication (bcrypt-hashed passwords, httpOnly cookie tokens), and SQLAlchemy-backed database persistence. These components satisfy FR013–FR015.

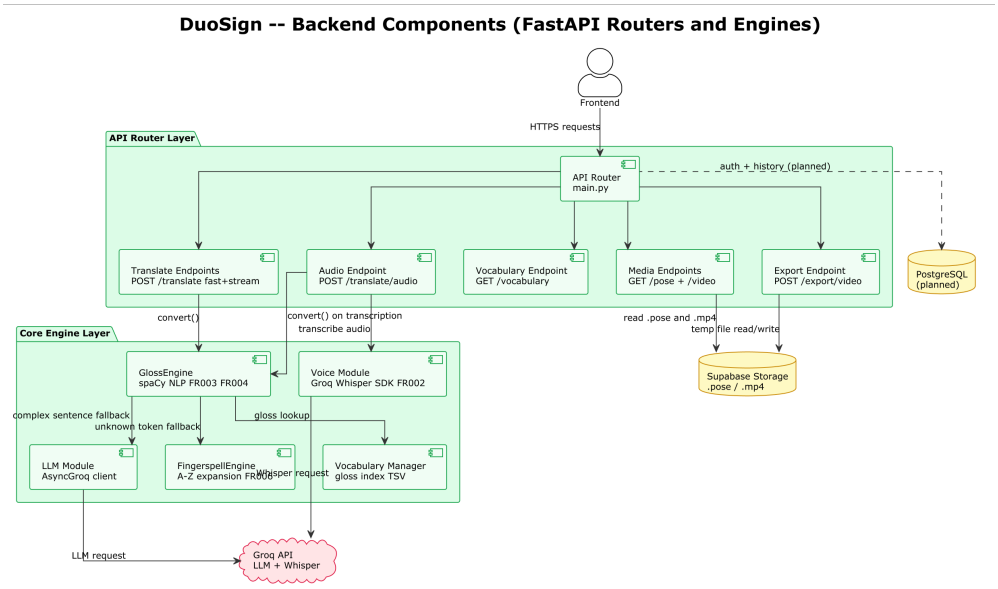


Figure 3.4: Component diagram of the FastAPI backend container.[9]

3.2 System Design

The system design outlines the detailed blueprint for DuoSign. It translates the high-level architecture into specific design decisions that guide implementation. This section covers the core design principles, the pose playback strategy, the data model, interaction diagrams, user interface design considerations, and security design.

3.2.1 Design Principles

Seven design principles shaped the decisions across architecture, data model, and user interface:

- **Modularity:** The system is decomposed into independent, API-connected services for text-to-gloss conversion, pose extraction, avatar animation, and UI rendering. Each service can be developed, tested, deployed, and replaced independently. A researcher can swap the NLP module for a neural machine translation model by conforming to the same API contract, without modifying any other part of the pipeline. This directly satisfies NFR007 (scalability/extensibility).

- **Performance:** The system is optimised to ensure that end-to-end translation and initial avatar rendering completes under 2,000 milliseconds for a standard sentence. The `.pose` file playback strategy (Section 3.2.2) reduces client-side processing by bypassing live MediaPipe inference for pre-extracted glosses, enabling consistent 60 FPS playback even on lower-powered devices. This satisfies NFR002.

- **Scalability:** The architecture separates stateless NLP processing from stateful user data, allowing each layer to scale independently. The backend handles only lightweight text processing and audio forwarding while all compute-intensive rendering operations run client-side, eliminating server-side video handling entirely. This satisfies NFR007.

- **Accessibility:** The user interface provides multiple input modalities (text and voice), includes text transcripts of gloss sequences for users who cannot view animations, supports keyboard navigation, maintains minimum 44×44 pixel touch targets, and meets WCAG AA colour contrast requirements. This satisfies NFR003.

- **Extensibility:** The file-based lexicon allows new signs to be added by dropping `.pose` files into the correct directory with the appropriate gloss identifier, without requiring code changes, redeployment, or retraining. The modular design ensures vocabulary can grow independently of the core pipeline.

- **Data Privacy:** Data privacy is enforced at the architecture level; the concrete controls are defined in Section 3.2.6. This satisfies NFR006.

- **Reproducibility:** As an open-source baseline system, all architectural decisions prioritise transparency and replicability. The fully open-source codebase, documented

API contracts, and standardised data formats allow other researchers to replicate results and extend the system.

3.2.2 Video-Driven Avatar Playback and Pose-Based Skeleton Fallback

The core animation design in DuoSign separates live avatar rendering from precomputed skeletal reference data. Sign videos from the WLASL dataset [23] are the primary animation source: the browser fetches each video from Supabase Storage CDN, plays it through a hidden <video> element, and sends decoded frames through the MediaPipe Tasks Vision `HolisticLandmarker` running as WebAssembly. The resulting landmark arrays (33 body joints, 21 left-hand joints, and 21 right-hand joints) are fed into Kalidokit, which solves them into VRM bone rotation quaternions applied in a `requestAnimationFrame` loop at 60 FPS. This is the `useVideoEngine` path, grounded directly in recordings of real signers from the WLASL corpus [23].

The limitation of this approach is the Kalidokit solver itself. Automated landmark-to-bone conversion does not always produce a visually correct sign: perspective issues in the source video, partial occlusion of hands, or quaternion discontinuities between frames can cause the avatar to hold an unnatural pose for a token. To address this, DuoSign includes a skeleton overlay path. Offline, each sign video in the WLASL dataset was processed through the same MediaPipe pipeline and the resulting landmark data was serialised into a compact binary `.pose` file, one per gloss. The `SkeletonCanvas` component reads these files and renders the raw 2D skeleton alongside the VRM avatar when the user toggles skeleton view. This gives a ground-truth reference of what the sign should look like at the landmark level, independent of the avatar's quaternion interpretation.

In practice: the VRM avatar plays the sign in 3D; the skeleton canvas shows what MediaPipe actually extracted from the source video. Where they match, the sign is correct. Where they diverge, the skeleton is the reliable one. See Figure 3.5.

Implementation constraint: direct `.pose` driving of the avatar. During implementation, I tested whether the avatar could be driven directly from precomputed `.pose` files instead of streaming source video frames through MediaPipe and Kalidokit. In practice, this path was

not stable. The pose structures produced by the Python pose-format tooling did not align cleanly with the frame structure expected by Kalidokit's MediaPipe-oriented reader in the browser.

To verify this, I reverse-engineered the runtime data path: landmark frames passed through the Kalidokit media flow were extracted to JSON and then fed back into the avatar rigging path. The result was consistently poor, including jittery movement, unnaturally fast playback, and frame-timing errors that broke the natural signing rhythm. By contrast, the video-stream path preserves the frame cadence Kalidokit was designed for, because it matches the library's original real-time camera input assumptions. For this reason, DuoSign keeps video-driven avatar animation as the primary path and uses `.pose` files for skeleton reference and fallback rendering.

Coordinate-system normalisation as an architectural decision. A cross-cutting design constraint in this pipeline is coordinate-system compatibility between MediaPipe and Three.js/VRM. MediaPipe emits landmarks in normalised screen space (origin top-left) where the Y-axis increases downward, while Three.js/VRM rigs assume world space where Y increases upward. Without correction, vertical motion is inverted during retargeting. DuoSign therefore applies a boundary transformation before Kalidokit/VRM rigging:

$$y' = 1.0 - y \tag{3.1}$$

where y is the MediaPipe Y-coordinate and y' is the corrected Y-coordinate in the avatar coordinate frame. This normalisation is enforced at the landmark-to-rig interface for both runtime video landmarking and precomputed `.pose` playback, so downstream animation logic operates in a single consistent coordinate space.

3.2.3 Data Model

Current Prototype State (localStorage). In the current DuoSign prototype, persistent state is held in browser-local stores rather than a server-side database. Translation history is serialized to `localStorage` under a dedicated key by the `useHistory` hook. Each history entry stores the original input text, gloss token sequence (JSON array), input type

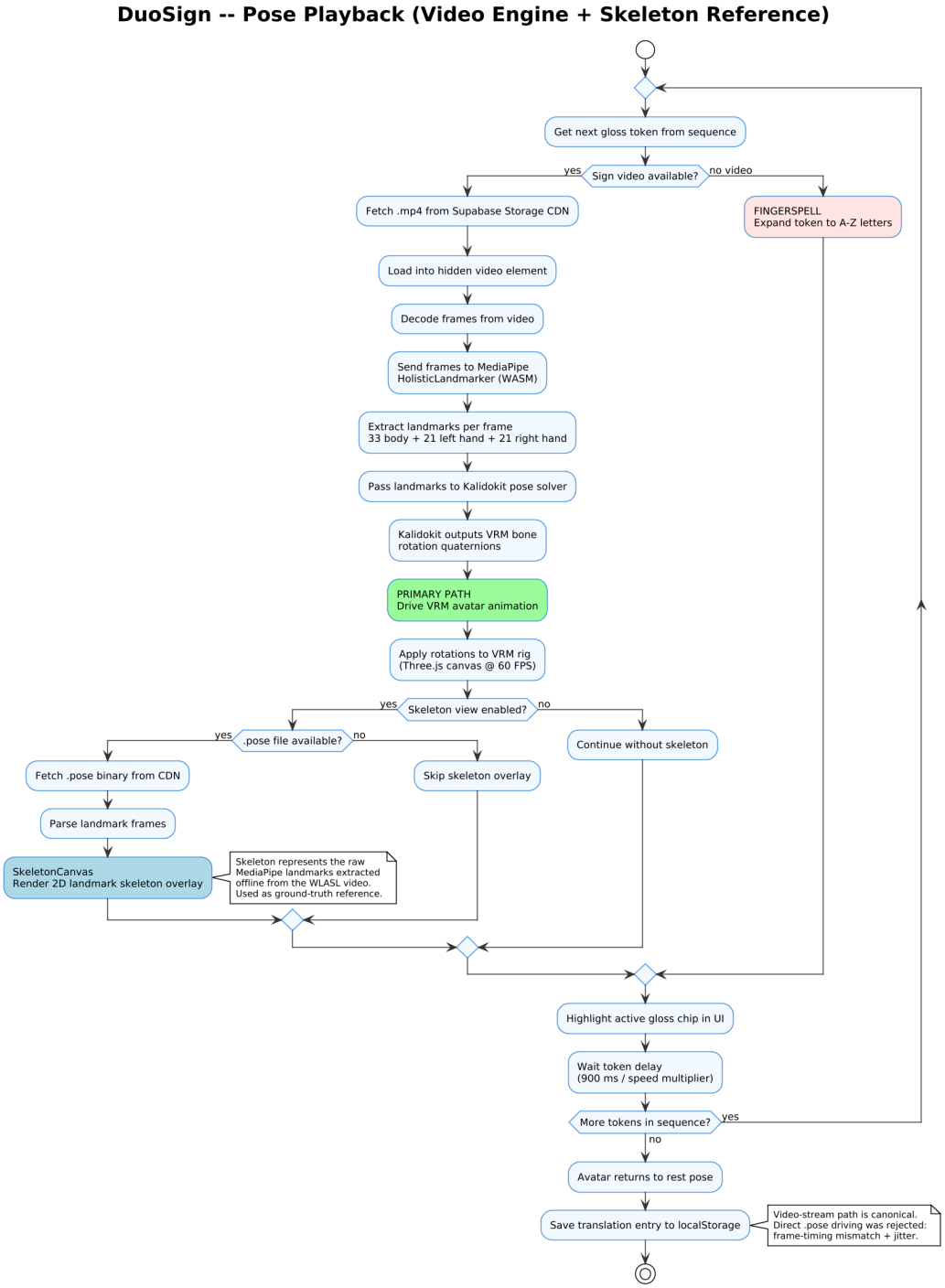


Figure 3.5: DuoSign's animation architecture: sign videos drive real-time VRM avatar playback via MediaPipe Tasks Vision and Kalidokit (primary path), while precomputed `.pose` files provide the pose-based skeleton fallback overlay when VRM quaternion output diverges from source landmarks (FR005, FR008, FR009).

("typed" or "voice"), display-oriented date fields, and an export flag. A FIFO[10] policy caps history at 100 entries. User preferences, including avatar model selection, color accent, animation speed, accessibility toggles, and voice settings, are serialized separately by SettingsProvider through the useSettings hook and rehydrated on page load.

Guest usage tracking uses a session-based browser identifier: the useGuestBanner hook maintains a translation count in localStorage and displays a soft registration prompt when the configured threshold is reached. No hashed identifier or server record is created in the prototype.

Production Database Schema. The production schema has four tables. users stores registered accounts with UUID[11] primary keys and bcrypt-hashed passwords (FR013). sessions manages JWT refresh tokens with cascading delete on account removal. guest_usage records a one-way hashed browser identifier and translation count, never raw IP addresses or personally identifying information (NFR006, NFR017). translations stores translation history for logged-in users: input text, input method, gloss output as a JSONB[12] array, and a creation timestamp (FR014–FR015). The schema is designed and migrations are written; deployment requires connecting the Supabase PostgreSQL instance.

Asset Storage Structure. In the current prototype, sign language media assets are stored in a bucket/ directory mounted beside the application. The structure separates raw videos (one .mp4 clip per gloss class, selected by quality score from the WLASL dataset [23]) from precomputed .pose files. A gloss_video_mapping.json index maps each gloss name to its selected video and pose asset. In production deployment, this directory is replaced by a Supabase Storage bucket; the FastAPI backend reads the asset base URL from an environment variable, so no code changes are required to switch between local and cloud storage. The /api/pose/{gloss} and /api/video/{gloss} endpoints serve binary asset responses with HTTP cache headers (Cache-Control: max-age=86400), allowing browsers to avoid refetching commonly used signs. A /avatars/ directory within the Next.js public/ folder hosts the four VRM avatar models.

[10]FIFO = First In, First Out.
[11]UUID = Universally Unique Identifier.
[12]JSONB = Binary JSON storage format in PostgreSQL that supports indexing and efficient querying.

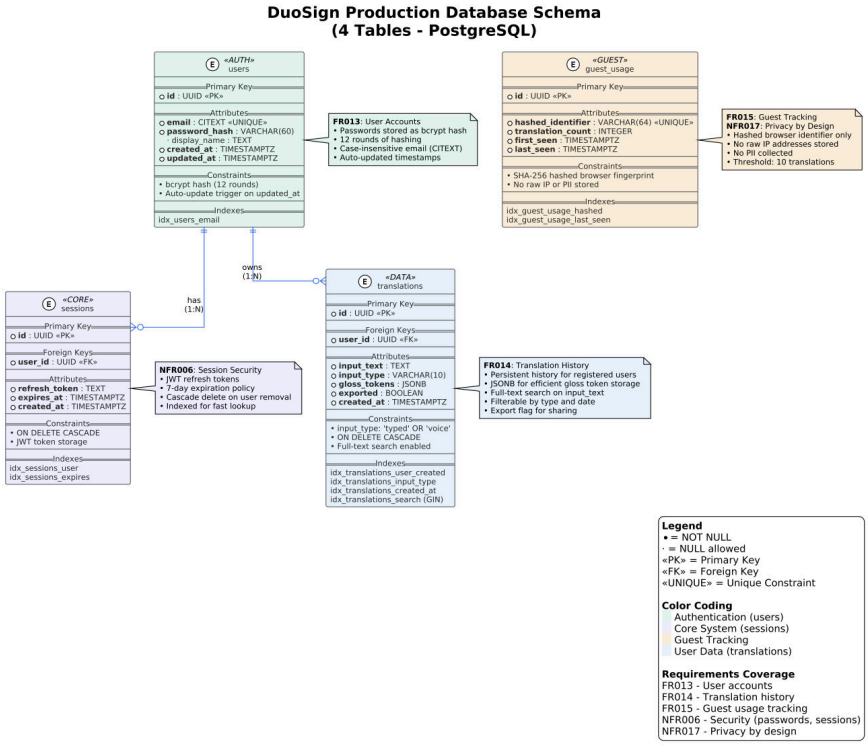


Figure 3.6: Entity relationship diagram for the DuoSign production database schema (FR013–FR015, NFR006, NFR017).

3.2.4 Interaction Diagrams

Activity diagrams document the dynamic behaviour of the four most important system flows.

3.2.4.1 Full Translation Pipeline Activity

The primary activity diagram outlines the complete translation pipeline from text input to avatar animation. See Figure 3.7. The process begins when a user submits English text, either typed or transcribed from voice input. The text is sent to the FastAPI backend via `POST /api/translate/stream` using Server-Sent Events. The backend immediately fires the spaCy rule-based pipeline and emits a `rule_based` event with the initial gloss tokens. If the confidence score is below 0.8 or the sentence contains idiomatic patterns, an asynchronous LLM quality check is fired; if the LLM responds within a configurable timeout, an `llm_quality` event is emitted with the improved result. A `done` event signals completion.

Upon receiving the gloss token array, the frontend iterates through each token sequentially. For each token, the `useVideoEngine` hook fetches the source sign video from Supabase Storage CDN, plays it through a hidden <video> element, and pipes decoded frames through MediaPipe Tasks Vision to drive the VRM in real time. If no source video is found for the token, `useVideoEngine` delegates to `usePosePlayer` as the fallback: the hook checks an in-memory `Map` cache for pre-parsed frame data; on a cache miss it fetches the `.pose` binary file, parses it, caches it, and drives the VRM from the precomputed landmark sequences. If neither asset exists for a word, the `FingerspellEngine` decomposes it into individual letter tokens. After all tokens have played, the system checks the guest usage counter in `localStorage` and, if exceeded, displays a soft registration prompt. The translation entry is automatically appended to the `localStorage` history store.

3.2.4.2 Voice Input Flow

The voice input flow branches from the main translation pipeline and covers both microphone paths. See Figure 3.8. The user clicks the microphone icon in the translation interface. The frontend first attempts to use the Web Speech API. If the browser supports it and microphone permission is granted, native speech recognition starts; the interface enters a listening state with visual feedback, the user speaks, and the browser returns a local transcription. The text is inserted into the input field for review before submission. From there, the flow is identical to typed input.

If the Web Speech API is unavailable or if the user chooses the alternative voice path, the frontend instead initiates a `MediaRecorder` session. The user speaks and the browser captures the audio as a `.webm` blob. On stop, the frontend uploads the blob to `POST /api/translate/audio` with an optional BCP-47 language hint. The backend forwards the audio to the Groq Whisper API, retrieves the transcription, passes it through the GlossEngine, and returns the gloss tokens directly. The transcribed text is displayed to the user alongside the avatar animation.

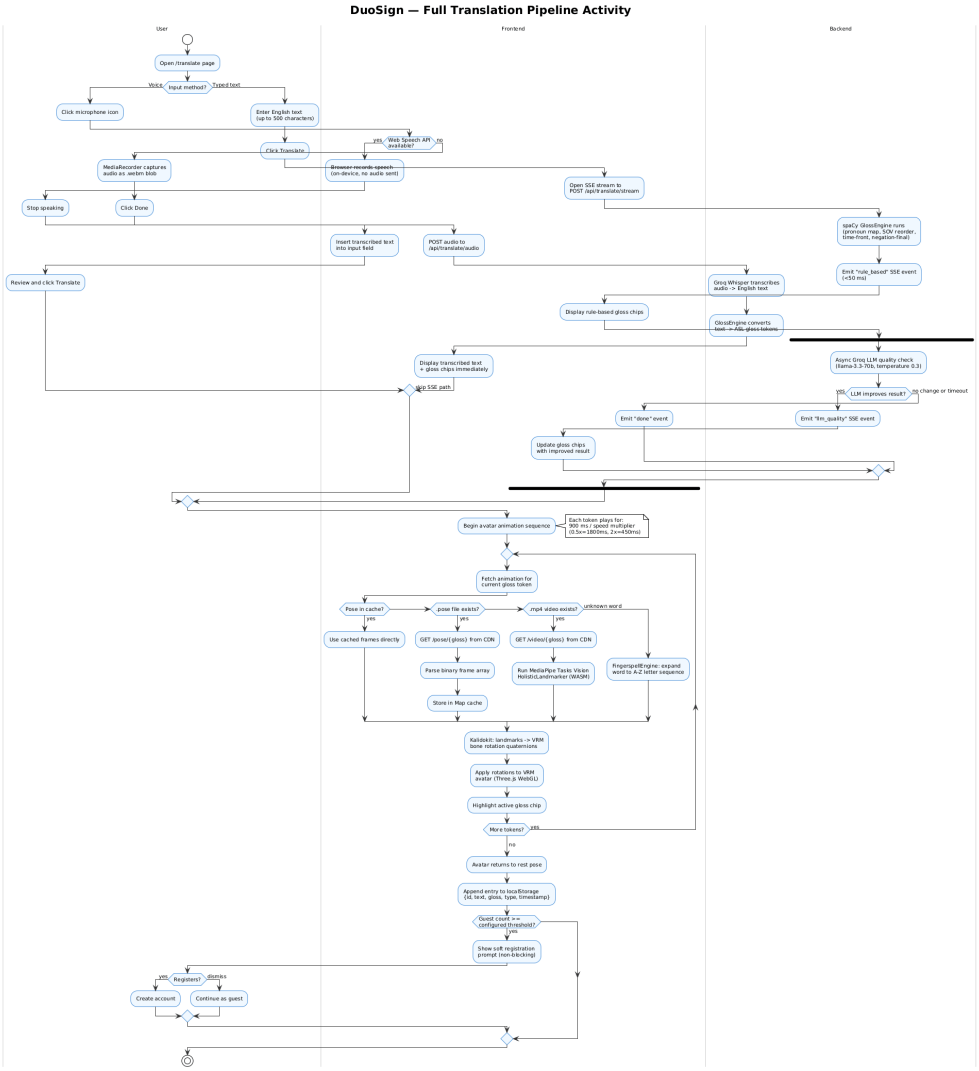


Figure 3.7: Activity diagram of the full translation pipeline from text input to avatar animation.

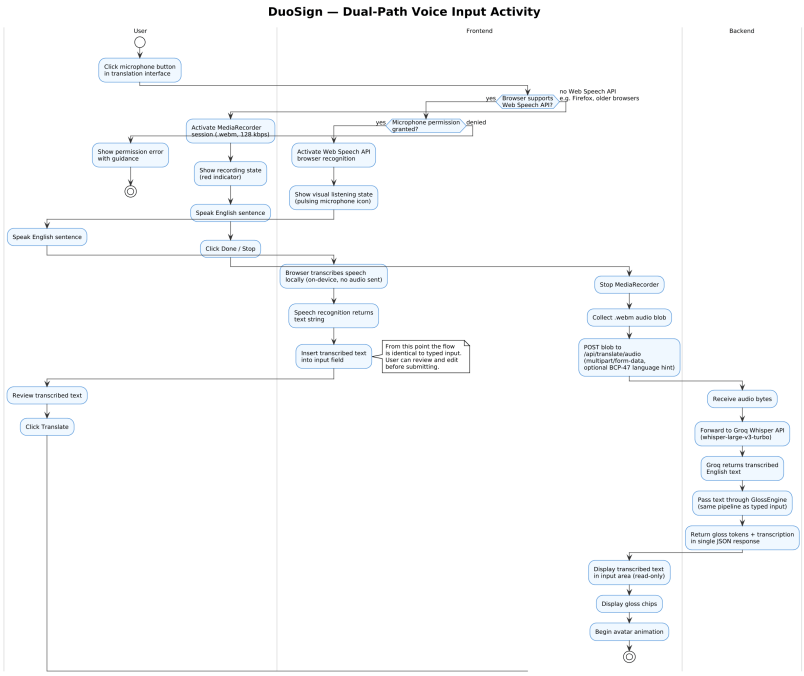


Figure 3.8: Activity diagram of the dual-path voice input flow (Web Speech API and Groq Whisper paths).

3.2.4.3 Guest-to-Registered User Conversion Flow

Guest users are tracked by a session-based browser identifier stored in `localStorage`. When a guest reaches the configured translation threshold, a soft registration prompt appears after the current translation completes. Figure 3.9 shows this conversion flow. Registration is optional and appears only after the user has received value. The prompt offers two choices: create a free account or continue as a guest with reduced access. If dismissed, the user continues in limited mode. If accepted, the user completes a short form (email and password). On success, a JWT is issued in an httpOnly cookie, and the user returns to the translation screen without losing current input. History from that point is stored server-side; previous guest history is not migrated.

3.2.4.4 Video Export Flow

The video export flow allows users to download an MP4 recording of the avatar signing a translated gloss sequence. See Figure 3.10. The user clicks the *Export* button in the avatar panel or on a history entry. The frontend opens the `ExportVideoModal`, which renders a hidden `AvatarCanvas` and immediately begins recording via `useCanvasRecorder`.

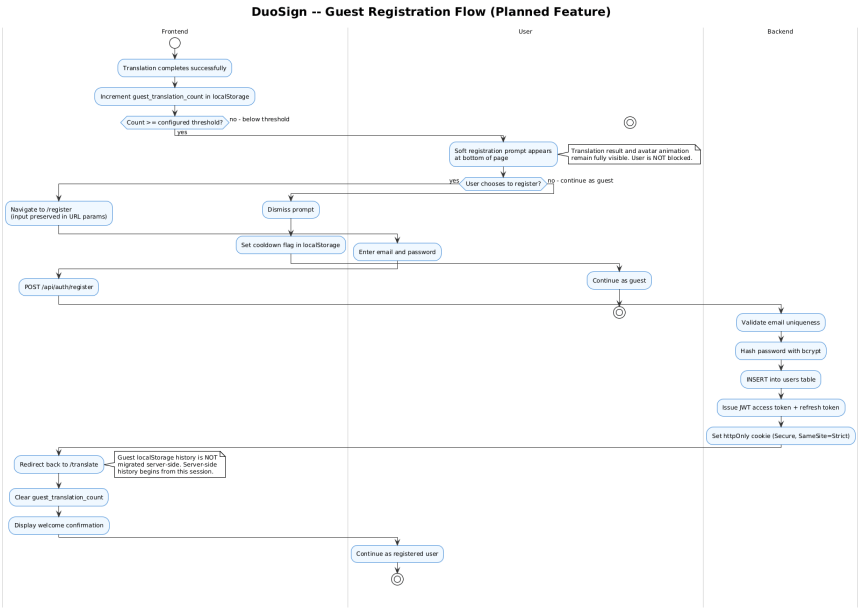


Figure 3.9: Activity diagram of the guest-to-registered user conversion flow.

Two recording paths are used based on browser capability. In Chrome, Firefox, and Edge, `canvas.captureStream(30)` produces a `MediaStream` fed to a `MediaRecorder` at 8 Mbps, producing a `.webm` blob. In Safari, where `captureStream` is unsupported, JPEG frames are sampled at 15 fps via `canvas.toDataURL` and accumulated as a base64 array. Once the animation sequence completes and the recording stops, the blob or frame array is uploaded to `POST /api/export/video`. The FastAPI backend transcodes the input to H.264 MP4 at 1280×720, CRF 18, with `+faststart`, using the bundled `imageio-ffmpeg` binary. The resulting MP4 is returned to the client, which creates a blob URL and triggers download as `duosign-export.mp4`. The history entry for this translation is marked as exported. This flow satisfies FR012.

3.2.5 User Interface Design

The interface was designed for the five user groups identified in Chapter 2: deaf and hard-of-hearing users, hearing users communicating with deaf colleagues, ASL learners, content creators, and researchers. These groups use the same translation screen for different goals. Wireframes were developed in Figma, and representative screens are included in Appendix B.

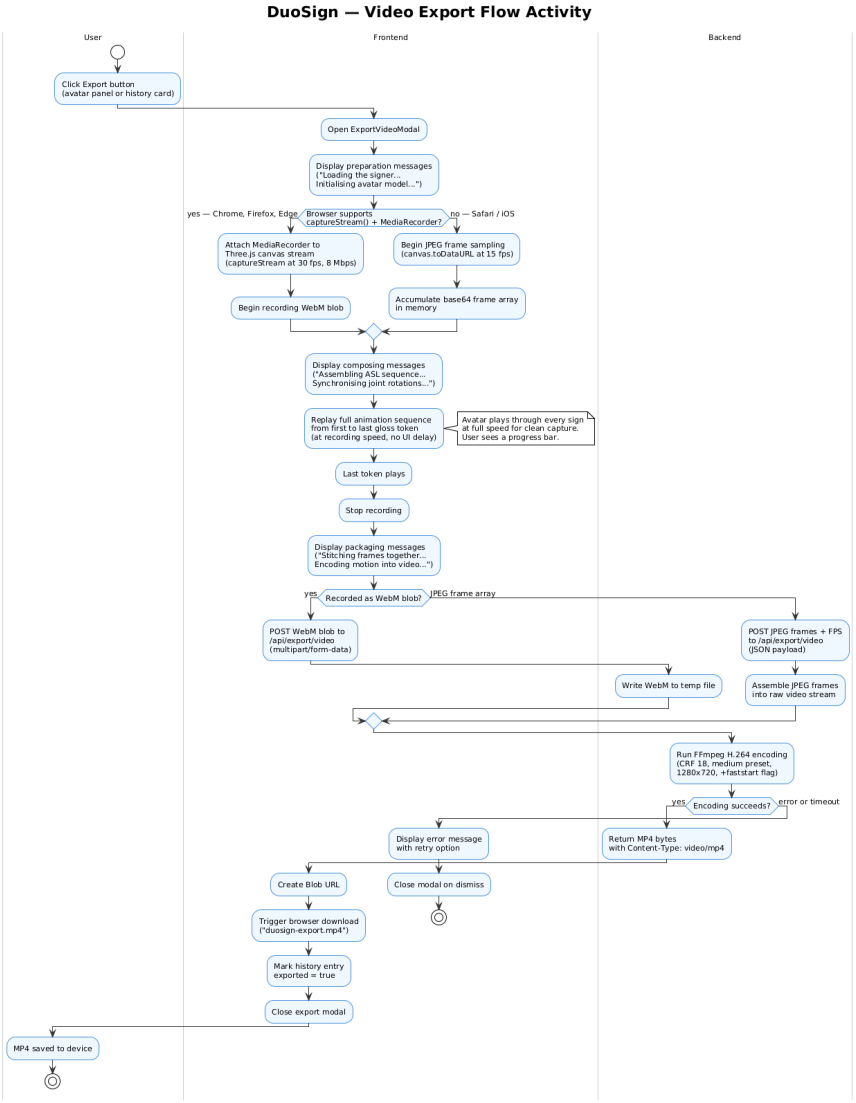


Figure 3.10: Activity diagram of the video export flow from canvas capture to MP4 download.

Layout Principles. The interface follows a two-panel structure: input on the left (or top on mobile viewports) and avatar output on the right (or bottom). This mirrors the interaction pattern of successful translation tools such as Google Translate and ensures spatial separation between where the user acts and where the system responds. The layout is responsive and adapts to desktop (≥1024 px: two-column grid), tablet, and mobile (<1024 px: stacked column with sticky input panel) viewport sizes.

Typography. The design system employs three typefaces, each serving a distinct purpose: Instrument Serif for the DuoSign logo and brand identity, Work Sans Bold for headings and all-caps labels, and Inter Regular for body text and UI elements. This combination provides clear visual hierarchy while maintaining readability across screen sizes.

Key Components. The translation page comprises three primary panels. The **Input Panel** contains the text input area (maximum 500 characters), a state-aware microphone button, a submit button, and a character count indicator. The **Avatar Panel** contains the Three.js canvas rendering the VRM avatar, loading state indicators, playback controls (play, pause, previous/next token, replay, speed cycle), display mode selector (avatar, skeleton, hidden), view mode selector (close-up, full body, 3D world), avatar model switcher, export button, and a fullscreen toggle. The **Gloss Panel** displays token chips horizontally, with the active token highlighted during playback and a distinct visual treatment for fingerspelled words. Every interactive element is designed with four explicit states: default, hover, active/pressed, and disabled. The avatar panel requires additional loading, playing, paused, and error states.

Accessibility Considerations. The avatar panel includes a text transcript of the gloss sequence for users who cannot view the animation. All interactive elements support keyboard navigation with a minimum 44×44 pixel touch target. Colour contrast meets WCAG AA as a minimum. The voice input button uses shape change rather than colour alone to indicate state, ensuring usability for users with colour vision deficiency.

3.2.6 Security Design

Security is a cross-cutting concern that informs decisions across all layers of the architecture. The security design for DuoSign addresses API protection, data privacy, and

guest tracking.

Authentication. JWT access tokens are issued on login and stored in httpOnly cookies rather than `localStorage`, preventing browser scripts from reading them. Access tokens expire after 24 hours with refresh token rotation on renewal. Passwords are hashed with bcrypt (minimum 12 rounds) before storage.

API Security. The FastAPI backend enables CORS[13] only for explicitly configured development and deployment origins. All text inputs are validated through Pydantic request models before entering the NLP pipeline. Translation endpoints are rate-limited to 60 requests per minute per IP address to reduce abuse.

Data Privacy. Video frames used for avatar landmarking are processed client-side via WebAssembly; no raw video reaches the backend. Voice input follows two paths. With Web Speech API, transcription stays in the browser and audio does not leave the device. With Groq Whisper, spoken audio is sent to FastAPI, forwarded for transcription, and then discarded; only text proceeds. Translation history is stored in `localStorage`; no personal interaction data is retained on the server beyond each request.

Guest Tracking. Guest usage identifiers are one-way hashed before server-side storage. The server stores only a count and timestamp, never a raw IP address or personally identifying information.

3.3 Deployment Architecture

This section describes how DuoSign is deployed beyond a local development machine, mapping each component in the container diagram to a hosting provider. The choices are deliberately aligned with the project's constraints: everything runs within free or low-cost tiers, no dedicated infrastructure management is required, and the entire system can be reproduced by anyone following this documentation.

[13]CORS = Cross-Origin Resource Sharing, a browser security mechanism controlling cross-origin HTTP requests.

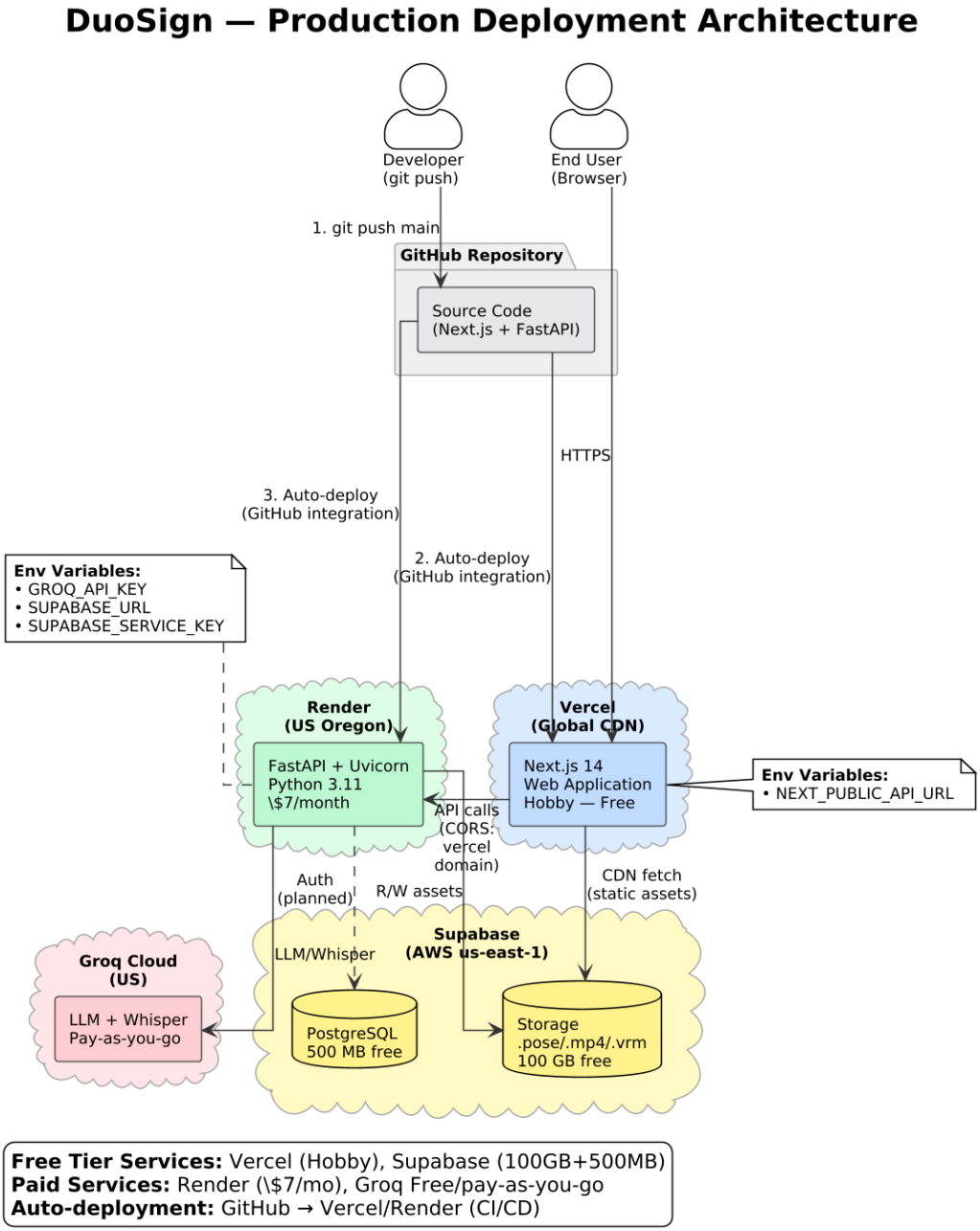


Figure 3.11: Deployment architecture for DuoSign. The Next.js frontend is hosted on Vercel and calls the FastAPI backend on Render for translation, transcription, and export endpoints. Media assets are delivered through Supabase Storage CDN directly to the browser, persistent data is stored in Supabase PostgreSQL, and Groq API provides Whisper transcription and optional LLM quality-refinement calls.

3.3.1 Hosting Stack

Table 3.1 summarises the production hosting plan for each container.

Table 3.1: DuoSign production hosting stack.

Container	Provider	Plan	Monthly Cost
Next.js Frontend	Vercel	Hobby (free)	$0
FastAPI Backend	Render	Free / Starter	$0–$7
Media Asset Store	Supabase Storage	Free (100 GB)	$0
PostgreSQL Database	Supabase	Free (500 MB DB)	$0
LLM + Whisper	Groq API	Pay-as-you-go	<$1

Next.js Frontend — Vercel. Vercel is the natural deployment target for Next.js applications: it is built by the same team, requires zero configuration for App Router projects, and provides a global CDN with automatic HTTPS on its free Hobby plan. Each `git push` to the main branch triggers an automatic deployment. Static assets and JavaScript bundles are served from Vercel's edge network; only API calls leave the browser to reach the FastAPI backend.

FastAPI Backend — Render. The FastAPI service is deployed on Render, a platform-as-a-service provider for Python web services. A `render.yaml` file at the repository root defines the build command (`pip install -r requirements.txt`) and start command (`uvicorn main:app -host 0.0.0.0 -port $PORT`). The Python environment and dependencies, including spaCy and the bundled FFmpeg binary from `imageio-ffmpeg`, are installed at build time, so no system-level packages are required on the host.

Media Asset Store — Supabase Storage. Sign language reference videos, precomputed `.pose` files, and VRM avatar models are stored in Supabase Storage. Supabase provides 100 GB on the free plan, an S3-compatible API, built-in CDN delivery, and public asset URLs. Because Supabase also hosts PostgreSQL, storage and database remain under one platform and one credential scope. At this project scale, roughly 2,500 sign videos at about 1.5 MB each plus matching `.pose` files remain well within the free tier. The FastAPI backend reads asset paths from environment variables pointing to Supabase public URLs.

Capacity Estimate. At 100 simultaneous users making one translation request per minute, the FastAPI backend receives about 100 requests per minute. Each request is lightweight, with text input and JSON gloss output, so one Render Starter instance (512 MB RAM, shared

CPU) is sufficient. Media assets are fetched directly from Supabase CDN by the browser, so these requests do not load the FastAPI backend. Supabase free-tier egress is 5 GB per month; estimated usage at this traffic level is about 1.5 GB per month, which stays within the free allowance.

3.3.2 Scalability Considerations

The current deployment is sized for a small user base, up to about 100 concurrent users, which fits this academic implementation. Scaling does not require structural redesign; the path is horizontal. The FastAPI backend is stateless and keeps no in-memory session state between requests, so additional Render instances can be placed behind a load balancer without code changes. The frontend is already distributed through Vercel's CDN. Media storage scales through Supabase plan upgrades or migration to Cloudflare R2[14].

Microservices, Kubernetes, and container orchestration are intentionally outside the scope of this project. One FastAPI process handles translation, audio, vocabulary, media, and export endpoints together. This keeps operations simple for a research prototype and avoids infrastructure overhead that is unnecessary at the current scale. If demand increases, the modular design still allows service-level separation later.

3.4 Summary

This chapter has presented DuoSign's architecture and design using three C4 levels. It detailed the modular pipeline, the split between server-side NLP and client-side pose extraction/rendering, and the rationale behind these choices. It also documented the FSD frontend, FastAPI backend, pose playback strategy with `.pose` fallback, storage and deployment setup, and the main interaction flows (translation, voice, registration, and export). The chapter closes with interface and security design decisions tied to Chapter 2 requirements.

Throughout the chapter, design decisions were mapped to Chapter 2 requirements: the FSD frontend supports FR001 (translation interface), FR014 (history), and FR009 (display modes); the FastAPI NLP service supports FR003 (text-to-gloss) and FR002 (voice

[14]R2 = Cloudflare's object storage service with S3-compatible APIs.

transcription); the pose playback strategy supports FR005 (avatar animation) and FR008 (playback controls); the dual-path voice design supports FR002 while addressing privacy trade-offs; the export flow supports FR012; and authentication plus persistence design support FR013–FR015. Together, these decisions form the implementation blueprint used in Chapter 4.

Chapter 4: Implementation

This chapter explains how DuoSign was built in practice. Rather than restating the full architecture from Chapter 3, it focuses on implementation-level decisions: the tools actually used, how the main subsystems were assembled, and the practical trade-offs made during development.

4.1 Software, Tools, and Technologies

DuoSign uses the stack described in Chapter 3 (Section 3.1). This chapter therefore records only implementation-specific observations that affected development rather than re-listing each framework in full.

The most important frontend observation was performance: the Three.js scene typically required about 400–600 ms to initialise, which shaped loading behaviour and justified progressive interface feedback while the avatar renderer became interactive. A second practical finding was that Kalidokit remained useful as a bridge between MediaPipe landmarks and VRM bone rotations, but its maintenance status and solver artefacts meant that some signs still required manual tuning and careful debugging during playback.

4.2 Software Development Approach

DuoSign was built through a solo adaptation of Agile development. Work was organised into short implementation cycles, with each cycle producing a usable increment that could be tested before the next subsystem was added. This mattered because several decisions — especially around pose driving, avatar playback, export, and deployment — only became clear after working prototypes were exercised end to end.

In practice, development prioritised a minimal usable pipeline first and then improved each subsystem in cycles. This kept the work responsive to technical uncertainty while preserving a modular structure: frontend and backend responsibilities stayed clearly separated, features were added in layers, and interfaces were kept stable enough to support debugging,

testing, and later extension.

4.3 Implementation Details

DuoSign is built and integrated using multiple interacting components. This section details the technical execution of each component, covering project structure, the text-to-gloss translation engine, WLASL video acquisition and pose extraction, the avatar animation system, video export, user interfaces, and security considerations.

4.3.1 Project File Structure

The DuoSign system is organised into a modular file structure that separates offline data processing scripts from the runtime application code.

The frontend follows the Feature-Sliced Design described in Chapter 3 (Section 3.1.1.1), so the implementation focus here is how that structure supported the finished repository layout rather than re-explaining the pattern.

The complete project structure is as follows:

This organisation enforces a strict dependency rule: layers may only import from layers below them in the hierarchy (`app` → `widgets` → `features` → `entities` → `shared`), preventing circular dependencies and ensuring that domain logic remains decoupled from presentation.

4.3.2 Client State and Asset Storage

The storage split introduced earlier in this chapter is reflected directly in the runtime implementation. Translation history and user preferences remain in browser `localStorage`, while reusable sign videos and `.pose` assets are loaded from the shared media bucket.

At runtime, the main implementation concern is not persistence itself but retrieval efficiency: media routes emit cache headers, and parsed landmark data is reused in memory to reduce repeated processing during playback.

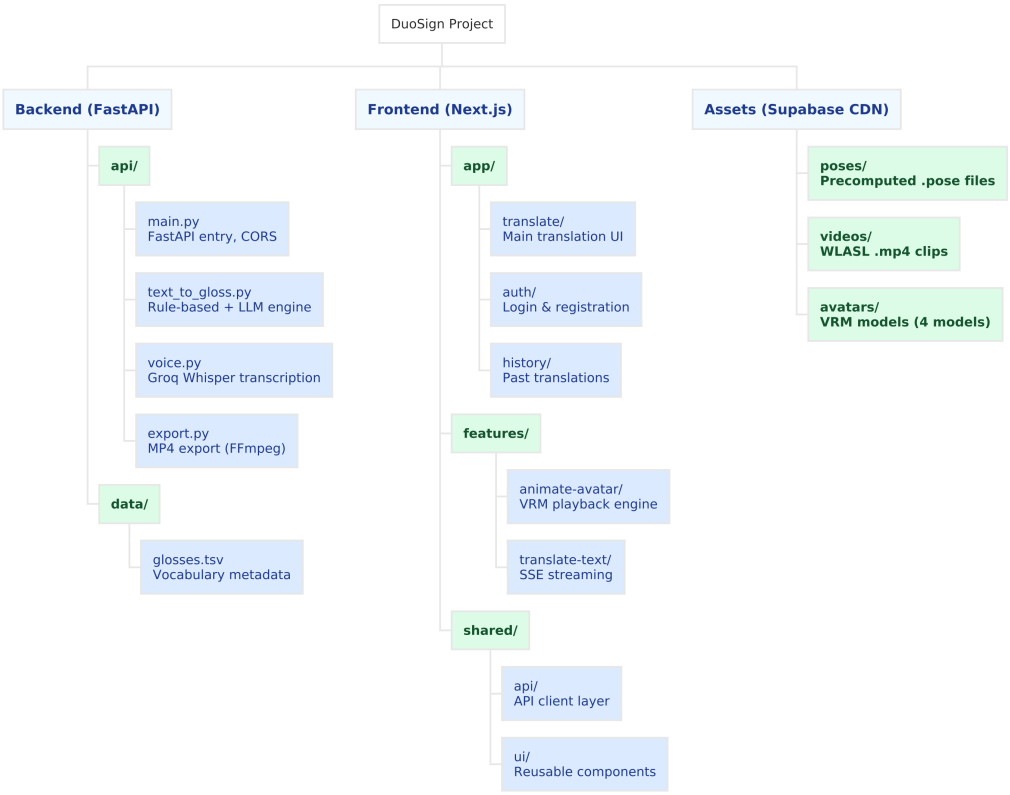


Figure 4.1: Project structure diagram for DuoSign, showing the high-level repository layout across frontend, backend, media asset storage (`bucket/`), and supporting configuration files used during development and deployment.

4.3.3 Text-to-Gloss Translation Engine

The text-to-gloss translation engine forms the core NLP component of DuoSign, converting standard English text into ASL gloss notation. The engine is implemented in `text_to_gloss.py` as a FastAPI service with three processing stages: text normalisation, grammatical transformation, and lexicon validation.

4.3.3.1 Text Normalisation

The normalisation stage cleans raw input text to produce a consistent token stream. Operations include lowercasing, punctuation removal, and contraction expansion (e.g., "don't" → "do not", "won't" → "will not"). The expanded forms ensure that subsequent POS tagging and dependency parsing operate on complete grammatical structures rather than contracted forms that may be ambiguously tagged.

4.3.3.2 Grammatical Transformation

The grammatical transformation stage converts English word order and structure into ASL-appropriate gloss sequences using spaCy's dependency parser and POS tagger. The rule-based system applies three principal transformations:

- **Reordering:** English's Subject-Verb-Object (SVO) order is transformed to ASL's Time-Topic-Comment structure. Temporal markers are fronted to the beginning of the gloss sequence (e.g., "I went to the store yesterday" → `YESTERDAY STORE I GO`).
- **Dropping:** Articles (*a*, *an*, *the*) and copulas (*is*, *am*, *are*) are removed where appropriate, as these do not have direct equivalents in ASL grammar.
- **Pronoun and Synonym Mapping:** Personal pronouns are converted to IX (Index) markers (I → IX-1, you → IX-2, he/she → IX-3, we → IX-1+, they → IX-3+) for spatial reference in ASL [32]. Words not present in the lexicon are mapped to known synonyms (e.g., "wealthy" → `RICH`, "automobile" → `CAR`).

A confidence score is computed as the proportion of output tokens that have known gloss assets. When this falls below 0.8 or idiomatic patterns are detected, an asynchronous LLM quality check fires against the Groq API and, if it responds within the configured timeout, an improved result is emitted as a second SSE event.

4.3.3.3 Lexicon Validation and Fallback

Each produced gloss token is validated against the loaded vocabulary, sourced from `glosses.tsv` and the bucket's gloss mapping. The validation follows a three-tier fallback strategy:

1. **Direct match:** The gloss class name exists in the vocabulary and has an associated pose asset.

2. **Synonym fallback:** If the primary gloss is missing, the system checks known synonyms before escalating.

3. **Fingerspelling fallback:** If no gloss class match or synonym is found, the system decomposes the word into individual letters and produces a sequence of fingerspelling tokens using the ASL manual alphabet (e.g., "Netflix" → `N-E-T-F-L-I-X`).

4.3.3.4 Algorithm: Text-to-Gloss Conversion Pipeline

Algorithm 1 formalises the `TextToGloss.convert()` method that drives every translation request. Its high-level structure is adapted in part from the open-source SignMT workflow reported by Moryossef et al. [26], particularly in the separation of conversion stages and fallback logic, although the concrete rules and runtime paths used here are specific to DuoSign. The two-path design — rule-based first, LLM only when needed — gives the system its responsiveness: the rule-based path completes in under 50 ms, so most requests return immediately. The background quality check fires concurrently and, if it returns within the 3-second timeout, upgrades the result silently over the SSE channel.

Algorithm 2 details the rule-based sub-pipeline. The key design decision is that tokens are kept in their original parse order throughout filtering and glossification; reordering is

Algorithm 1 Text-to-Gloss Conversion — Main Pipeline

Require: English sentence s, vocabulary V, quality-check flag q
Ensure: `GlossResult` containing token list, method, confidence

$s \leftarrow \textsc{Trim}(s)$
if $s = \emptyset$ **then**
    return empty result
end if
$k \leftarrow (s.\text{lower}(), q)$ ▷ cache key separates fast-mode from full-mode results
if $k \in$ cache **then**
    return cache$[k]$
end if
Step 1: Idiom short-circuit
for each $(i, g) \in$ IDIOM_MAP **do**
    if $i \subseteq s.\text{lower}()$ **then**
        return result$(g,$ `rule_based`$, 1.0)$
    end if
end for
Step 2: Rule-based conversion (always runs, <50 ms)
$r \leftarrow \textsc{RuleBasedConvert}(s, V)$ ▷ see Algorithm 2
Step 3: LLM paths
if r.needs_llm **then** ▷ confidence < 0.8, idiom, or complex grammar
    $g_\ell \leftarrow \text{LLMTranslate}(s, V, r.\text{gloss})$ ▷ mandatory, no timeout
    if $g_\ell \neq \varnothing$ **then**
        $r \leftarrow \textsc{ApplyLLM}(r,\ g_\ell,$ `llm`$)$
    end if
    cache$[k] \leftarrow r$; **return** r
end if
if q **and** $(|r.\text{tokens}| \geq 2$ **or** $s.\text{endsWith}(?))$ **then**
    $g_q \leftarrow \text{LLMTranslate}(s, V, r.\text{gloss},\ \text{timeout} = 3.0\text{s})$ ▷ background quality check
    if $g_q \neq \varnothing$ **and** $g_q \neq r.\text{gloss}$ **then**
        $r \leftarrow \textsc{ApplyLLM}(r,\ g_q,$ `llm_quality`$)$
    end if
end if
cache$[k] \leftarrow r$; **return** r

```
POST /api/translate/stream          # SSE streaming (primary path)
{ "text": "I am going to the store tomorrow" }

SSE events:
  data: {"event":"rule_based","tokens":["TOMORROW","STORE","IX-1","GO"],
         "method":"rule_based","confidence":0.92}
  data: {"event":"done"}

# Optional second event if LLM improves within timeout:
  data: {"event":"llm_quality","tokens":["TOMORROW","STORE","I","GO"],
         "method":"llm_quality","confidence":0.97}

POST /api/translate/fast            # Rule-based only, <50ms
POST /api/translate                 # Full pipeline with LLM quality
POST /api/translate/audio           # Groq Whisper → translate
```

Figure 4.2: Text-to-gloss conversion pipeline in DuoSign. Input text is normalised, transformed by the rule-based ASL grammar engine, and confidence-scored against the vocabulary. Low-confidence or complex inputs trigger an optional LLM quality-refinement path, and results are returned through the streaming API as rule-based and optional quality-improved events.

applied as a series of targeted structural moves rather than by decomposing the sentence into category buckets. This preserves modifier–noun relationships automatically — an adjective already beside its noun requires no special rule to stay there after SOV reordering.

4.3.4 WLASL Pose Extraction Pipeline

With the downloaded video corpus established, the offline processing workflow transformed each short clip into a structured landmark tensor suitable for avatar animation at runtime. This preprocessing stage is conceptually separate from the shipped runtime: the frontend consumes prepared `.pose` assets and only invokes MediaPipe Tasks Vision when animating directly from source videos in the browser.

4.3.4.1 Pipeline Architecture

The extraction pipeline follows a multi-stage architecture with parallelised execution. Eight worker processes operated concurrently, each maintaining its own MediaPipe Holistic instance to avoid shared state. Video frames were decoded sequentially, landmarked, scored

Algorithm 2 Rule-Based ASL Grammar Transformation (RuleBasedConvert)

Require: English sentence s, vocabulary V
Ensure: Gloss token list G, confidence score c, `needs_llm` flag

doc $\leftarrow$ spaCy.parse(s)
Step 1: Filter (keep content words in parse order)
$K \leftarrow [\,]$
for each token $t \in$ doc **do**
  if $t \in$ WH_WORDS **then**
    append t to K
    continue
  end if
  if t.pos $\in$ {`PUNCT`, `DET`, `ADP`, `SPACE`, `SYM`} **then**
    continue
  end if
  if t.dep $\in$ {`aux`, `auxpass`} **then**
    continue
  end if
  if t.lemma $\in$ BE_LEMMAS **and** IsCopular(t) **then**
    continue
  end if
  if t.pos = `PART` **and** $t \notin$ {`not`, `n't`} **then**
    continue
  end if
  if $t \in$ FILLER_WORDS $\cup$ POSSESSIVE_PRONOUNS **then**
    continue
  end if
  append t to K
end for
Step 2: Glossify each token
$G \leftarrow$ Glossify(K, V) ▷ NOT→NOT; WH→upper; NUM→sign; PRON→IX; NOUN/VERB→lemma; unknown→fingerspell
$G \leftarrow$ ExpandFingerspelling(G) ▷ "N-A-N-A" → [N, A, N, A]
Step 3: Targeted ASL reordering
$G \leftarrow$ ReorderSOV(G) ▷ find ROOT verb; move it after last direct object
$G \leftarrow$ ModifiersAfterNoun(G, `dep=amod`, `pos=ADJ`) ▷ "BIG RED CAR" → "CAR BIG RED"
$G \leftarrow$ ModifiersAfterNoun(G, `dep=nummod`, `pos=NUM`) ▷ "THREE DOG" → "DOG THREE"
$G \leftarrow$ MoveToFront(G, `is_time_word`) ▷ temporal context: YESTERDAY ...
$G \leftarrow$ InsertAfterGroup(G, `is_location`, `after=is_time_word`) ▷ location follows time
$G \leftarrow$ MoveToEnd(G, `is_wh_word`) ▷ "WHERE IS BATHROOM?" → BATHROOM WHERE
$G \leftarrow$ MoveToEnd(G, g = `NOT`) ▷ negation final: SHE PIZZA LIKE NOT
Step 4: Confidence
$R \leftarrow \{g \in G : g \notin \text{IX-markers} \cup \{\text{NOT}\}\}$
$c \leftarrow |R \cap V| \,/\, |R|$ ▷ proportion of content tokens in vocabulary
needs_llm $\leftarrow c < 0.8 \;\vee$ complex_grammar(s) $\vee$ (s.endsWith(?) $\wedge$ |doc| > 6) **return** G, c, needs_llm

for quality, and serialised into reusable pose assets. The important interface between offline extraction and online playback is that offline processing produces fixed-shape landmark data; online playback either parses those `.pose` files into frame arrays, or derives equivalent landmark sets on the fly via the browser `HolisticLandmarker`. In both cases, Kalidokit receives a consistent landmark structure for rig solving.

For each video, the pipeline proceeded through the following stages:

1. **Video decoding:** OpenCV's `VideoCapture` interface opened the video file, extracted frame rate and frame count metadata, and emitted frames one at a time as RGB `numpy` arrays. BGR-to-RGB colour space conversion was applied at read time, as MediaPipe's inference graph expects RGB input.

2. **Landmark detection:** MediaPipe Holistic processed each frame, returning pose, face, and hand landmarks. Missing regions were represented explicitly so that frame shapes remained consistent across videos with partial detection.

3. **Landmark reduction:** The 33-point BlazePose skeleton was reduced to 13 upper-body landmarks: nose (index 0), left and right shoulders (11, 12), elbows (13, 14), wrists (15, 16), and the hand-to-wrist connection points—pinky, index, and thumb tips (17–22). Lower-body landmarks (hips, knees, ankles, feet) were discarded, as sign language articulation is exclusively performed by the upper body. This reduction produced a final layout of 523 landmarks per frame: 13 pose + 468 face + 21 left hand + 21 right hand.

4. **Quality scoring:** A composite quality metric Q was computed for each video:

$$Q = 0.30 \cdot r_{\text{pose}} + 0.25 \cdot r_{\text{face}} + 0.20 \cdot \bar{r}_{\text{hands}} + 0.15 \cdot \bar{c} + 0.10 \cdot c_{\text{complete}} \tag{4.1}$$

where r_{pose} is the pose detection rate, r_{face} is the face detection rate, $\bar{r}_{\text{hands}}$ is the mean bilateral hand detection rate, $\bar{c}$ is the mean per-landmark confidence, and c_{complete} is the proportion of frames where all four regions were simultaneously detected. Videos failing to achieve a minimum pose detection rate of 70% were excluded from candidacy.

5. **Serialisation:** The assembled landmark tensor of shape $(T \times 523 \times 4)$—encoding $(x, y, z, \text{visibility})$ per landmark per frame—was written to disk as a compressed NumPy archive with the `.pose` extension. Atomic writes were used: data was first written to a `.tmp` temporary file, then renamed, guaranteeing that a crash during serialisation could not produce a corrupted output file.

4.3.4.2 Performance Results

The pipeline was run to completion on an Apple MacBook M4 Pro without requiring cloud compute. The following metrics were recorded:

Table 4.1: WLASL Pose Extraction Pipeline Performance

Metric	**Value**
Total videos processed	11,980
Processing time	4,548 seconds (≈76 minutes)
Success rate	100%
Average throughput	2.6 videos/second
Worker processes	8 (parallel)
Memory per worker	<500 MB

The Apple Neural Engine's deep learning acceleration contributed measurably to the 2.6 videos/second throughput.

4.3.4.3 Quality-Based Pose Selection

Following extraction, the selection script consumed the quality scores per video and selected the single highest-scoring representative for each gloss class. The important point is that selection does not introduce a second weighting scheme: it reuses the same composite metric defined in Equation 4.1. All candidate videos for a given gloss were therefore scored during extraction, then sorted by the same score Q before the winner was checked against the minimum acceptance threshold. The top-ranked candidate was designated the primary animation asset, while lower-ranked alternatives that remained reasonably close in quality were retained as fallbacks. This produced a flat dictionary keyed by gloss name, containing the selected video identifier, detection rates, confidence average, and composite rank score.

This mapping file serves as the runtime lexicon index consulted by the frontend before initiating any animation request.

Algorithm 3 formalises this ranking stage explicitly as a reuse of the extraction-time quality metric rather than a separate selection-specific formula. Pose and face reliability remain explicit, but hand visibility is aggregated bilaterally as $\bar{r}_{\text{hands}}$, while $\bar{c}$ and c_{complete} preserve information about landmark confidence and full-frame completeness during ranking.

Algorithm 3 Quality-Based Pose Selection from WLASL Corpus

Require: Set of candidate videos $\mathcal{V}_g$ for gloss g
Require: Weights from Equation 4.1
Require: Acceptance threshold τ=0.40
Ensure: Best `.pose` asset for g; ranked fallback list; `rankings.json` entry

ranked $\leftarrow$ [ ]
for each video $v \in \mathcal{V}_g$ **do**
    Run `video_to_pose -format mediapipe` on v ▷ subprocess per worker
    $r_{\text{pose}}, r_{\text{face}}, r_l, r_r \leftarrow$ DetectionRates(v.poseFile) ▷ fraction of frames each component detected
    $\bar{r}_{\text{hands}} \leftarrow (r_l + r_r)/2$
    $\bar{c}, c_{\text{complete}} \leftarrow$ ConfidenceStats(v.poseFile)
    $Q(v) \leftarrow 0.30 \cdot r_{\text{pose}} + 0.25 \cdot r_{\text{face}} + 0.20 \cdot \bar{r}_{\text{hands}} + 0.15 \cdot \bar{c} + 0.10 \cdot c_{\text{complete}}$
    append $(v, Q(v))$ to ranked
end for
ranked $\leftarrow$ SortDescending(ranked, key = Q)
if ranked = [ ] **then**
    warn `no usable video for gloss` g ▷ exclude gloss from live lexicon **return** $\varnothing$
end if
best $\leftarrow$ ranked[0]
if Q(best) $< \tau$ **then**
    warn `no usable video for gloss` g ▷ top candidate fails acceptance threshold **return** $\varnothing$
end if
backups $\leftarrow$ [$(v, q) \in$ ranked[1:4] : $q \geq 0.7\tau$]
Copy best.poseFile $\rightarrow$ `best/{g}.pose`
Copy best.videoFile $\rightarrow$ `videos/{g}.mp4`
Write `rankings.json` entry: $\{g$: [best, backups]$\}$ **return** best

4.3.5 Pose Serialisation and Runtime Loading

The runtime code consumes binary `.pose` files directly. The frontend's `usePosePlayer` hook fetches a `.pose` asset from `/api/pose/{gloss}`, parses the binary structure in

JavaScript into per-frame landmark arrays, and caches the result in an in-memory `Map` for reuse within the page session. This design removes an extra transformation step from the delivery pipeline: a pose file is fetched once, parsed, and then replayed at a target frame rate while Kalidokit solves upper-body pose, both hands, and facial motion for each frame. If a gloss has no corresponding pose asset, the player degrades gracefully by briefly holding the current pose rather than breaking the sequence.

Alongside this pose-player path, the `useVideoEngine` hook provides a video-driven landmarking alternative. That hook fetches an `.mp4` gloss video, processes each frame through MediaPipe Tasks Vision `HolisticLandmarker` in the browser, and applies the resulting landmarks to the same VRM rigging pipeline. The system therefore supports both precomputed-pose playback and video-driven landmarking through a shared animation interface.

For architectural consistency, the coordinate-system design decision is documented in Chapter 3 (Section 3.2.2) and implemented unchanged in this runtime path.

4.3.6 Avatar Rendering and Animation

This layer renders signing motion in real time and keeps playback stable across desktop and mobile browsers. For each frame, landmark inputs are converted into avatar motion and applied continuously in the render loop.

Users can switch between close-up, full-body, and 3D world camera views depending on readability needs. A skeleton display mode provides a simplified landmark view, and a lightweight stats overlay supports runtime debugging during development.

4.3.6.1 Kalidokit Limitations in Practice

Kalidokit was a useful starting point for getting the avatar pipeline working, and it made it possible to turn pose landmarks into visible signing motion in the browser. In practice, though, it was not equally smooth or reliable for every sign. Simpler signs worked better, while more complex handshapes, palm orientation, contact points, and rest positions were less consistent. For this reason, DuoSign uses the VRM stack that proved most stable

with Kalidokit in the final implementation, but the limits of that setup are part of why smoother signing remains future work. A fuller discussion of these limitations appears in Chapter 6, while the appendix provides both the parameter-level matrix (Appendix G) and a short implementation findings matrix (Appendix H).

4.3.7 Video Export Pipeline

DuoSign implements client-to-server avatar recording that converts a live signing animation into a downloadable MP4 file. The implementation spans three interacting components: the `ExportVideoModal` React component, the `useCanvasRecorder` hook, and the `POST /api/export/video` FastAPI endpoint.

4.3.7.1 Client-Side Canvas Capture

When the user clicks *Export*, the export modal renders a hidden 16:9 avatar canvas and passes that canvas to the recorder hook. The browser then chooses the most suitable capture method automatically: browsers with standard canvas-stream support use `captureStream()` and `MediaRecorder` to record the animation directly, while Safari and iOS fall back to frame-by-frame JPEG capture for later server conversion. In both cases, the modal coordinates recording and playback so that capture starts slightly before animation begins and ends with a short delay after playback, preserving the full signed sequence in the exported result.

4.3.7.2 Server-Side FFmpeg Conversion

The captured media is uploaded to the export endpoint as multipart form data, where the backend converts it to a downloadable MP4 file. Standard browsers usually send a recorded WebM stream, while Safari-style fallback capture sends a sequence of JPEG frames; both inputs are normalised through the same FFmpeg-based packaging step with web-compatible H.264 settings. Because the service uses a bundled FFmpeg binary, deployment does not depend on a separate host-level FFmpeg installation.

4.3.7.3 User Experience

The modal displays three phases of cycling status messages: *preparing* (while the VRM loads), *composing* (during animation playback), and *packaging* (during server-side FFmpeg conversion). A continuous progress bar runs from 0% (start of recording) to 55% (end of playback) to 100% (download ready), giving users meaningful feedback without polling the server. The close button is disabled during recording and processing to prevent accidental cancellation. This implementation satisfies FR012.

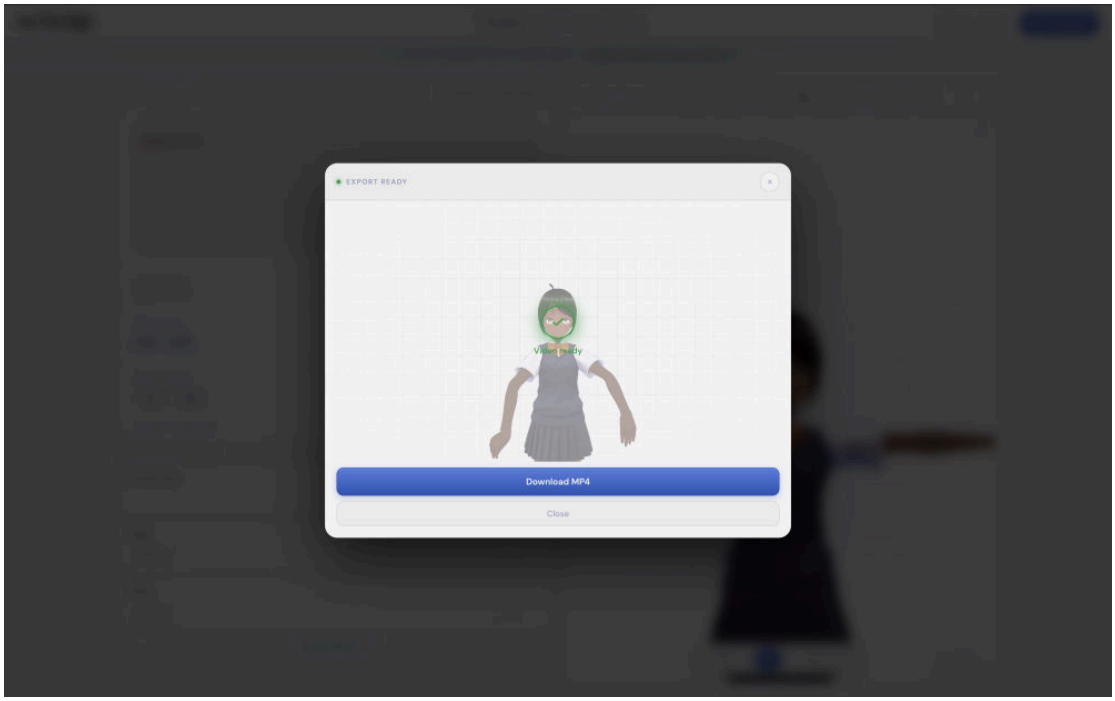


Figure 4.3: Export video modal UI showing the three-phase recording experience. Left: *preparing* phase while the VRM model loads into the hidden canvas. Centre: *composing* phase during avatar animation playback. Right: *packaging* phase during server-side FFmpeg conversion, with progress bar at 100% before download begins.

4.3.8 User Interfaces and Functionalities

The user interface provides the primary interaction surface for DuoSign. The design prioritises clarity, accessibility, and immediate feedback. The current implementation delivers four distinct pages within the Next.js web application: landing, translation, history, and settings.

4.3.8.1 Translation Interface

The main translation view adopts a two-panel layout: on desktop (≥1024 px), the `InputPanel` occupies the left column and the `AvatarPanel` occupies the right; on mobile, the panels stack vertically with the input panel sticky at the bottom. On submission, the frontend calls `POST /api/translate/stream` via SSE, receives an immediate rule-based gloss sequence, and may later receive an LLM-refined update over the same channel. Gloss tokens are rendered as `GlossChip` components with the active token highlighted during playback. Playback controls (play, pause, previous/next token, replay, speed cycle) and display mode controls (avatar, skeleton, hidden; close-up, full body, 3D world) are embedded within the `AvatarPanel` widget.

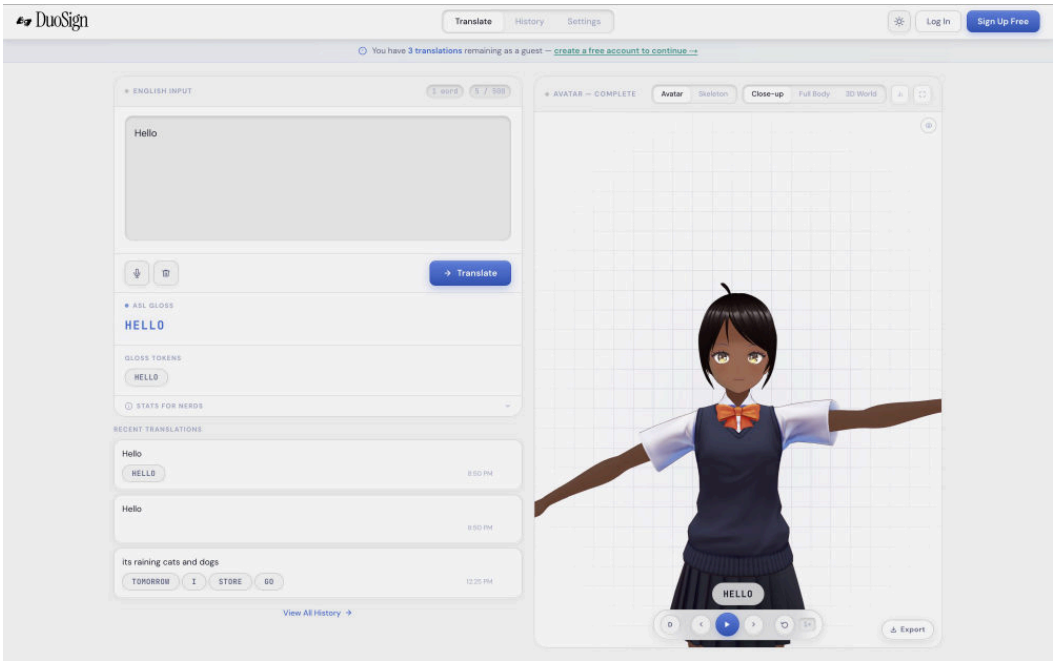


Figure 4.4: DuoSign translation interface on desktop (≥1024 px). The `InputPanel` (left column) contains the text input area, microphone button, submit control, and character counter. The `AvatarPanel` (right column) displays the Three.js VRM canvas with playback controls, display mode selectors, and export button. Active gloss tokens are highlighted in the chip row below the avatar.

4.3.8.2 Avatar Customisation

Users can switch avatar models and visual preferences directly from the interface. The selected model is loaded into the avatar canvas, while colour and appearance settings



Figure 4.5: DuoSign translation interface on mobile (<1024 px). Panels stack vertically with the avatar canvas occupying the upper portion of the viewport and the input panel fixed at the bottom, maintaining touch-target sizes at a minimum of 44×44 px.

are applied from the settings panel. A skeleton mode is also available for learners who want a simplified landmark view. Together, these features support FR005, FR009, FR010, and FR011.

4.3.8.3 Translation History

The history view is implemented entirely on the client side via the `useHistory` hook. It displays previously translated phrases grouped by date (Today, Yesterday, and named dates), with each entry showing input type icon, input text, timestamp, and metadata badges (gloss count, input type, exported status). Entries are expandable to reveal gloss token chips and action buttons: **Replay** (pushes saved text back to the translate route and auto-plays), **Export** (opens `ExportVideoModal` for any history entry), **Edit** (pre-fills input on translate page), and **Delete**. Filtering is available by date range (Today, This Week, This Month, All Time), input type (Typed, Voice), and free-text search. An "Exported" segment control limits the view to entries that have been exported as video. This satisfies FR014.

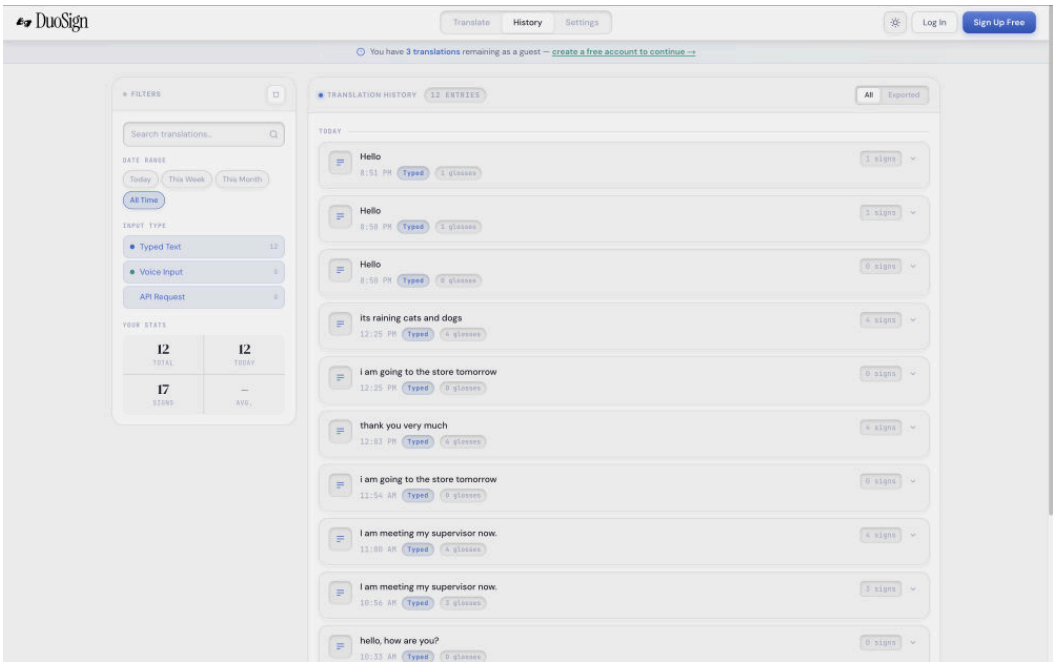


Figure 4.6: Translation history page. Entries are grouped by date (Today, Yesterday, named dates) and display input type icon, original text, timestamp, and metadata badges (gloss count, input method, exported flag). The expanded entry reveals the gloss token chip sequence and four action buttons: **Replay**, **Export** (reopens `ExportVideoModal`), **Edit** (pre-fills translate input), and **Delete**. The filter bar (top) supports date range, input type, search, and exported-only views.

4.3.8.4 Settings Interface

The settings page exposes application preferences through a local-only control surface organised into sidebar-navigated sections. A Save/Discard mechanism tracks dirty state and provides visual confirmation on save.

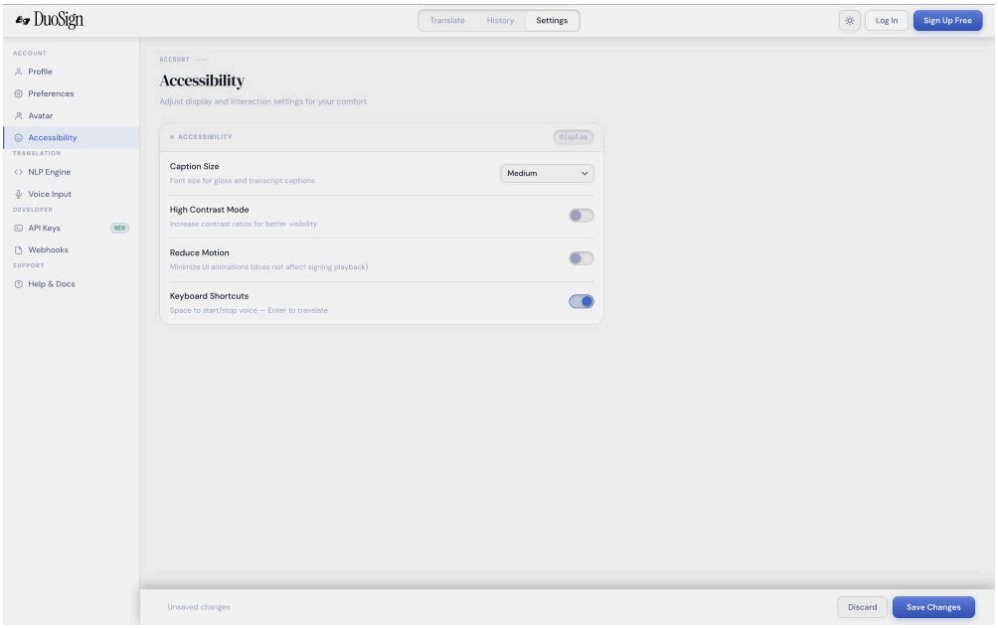


Figure 4.7: Settings page with sidebar navigation across three sections: **Account**, **Translation**, and **Developer**. Settings are persisted to `localStorage` via `SettingsProvider`; unsaved changes are tracked with a dirty-state indicator and a Save/Discard action bar.

4.4 Integration of Components

The integration of DuoSign's components follows the API contracts defined in Chapter 3. In practice, the frontend calls a small set of FastAPI endpoints that handle translation, voice transcription, vocabulary lookup, media retrieval, export, and health checks. The exact routes are documented in the architecture diagrams, so they are not repeated here.

The runtime flow is straightforward: user input is sent from the frontend to the backend, gloss output is returned and rendered as chips, and the playback layer fetches the required pose or video assets for animation. Kalidokit then maps landmark data to VRM bone rotations in the avatar renderer. This split keeps language processing, animation, and asset management loosely coupled, which supports NFR007 (scalability) and NFR008 (maintainability).

Chapter 5: Testing and Results

5.1 Introduction

This chapter reports how DuoSign was tested and evaluated across three areas: component-level software testing, accessibility compliance, and a small human-centred pilot study. Because the user-study dataset remained limited, the findings are presented as formative pilot results rather than as evidence from a full confirmatory study.

The evaluation was guided by three core hypotheses:

1. **H1:** DuoSign achieves a perceived sign recognition accuracy of at least 80% as reported by participants.
2. **H2:** A majority of participants prefer the animated avatar/the 2D skeleton over a plain text or gloss-token display.
3. **H3:** Participants recorded a System Usability Scale (SUS) score above 68 (above-average usability).

5.2 Component Testing

A total of 124 automated tests were run across the DuoSign backend and frontend, and all 124 passed.

Table 5.1: Automated component and unit test coverage summary.

Module	Layer	Tests	Test results
`text_to_gloss.py`	Backend	12	OK
`llm.py`	Backend	8	OK
API endpoint contracts	Backend	9	OK

Continued on next page

Module	Layer	Tests	Test results
`fingerspell.py`	Backend	4	OK
`voice.py`	Backend	5	OK
`export.py`	Backend	4	OK
`vocabulary.py`	Backend	2	OK
`/api/translations`	Frontend	11	OK
`/api/translations/[id]`	Frontend	7	OK
`/api/guest-usage`	Frontend	5	OK
`/api/pose/[gloss]`	Frontend	5	OK
`/api/pose-video/[gloss]`	Frontend	3	OK
`/api/testing/*`	Frontend	10	OK
`useTranslate` hook	Frontend	13	OK
Fingerspell token expansion	Frontend	6	OK
`useHistory` hook	Frontend	6	OK
`useGuestLimit` hook	Frontend	4	OK
`Button` component	Frontend	5	OK
`Modal` component	Frontend	5	OK
Total		**124**	**OK**

5.3 Accessibility Compliance

DuoSign was also audited against key WCAG 2.1 AA requirements using a combination of manual inspection, colour-contrast calculation, keyboard-path testing, and an `axe-core` 4.11 audit on the `/translate` view. The automated audit returned 0 violations, and the corresponding Lighthouse accessibility score was 100/100.

Table 5.2: Colour contrast checks against WCAG 2.1 AA.

Element	Pair	Ratio	Result
Primary text (dark mode)	`#edf0ff` on `#14161d`	15.93:1	Pass
Secondary text (dark mode)	`#8b94b8` on `#14161d`	6.05:1	Pass
Tertiary text (dark mode)	`#737fa0` on `#14161d`	4.54:1	Pass
Primary text (light mode)	`#1a1c28` on `#f3f3f3`	15.26:1	Pass
Accent button label	`#ffffff` on `#5b8ef0`	3.20:1	Pass (large/bold)
Error text	`#f87171` on `#14161d`	6.55:1	Pass
Success indicator	`#4ade80` on `#14161d`	10.37:1	Pass

Table 5.3: WCAG 2.1 AA accessibility summary for DuoSign.

Criterion	Status
1.4.3 Contrast (minimum)	Pass
2.1.1 Keyboard	Pass
2.4.7 Focus visible	Pass
4.1.2 Name, Role, Value	Pass
2.5.5 Target size	Pass
1.4.11 Non-text contrast	Pass

Overall, these checks suggest that the prototype meets the baseline WCAG 2.1 AA requirements examined here and was accessible enough for further user testing.

5.4 User Study Design

5.4.1 Ethical Approval and Participants

The study received IRB approval from Ashesi University prior to participant recruitment and data collection. Following approval, demo access links were shared with prospective participants, who were invited to interact with the system and complete a post-session survey.

In total, 36 participants were registered in the study dataset. However, only 5 post-session surveys were submitted and were usable for analysis. This low response rate meant that the findings from the user study were treated as formative pilot results rather than as evidence for broad generalisation.

5.4.2 Instruments

The evaluation used a small set of complementary instruments to capture both numeric and open-ended responses. The System Usability Scale (SUS) was used to measure perceived usability through a standard questionnaire. Likert-scale items captured views on avatar naturalness, output clarity, and transition smoothness. Open-ended questions gave participants space to comment on usability, visual quality, and possible real-world uses of the system.

Taken together, these instruments made it possible to assess not just whether participants found DuoSign usable, but also how they experienced the visual output and where they thought it might be useful.

5.4.3 Technical Performance

Table 5.4: Technical performance summary.

Component	**Metric**	**Target**	**Observed**
Text-to-Gloss (rule)	Mean latency	<50 ms	2.5 ms
	Cache hit latency	<5 ms	< 1 ms
LLM fallback	Timeout bound	3,000 ms	3,000 ms
	Mean API latency	—	2,950 ms
WLASL Extraction	Corpus coverage	100%	100%
	Throughput	—	2.6 vid/s
Avatar Renderer	Frame rate	60 FPS	60 FPS (display-synced; headless: 28–115)
	End-to-end latency	—	~1,200 ms cold / ~370 ms cached
Backend API	Test pass rate	100%	100% (46/46)

Browser-facing performance stayed at the display-synchronised target of 60 FPS. End-to-end latency from pressing *Translate* to first visible animation was about 1.2 s cold and 370 ms cached, with remote sign-file retrieval as the main bottleneck.

5.4.4 System Usability Scale

SUS scores were computed using the standard formula (odd items: score − 1; even items: 5 − score; sum × 2.5).

Table 5.5: Overall SUS results.

Mean SUS	SD	Band
73.50	15.47	Good

The mean SUS score of 73.50 is above the standard benchmark of 68 for above-average usability. The five returned SUS scores were 50.0, 77.5, 90.0, 67.5, and 82.5. The scores vary, but they still point to a generally positive usability result.

5.4.5 Expert Sign Review

To complement the participant survey data, an expert domain evaluation, informed by heuristic evaluation principles [27, 41], was carried out with the accessibility team at Ashesi University. Their feedback was qualitative and centred on how easy the avatar output was to interpret; no participant-level recognition percentage was computed in this pilot study. They responded well to the idea behind the project, the overall system design, and the inclusion of the 2D skeleton as a supporting visual aid. Their main concern, however, was the animated avatar itself. They found its signing harder to follow and often relied on the 2D skeleton to confirm the intended sign. Put simply, the overall concept was well received, but the avatar remained the weakest part of the current output pipeline.

5.4.6 Avatar Quality

Participant ratings offer a little more detail on how the avatar was perceived. Table 5.6 summarises the Likert-scale ratings for avatar naturalness, clarity, and transition smoothness, and helps place the qualitative feedback above in context.

Table 5.6: Ratings from returned post-session surveys (1–5 scale).

Dimension	Mean	SD
Avatar naturalness	3.25	1.26
Clarity of avatar output	3.00	1.22
Sign transition smoothness	2.60	1.82

5.5 User Acceptance

User acceptance was assessed using SUS scores, willingness-to-use responses, and short free-text comments from the returned hearing-participant surveys. Of those who responded, 60% said they would probably or definitely use DuoSign in a real setting. This suggests cautious but positive acceptance, though the sample is too small and uneven to support broader claims about the target population.

5.6 User Feedback

The returned free-text survey comments, together with the single stored feedback-widget submission, pointed to a few clear themes:

5.6.1 Positive Feedback

- **Interface quality:** Respondents described the interface as clean and easy to navigate. One respondent wrote, “IT’S SUCH A LOVELY UI... it’s clean and smooth”.
- **Learning value:** Another theme was the system’s potential for self-paced sign-language learning. One respondent stated, “I really appreciate that I can use it to learn sign language at my own pace”.
- **Token and gloss visibility:** Users valued the explicit breakdown of words and letters shown by the interface. One respondent noted, “I like how the breakdown of the words and letters is visible”.

5.6.2 Constructive Feedback

- **Avatar motion quality:** The most common criticism concerned the readability and fluidity of the avatar. Representative comments included “The avatar’s movements can be a bit difficult to understand” and “The animated avatar is not as fluid so her signing is not really understandable”.

- **Latency and stability:** Multiple comments pointed to response time as an immediate issue. One survey response said, “It works slowly and kept freezing”.

5.6.3 Practical Use Contexts Identified

Hearing respondents identified the following real-world contexts in which DuoSign could be practically useful:

- sign language learning and self-study;
- lightweight browser-based communication support;
- classroom or demonstration settings where gloss breakdown helps explain the translation process.

5.7 Discussion

5.7.1 Hypothesis Evaluation

Overall, the evidence offers mixed support for the study hypotheses.

- **H1:** This hypothesis was not confirmed. The pilot study did not produce a quantitative perceived-recognition percentage against the 80% target, and the qualitative feedback indicated that the avatar was harder to interpret than expected.
- **H2:** This hypothesis was partially supported. Visual output was clearly valued, but the 2D skeleton appeared to be more reliable than the animated avatar for sign verification.
- **H3:** This hypothesis was supported within the pilot sample. The mean SUS score of 73.50 was above the standard benchmark of 68, indicating above-average usability.

These results suggest that DuoSign works best, for now, as a usable and technically functional prototype. The clearest area for improvement is avatar quality, especially if stronger claims are to be made about real-world communication effectiveness.

5.7.2 Limitations

The findings in this chapter should be read as pilot results rather than as definitive evidence of real-world performance.

A fuller discussion of the methodological, technical, and communication-related limitations is given in Chapter 6, including the implications of the small sample size, the prototype status of the system, and the gap between technical correctness and practical signing effectiveness.

5.8 Analysis of Test Results

Overall, the results show that DuoSign is a usable and technically sound prototype. It performed well in component testing, met the audited WCAG 2.1 AA accessibility criteria, and achieved an encouraging SUS score. The main weakness was the avatar, especially in terms of clarity and smoothness. So while DuoSign has a solid technical base, it still needs refinement before stronger claims can be made about real-world communication effectiveness.

Chapter 6: Conclusions and Recommendations

DuoSign met its core objectives. This chapter discusses the main limitations, outlines directions for future work, and gives an overall assessment of the project.

6.1 Limitations and Challenges

Although DuoSign met its core objectives as a research prototype, the results in Chapter 5 also show that the system still has some important limitations. Some of these are technical, while others come from the design and scope of the evaluation itself. Together, they affect both the quality of the signing output and the strength of the claims that can reasonably be made from the pilot study.

- **Direct pose-driving constraints:** As discussed in Chapter 3 (Section 3.2.2), direct playback of pose files on the avatar was not viable because of frame-timing incompatibilities with Kalidokit. This meant that the intended animation pipeline could not be used in its simplest form and that the final implementation had to rely on a more constrained workaround, which reduced motion fidelity and limited control over avatar behaviour.

- **ASL-only dataset scope:** The WLASL corpus is ASL-specific, so DuoSign currently produces ASL-based output regardless of the signing background of the user. This limits the system's relevance for users in Ghana and other contexts where the dominant signed language is not ASL, since vocabulary, grammar, and sign usage may differ substantially.

- **Avatar solver artefacts:** The Kalidokit solver introduced visible artefacts, including quaternion instabilities, distortions from source-video camera-angle variation, and reduced handshape clarity when the hands were partially occluded. These issues affected the readability of the avatar output and were reflected in both participant feedback and expert review.

- **Vocabulary coverage limits:** Although fingerspelling provides a complete fallback strategy for out-of-vocabulary items, extended fingerspelled output is slower and harder to interpret than native lexical signing. The 11,980-clip WLASL corpus provides substantial coverage for a prototype, but it does not capture the full vocabulary needed for natural everyday communication, including many names, specialised terms, and longer expressions.

- **Small pilot-study sample:** Although 36 participants were registered in the dataset, only 5 usable post-session surveys were returned. This means that the findings should be treated as formative pilot evidence rather than something that can be generalised too widely. The small sample size limits how much can be claimed about usability, preference, and perceived sign-recognition accuracy.

- **Limited evidence for communication effectiveness:** The strongest findings on sign quality came from qualitative evidence, especially expert review and participant free-text comments, rather than from large-scale quantitative comprehension measures. This was useful for identifying the main weaknesses in the system, but it does not yet provide enough depth for stronger claims about consistent sign understanding across different users and contexts.

Taken together, these limitations suggest that DuoSign is best seen, for now, as a working research prototype rather than a finished communication system. The platform is technically coherent and usable, but the signing output, linguistic scope, and evaluation base still need more work before stronger claims about real-world deployment can be made.

6.2 Future Work

The next stage of the project should focus on the research and engineering work that would make the system more complete, more locally relevant, and easier to evaluate properly. The priorities below follow directly from the limitations identified during implementation and in the pilot study.

- **Move to a stronger avatar pipeline:** One obvious next step is to test alternative avatar rigs or a purpose-built inverse-kinematics solver in place of Kalidokit. In the current system, Kalidokit's landmark-to-bone mapping introduced artefacts on more complex handshapes and often required manual quaternion correction. A better rigging stack, or a solver trained on sign-language motion data, would likely improve signing quality more directly.

- **Model non-manual grammar more fully:** The animation stack should be extended to handle facial grammar more explicitly, including eyebrow movement, eye gaze, mouth morphemes, and head tilt. This is one of the clearest remaining gaps in intelligibility for fluent signers [10, 28].

- **Localise beyond ASL:** The project should also move beyond an ASL-only dataset base by extending toward Ghanaian Sign Language (GhSL). This fits the original motivation of the work, which was always tied to low-resource and locally meaningful deployment contexts [37].

- **Expand and repeat the evaluation:** Larger and better balanced studies are still needed, ideally with Deaf and hard-of-hearing participants across different fluency levels. Future evaluations should report sentence-level comprehension, task-completion outcomes, and longer-term usability. Publishing reusable protocols and anonymised evaluation artefacts would also make the work easier to reproduce.

- **Improve robustness and domain coverage:** The translation pipeline still needs work on harder cases such as idioms, domain-specific terms, code-switching, and long compounds. It would also help to add confidence-aware fallback strategies so users can tell when the system is confident and when the output is only approximate.

6.3 Requirements Compliance and Objective Attainment

DuoSign set out to do three main things: provide sign-first access in a regular web browser, translate English into ASL gloss, and present that output through an animated avatar that people could actually use.

By that standard, the project was largely successful. The browser-based workflow was implemented, the translation pipeline works, and the system supports the main user features discussed earlier in the thesis, including typed and voice input, playback controls, export, and history. The avatar output was also usable, especially for simpler signs such as HELLO, although, as Chapter 5 showed, motion quality was not equally smooth across all signs.

Where the project remains incomplete is in the areas already discussed under limitations. DuoSign does not yet handle richer non-manual grammar well enough, and it has not yet been localised beyond ASL into languages such as GhSL. So while the main system goals were achieved, some of the larger long-term aims still remain future work rather than finished deliverables.

6.4 Conclusion

DuoSign is a useful step toward accessible, hardware-free English-to-ASL communication technology. By combining a hybrid NLP translation pipeline, a pre-processed WLASL pose corpus, and a browser-native 3D avatar renderer, the system shows that real-time sign-language generation can work as a standard web application without specialist infrastructure or device requirements. The evaluation reported in this thesis is still pilot-level, so the main contribution at this stage is the implemented system itself and the reproducible baseline it provides for future studies.

There are still clear weaknesses, especially around avatar expressiveness, dataset language scope, and direct pose-driving constraints. Even so, DuoSign offers a functional open-source baseline for accessible sign-language technology that could be adapted to other signed languages and deployment contexts. The future work identified in this chapter gives a practical path for improving the system so that it can better serve the communicative needs of Deaf communities in Ghana and beyond.

References

[1] IEEE Standard Glossary of Software Engineering Terminology.

[2] Allen, J., and Foulds, R. An approach to animating sign language: a spoken english to sign english translator system. In *IEEE 30th Annual Northeast Bioengineering Conference, 2004. Proceedings of the* (Springfield, MA, 2004), IEEE, pp. 43–44.

[3] Ben Yahia, N., and Jemni, M. Animating signing avatar using descriptive sign language. In *2013 International Conference on Electrical Engineering and Software Applications* (Hammamet, Tunisia, Mar. 2013), IEEE, pp. 1–5.

[4] Bragg, D., Koller, O., Bellard, M., Berke, L., Boudreault, P., Braffort, A., Caselli, N., Huenerfauth, M., Kacorri, H., Verhoef, T., Vogler, C., and Ringel Morris, M. Sign Language Recognition, Generation, and Translation: An Interdisciplinary Perspective. In *Proceedings of the 21st International ACM SIGACCESS Conference on Computers and Accessibility* (Pittsburgh PA USA, Oct. 2019), ACM, pp. 16–31.

[5] Brown, S. C4 model for visualising software architecture, 2018.

[6] Cao, Z., Hidalgo, G., Simon, T., Wei, S.-E., and Sheikh, Y. OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields. *IEEE Transactions on Pattern Analysis and Machine Intelligence 43*, 1 (Jan. 2021), 172–186.

[7] Choi, H.-T., Kim, D.-H., and Chang, W.-D. Recognition of hand gestures with different prior postures using EMG signals. *Journal of Internet of Things and Convergence 9*, 6 (Dec. 2023), 51–56.

[8] Delorme, M., Filhol, M., and Braffort, A. Animation Generation Process for Sign Language Synthesis. In *2009 Second International Conferences on Advances in Computer-Human Interactions* (Cancun, Mexico, Feb. 2009), IEEE, pp. 386–390.

[9] Duarte, A., Palaskar, S., Ventura, L., Ghadiyaram, D., DeHaan, K., Metze, F., Torres, J., and Giro-i Nieto, X. How2Sign: A Large-scale Multimodal Dataset for Continuous American Sign Language, Apr. 2021. arXiv:2008.08143 [cs].

[10] Emmorey, K. New Perspectives on the Neurobiology of Sign Languages. *Frontiers in Communication 6* (Dec. 2021), 748430.

[11] Eunice, J., J, A., Sei, Y., and Hemanth, D. J. Sign2Pose: A Pose-Based Approach for Gloss Prediction Using a Transformer Model. *Sensors 23*, 5 (Mar. 2023), 2853.

[12] Grieve-Smith, A. B. SignSynth: A Sign Language Synthesis Application Using Web3D and Perl. In *Gesture and Sign Language in Human-Computer Interaction*, G. Goos, J. Hartmanis, J. Van Leeuwen, I. Wachsmuth, and T. Sowa, Eds., vol. 2298. Springer Berlin Heidelberg, Berlin, Heidelberg, 2002, pp. 134–145. Series Title: Lecture Notes in Computer Science.

[13] Gu, Y., Oku, H., and Todoh, M. American Sign Language Recognition and Translation Using Perception Neuron Wearable Inertial Motion Capture System. *Sensors 24*, 2 (Jan. 2024), 453.

[14] Gu, Y., Zheng, C., Todoh, M., and Zha, F. American Sign Language Translation Using Wearable Inertial and Electromyography Sensors for Tracking Hand Movements and Facial Expressions. *Frontiers in Neuroscience 16* (July 2022), 962141.

[15] Hanke, T., Schmaling, C., Popescu, H., Glauert, J., Elliott, R., Kennaway, R., Tryggvason, J., Frowein, H., Verlinden, M., Zwitserlood, I., Gybels, G., Thomas, N., Henley, M., Anderson, T., Davies, B., Wells, M., and Ehrhardt, U. Animating Sign Language: The eSIGN Approach. Tech. rep., eSIGN (Essential Sign Language Information on Government Networks), Aug. 2004.

[16] Harel, D. Statecharts: a visual formalism for complex systems. *Science of Computer Programming 8*, 3 (June 1987), 231–274.

[17] Holden, D., Saito, J., and Komura, T. A deep learning framework for character motion synthesis and editing. *ACM Transactions on Graphics 35*, 4 (July 2016), 1–11.

[18] Hou, J., Li, X.-Y., Zhu, P., Wang, Z., Wang, Y., Qian, J., and Yang, P. SignSpeaker: A Real-time, High-Precision SmartWatch-based Sign Language Translator. In *The 25th Annual International Conference on Mobile Computing and Networking* (Los Cabos Mexico, Aug. 2019), ACM, pp. 1–15.

[19] Jacobson, I. *Object-oriented software engineering: a use case driven approach*. ACM Press ; Addison-Wesley Pub, [New York] : Wokingham, Eng. ; Reading, Mass, 1992.

[20] Kim, I.-H., and Jung, I.-H. A Study on Korea Sign Language Motion Recognition Using OpenPose Based on Deep Learning. *Journal of Digital Contents Society 22*, 4 (Apr. 2021), 681–687.

[21] Kimmelman, V., Imashev, A., Mukushev, M., and Sandygulova, A. Eyebrow position in grammatical and emotional expressions in Kazakh-Russian Sign Language: A quantitative study. *PLOS ONE 15*, 6 (June 2020), e0233731.

[22] Kipp, M., Heloir, A., and Nguyen, Q. Sign Language Avatars: Animation and Comprehensibility. In *Intelligent Virtual Agents*, D. Hutchison, T. Kanade, J. Kittler, J. M. Kleinberg, F. Mattern, J. C. Mitchell, M. Naor, O. Nierstrasz, C. Pandu Rangan, B. Steffen, M. Sudan, D. Terzopoulos, D. Tygar, M. Y. Vardi, G. Weikum, H. H. Vilhjálmsson, S. Kopp, S. Marsella, and K. R. Thórisson, Eds., vol. 6895. Springer Berlin Heidelberg, Berlin, Heidelberg, 2011, pp. 113–126. Series Title: Lecture Notes in Computer Science.

[23] Li, D., Opazo, C. R., Yu, X., and Li, H. Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison, 2019. Version Number: 2.

[24] Li, D., Yu, X., Xu, C., Petersson, L., and Li, H. Transferring Cross-domain Knowledge for Video Sign Language Recognition, 2020. Version Number: 2.

[25] Moradi, M., Kannan, D. D., Asadianfam, S., Kolivand, H., and Aldhaibani, O. A Review of Sign Language Systems. In *2023 16th International Conference on*

Developments in eSystems Engineering (DeSE) (Istanbul, Turkiye, Dec. 2023), IEEE, pp. 200–205.

[26] Moryossef, A., Müller, M., Göhring, A., Jiang, Z., Goldberg, Y., and Ebling, S. An Open-Source Gloss-Based Baseline for Spoken to Signed Language Translation, May 2023. arXiv:2305.17714 [cs].

[27] Nielsen, J., and Molich, R. Heuristic evaluation of user interfaces. In *Proceedings of CHI 1990* (Seattle, 1990), ACM, pp. 249–256.

[28] Nyst, V. Sign languages in West Africa. In *Sign Languages*, D. Brentari, Ed., 1 ed. Cambridge University Press, May 2010, pp. 405–432.

[29] Organization, W. H. Deafness and Hearing Loss, Mar. 2026.

[30] Papastratis, I., Dimitropoulos, K., and Daras, P. Continuous Sign Language Recognition through a Context-Aware Generative Adversarial Network. *Sensors 21*, 7 (Apr. 2021), 2437.

[31] Sandler, W. The Phonological Organization of Sign Languages. *Language and Linguistics Compass 6*, 3 (Mar. 2012), 162–182.

[32] Schlenker, P. Sign Language Semantics, Sept. 2024.

[33] Sehyr, Z. S., Caselli, N., Cohen-Goldberg, A. M., and Emmorey, K. The ASL-LEX 2.0 Project: A Database of Lexical and Phonological Properties for 2,723 Signs in American Sign Language. *The Journal of Deaf Studies and Deaf Education 26*, 2 (Mar. 2021), 263–277.

[34] Sommerville, I. *Software engineering*, tenth edition ed. Pearson, Boston, 2016.

[35] Stoll, S., Camgoz, N. C., Hadfield, S., and Bowden, R. Text2Sign: Towards Sign Language Production Using Neural Machine Translation and Generative Adversarial Networks. *International Journal of Computer Vision 128*, 4 (Apr. 2020), 891–908.

[36] Tateno, S., Liu, H., and Ou, J. Development of Sign Language Motion Recognition System for Hearing-Impaired People Using Electromyography Signal. *Sensors 20*, 20 (Oct. 2020), 5807.

[37] Vandeghinste, V., De Sisto, M., Kopf, M., Schulder, M., Brosens, C., Soetemans, L., Omardeen, R., Picron, F., Van Landuyt, D., Murtagh, I., Avramidis, E., and De Coster, M. Report on Europe's Sign Languages (ELE D1.40). Tech. rep., Zenodo, June 2023. Version Number: 1.0.

[38] Wang, F., Zhao, S., Zhou, X., Li, C., Li, M., and Zeng, Z. An Recognition–Verification Mechanism for Real-Time Chinese Sign Language Recognition Based on Multi-Information Fusion. *Sensors 19*, 11 (May 2019), 2495.

[39] Wang, Z., Zhao, T., Ma, J., Chen, H., Liu, K., Shao, H., Wang, Q., and Ren, J. Hear Sign Language: A Real-time End-to-End Sign Language Recognition System. *IEEE Transactions on Mobile Computing* (2020), 1–1.

[40] Yandex Front-End Architecture Team. Feature-Sliced Design (Overview), 2023.

[41] Yeratziotis, A., and Zaphiris, P. A Heuristic Evaluation for Deaf Web User Experience (HE4DWUX). *International Journal of Human–Computer Interaction 34*, 3 (2017).

[42] Yi, W., Zhang, Y., Meng, L., Cao, C., Lin, X., and Gao, S. CCC: cross-modal contrastive creator for end-to-end sign language generation. *Journal of King Saud University Computer and Information Sciences 38*, 1 (Jan. 2026), 2.

[43] Ylitie, H., Karras, T., and Laine, S. Efficient incoherent ray traversal on GPUs through compressed wide BVHs. In *Proceedings of High Performance Graphics* (Los Angeles California, July 2017), ACM, pp. 1–13.

[44] Zuo, R., Potamias, R. A., Ververas, E., Deng, J., and Zafeiriou, S. Signs as Tokens: A Retrieval-Enhanced Multilingual Sign Language Generator, 2024. Version Number: 3.

Appendices

A AI Use Declaration

A.1 Declaration Form

Project Title:	DUOSIGN: A GLOSS-BASED APPROACH TO AUTOMATED ASL GENERATION
Student:	Nana Kwaku Amoako
Student ID:	70032026
Degree Program:	Computer Science
Supervisor:	Kwabena Bamfo
Project Type:	Applied (Individual)
Submission Stage:	Final

I declare that the information below accurately captures my AI tool usage throughout this project. I accept full responsibility for the integrity and accuracy of all work submitted.

Student Signature: _____ Date: _____

Supervisor Acknowledgement: _____ Date: _____

A.2 AI Tools and Usage Summary

Table 1 summarises all AI tools used during DuoSign's development and their primary purpose in the project.

Table 1: AI tools used in DuoSign development.

Tool & Provider	**Primary Use in Project**
Claude Code (Anthropic, Opus/Sonnet 4)	Frontend component development, React hooks, TypeScript, animation pipeline, and canvas recording for the Next.js 14 implementation.
Codex / ChatGPT (OpenAI, GPT-5.4)	Code review, testing strategy, QA, debugging, and production-readiness checks across backend and frontend modules.

Tool & Provider	Primary Use in Project
ChatGPT (OpenAI, GPT-4o)	Diagram ideation and generation support (Mermaid/PlantUML) for architecture, activity, and deployment representations.
Anara (Writing assistant)	Thesis writing support for grammar, clarity, sentence structure, and consistent academic tone.
Grammarly (Grammarly Inc.)	Grammar and spelling checks during final proofreading of thesis chapters.

B Project Structure

B.1 Repository Layout and AI Contribution Per File

The DuoSign repository follows a monorepo structure with separate backend and frontend directories. The frontend uses the Feature Sliced Design (FSD) architecture.

All frontend components were built with AI assistance based on designs created manually in Figma; minor text-copy adjustments were made by hand.

Table 2: Project files with one-line descriptions and AI contribution type.

File Name	One-Line Description	Contribution
Backend		
api/main.py	FastAPI application entry point and endpoint registration.	Me Only
api/text_to_gloss.py	English-to-gloss translation pipeline.	AI + Me
api/fingerspell.py	Letter-by-letter fingerspelling fallback logic.	AI + Me
api/llm.py	LLM-based fallback translation module.	Me Only
api/vocabulary.py	Gloss vocabulary lookup and normalization helpers.	AI + Me
api/voice.py	Speech-to-text endpoint integration.	AI + Me

File Name	One-Line Description	Contribution
api/export.py	Video export API handling and orchestration.	AI + Me
wlasl_pose_pipeline.py	Offline WLASL pose extraction pipeline.	AI + Me
test_gloss.py	Interactive gloss testing script.	Me Only
data/glosses.tsv	WLASL-derived gloss vocabulary data file.	Me Only
Frontend		
app/page.tsx	Landing page implementation.	AI Only
app/translate/page.tsx	Main translation interface page.	AI Only
app/history/page.tsx	Translation history page.	AI Only
app/settings/page.tsx	User settings page.	AI + Me
app/auth/login/page.tsx	Login page UI and behavior.	AI + Me
app/auth/register/page.tsx	Registration page UI and behavior.	AI + Me
app/layout.tsx	Root app layout and shared wrappers.	AI Only
app/globals.css	Global styling tokens and theme variables.	AI + Me
features/animate-avatar/model/useVideoEngine.ts	Video-to-avatar animation engine hook.	AI + Me
features/animate-avatar/model/usePosePlayer.ts	Binary pose playback hook.	AI + Me
features/animate-avatar/model/useAvatarRenderer.ts	Three.js avatar rendering hook.	AI Only
features/animate-avatar/model/useCanvasRecorder.ts	Canvas recording/export hook.	AI Only
features/animate-avatar/model/usePlayback.ts	Playback state and controls hook.	AI Only

File Name	One-Line Description	Contribution
features/animate-avatar/model/useVRM.ts	VRM model loading hook.	AI + Me
features/animate-avatar/ui/AvatarCanvas.tsx	3D avatar rendering component.	AI Only
features/animate-avatar/ui/AvatarSwitcher.tsx	Avatar/model switching UI.	AI Only
features/animate-avatar/ui/ExportVideoModal.tsx	Export progress modal component.	AI + Me
features/animate-avatar/ui/StatsForNerds.tsx	Debug metrics display component.	AI Only
features/translate-text/model/useTranslate.ts	Translation request and stream hook.	AI + Me
features/translate-text/ui/GlossDisplay.tsx	Gloss output display component.	AI Only
features/translate-text/ui/VoiceRecordingPane.tsx	Voice input and recording pane.	AI + Me
features/translate-text/ui/DebugStats.tsx	Translation debug panel.	AI Only
features/skeleton-viewer/ui/SkeletonCanvas.tsx	Skeleton rendering canvas component.	AI + Me
features/skeleton-viewer/model/useSkeletonPlayer.ts	Skeleton playback control hook.	AI + Me
features/skeleton-viewer/lib/skeletonRenderer.ts	Skeleton drawing utility logic.	AI + Me
widgets/avatar-panel/AvatarPanel.tsx	Avatar playback controls panel widget.	AI Only
widgets/input-panel/InputPanel.tsx	Text input panel widget.	AI Only

File Name	One-Line Description	Contribution
widgets/navigation-bar/NavigationBar.tsx	Navigation bar and mobile drawer.	AI Only
widgets/recent-translations/	Recent translation list widget module.	AI Only
widgets/guest-banner/GuestBanner.tsx	Guest usage notice banner.	AI Only
shared/ui/	Shared reusable UI component library.	AI Only
shared/hooks/	Shared custom hooks (theme, history, settings).	AI Only
shared/providers/	Shared React context providers.	AI Only
shared/api/	API client wrappers and request helpers.	AI Only
entities/	Shared TypeScript types and domain models.	AI Only
Infrastructure and Data		
db.sql	Supabase database schema and SQL setup.	Me Only

C System Comparison Table

Table 3 presents the extended comparison of DuoSign against representative text-to-sign production systems referenced in Chapter 2. It is placed in the appendix to keep the main requirements chapter focused while still preserving the full comparative detail.

Table 3: Comparison of DuoSign with existing sign language production systems.

Feature	**eSIGN** [15]	**Text2Sign** [35]	**Sign.MT** [26]	**DuoSign**
Animation Source	HamNoSys notation	Neural pose + GAN video	Lexicon pose stitching	Real video pose extraction (MediaPipe + Kalidokit)
Text-to-Gloss Method	Rule-based	Neural MT (BLEU-4: 16.34)	Lemmatiser + rules	Hybrid rule-based + LLM fallback (streaming SSE)
Target Sign Language	British SL, German SL, Dutch SL	German SL (PHOENIX-14T)	Swiss German SL, French SL, Italian SL	American SL
Open Source	Partial	No	Yes	Yes (full pipeline)
Avatar Technology	VRML avatar	GAN-generated video	Skeletal pose overlay	VRM 3D avatar (Three.js) with skeleton mode
Deployment	Desktop	Research prototype	Web demo	Web application + REST API + Voice input
Video Export	N/A	N/A	N/A	Client-side canvas capture, server-side FFmpeg MP4
Unknown Word Handling	Manual notation required	Limited vocabulary	Fingerspelling	Synonym lookup + fingerspelling fallback

D Related-Works Comparison Matrix

Table 4 moves the broader literature comparison from Chapter 1 into the appendix so that the main discussion can stay focused on the narrative analysis rather than a large comparative matrix.

Table 4: Comparison of representative sign-language systems.

System	Direction	Hardware	Open	Naturalness	Adaptability	Low-resource fit
SignSpeaker [18]	Sign→Text	Smartwatch + phone	No	N/A	Data-dependent	Low (labeled sentence data needed)
DeepSLR [39]	Sign→Speech	Dual IMU/EMG bands	No	N/A	Data-dependent	Low (specialized wearables + annotation)
Myo-based [14]	Sign→Text	Myo armband	Partial	N/A	Limited task scope	Low (isolated signs, proprietary device)
eSIGN [15]	Text→Sign	Standard PC	Partial	Low	High (notation-based)	Medium (requires HamNoSys expertise)
SignSynth [12]	Text→Sign	Standard PC + Web3D	Partial	Low	High (notation-based)	Medium (manual authoring burden)
Motion-capture avatars [8]	Text→Sign	Tracking studio	No	High	Low	Low (high cost and setup time)
Text2Sign [35]	Text→Sign	GPU server	No	Medium	Low (corpus-specific)	Low (parallel corpus required)
How2Sign pipeline [9]	Dataset + generation	RGB-D capture setup	Partial	Medium	Medium	Low (large annotated capture required)
CCC [42]	Text→Sign	GPU server	Yes	Medium	Low (data-intensive)	Low (dense annotations required)
Signs as Tokens [44]	Text→Sign	GPU server	Yes	Medium	Medium	Low (large multilingual corpora needed)
Moryossef et al. [26]	Text→Sign	Standard PC	Yes	Medium	High (modular)	Medium–High (gloss + video lexicon needed)
DuoSign (ours)	**Text→Sign**	**Browser + standard webcam**	**Yes**	**Medium–High**	**High (modular, video-grounded)**	**High (no studio or GPU required)**

E User Study Instruments

E.1 Post-Session Questionnaire (SUS and Supplementary Items)

Table 5 reproduces the post-session questionnaire used during DuoSign testing. It combines one overall adoption question, three avatar-quality items, two open-ended feedback prompts, and the standard 10-item System Usability Scale (SUS).

Table 5: DuoSign post-session questionnaire and SUS items.

Type	**Question / Prompt**	**Response Format**
Use	Would you use DuoSign?	5-point choice: Definitely not / Probably not / Not sure / Probably yes / Definitely yes
Avatar	How natural did the avatar's signing look?	5-point rating from Poor (1) to Excellent (5)
Avatar	Could you understand what the avatar was signing?	5-point rating from Poor (1) to Excellent (5)
Avatar	Did the avatar movement feel smooth?	5-point rating from Poor (1) to Excellent (5)
Open	What did you like most about DuoSign?	Free-text response
Open	What needs the most improvement?	Free-text response
SUS-1	I think that I would like to use DuoSign frequently.	5-point Likert: Strongly Disagree (1) to Strongly Agree (5)
SUS-2	I found DuoSign unnecessarily complex.	5-point Likert: Strongly Disagree (1) to Strongly Agree (5)

Type	Question / Prompt	Response Format
SUS-3	I thought DuoSign was easy to use.	5-point Likert: Strongly Disagree (1) to Strongly Agree (5)
SUS-4	I think that I would need the support of a technical person to use DuoSign.	5-point Likert: Strongly Disagree (1) to Strongly Agree (5)
SUS-5	I found the various functions in DuoSign were well integrated.	5-point Likert: Strongly Disagree (1) to Strongly Agree (5)
SUS-6	I thought there was too much inconsistency in DuoSign.	5-point Likert: Strongly Disagree (1) to Strongly Agree (5)
SUS-7	I would imagine that most people would learn to use DuoSign very quickly.	5-point Likert: Strongly Disagree (1) to Strongly Agree (5)
SUS-8	I found DuoSign very cumbersome to use.	5-point Likert: Strongly Disagree (1) to Strongly Agree (5)
SUS-9	I felt very confident using DuoSign.	5-point Likert: Strongly Disagree (1) to Strongly Agree (5)
SUS-10	I needed to learn a lot of things before I could get going with DuoSign.	5-point Likert: Strongly Disagree (1) to Strongly Agree (5)

F Application Interface Screenshots

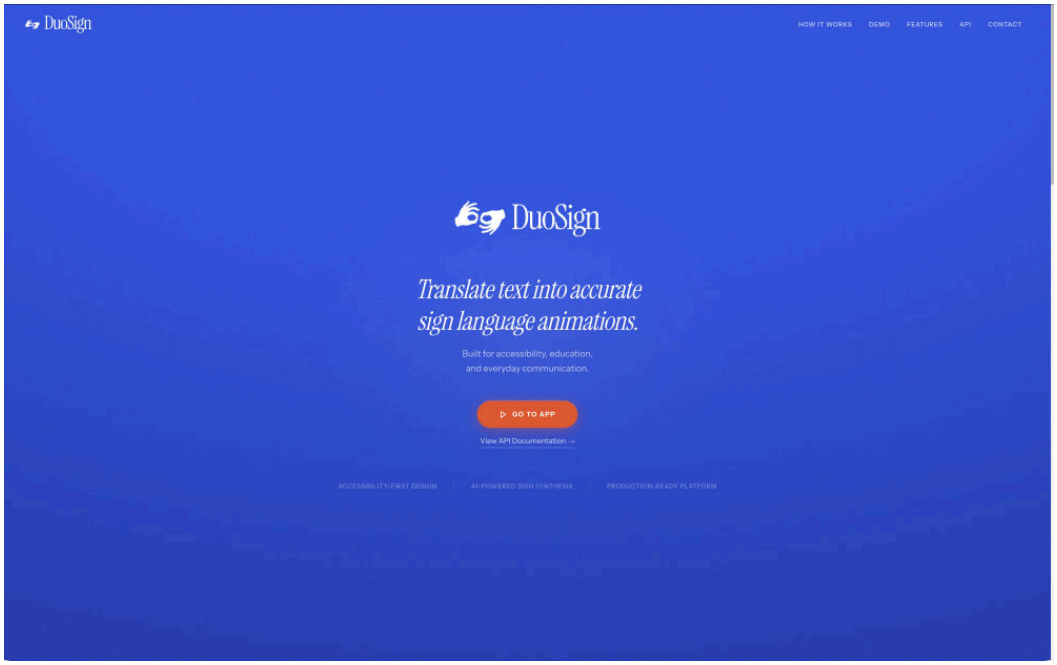


Figure 1: DuoSign landing page.

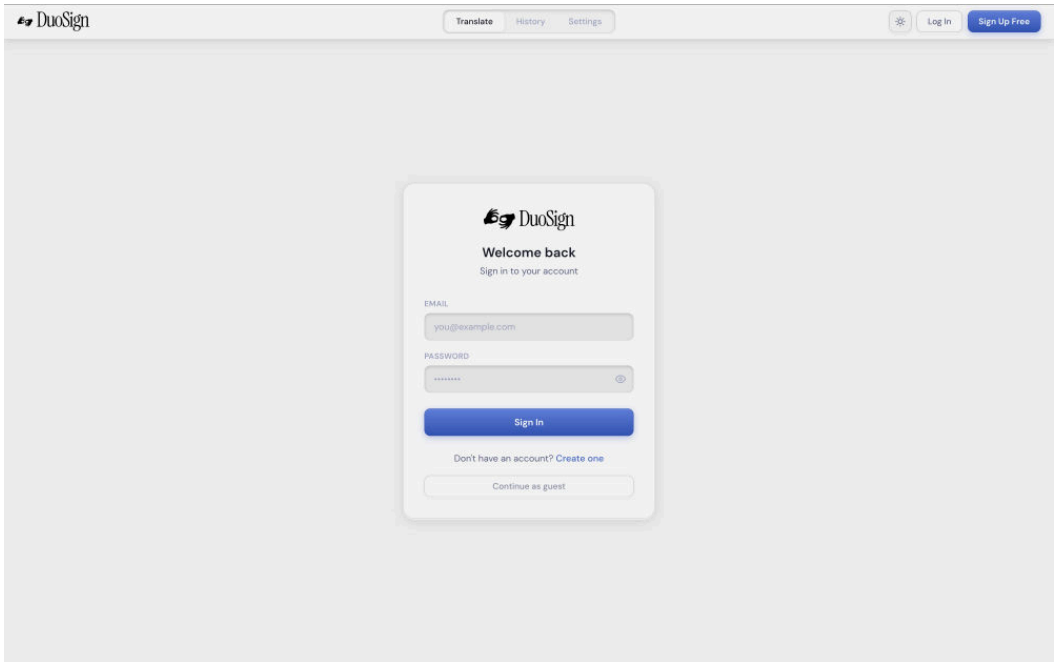


Figure 2: User login page.

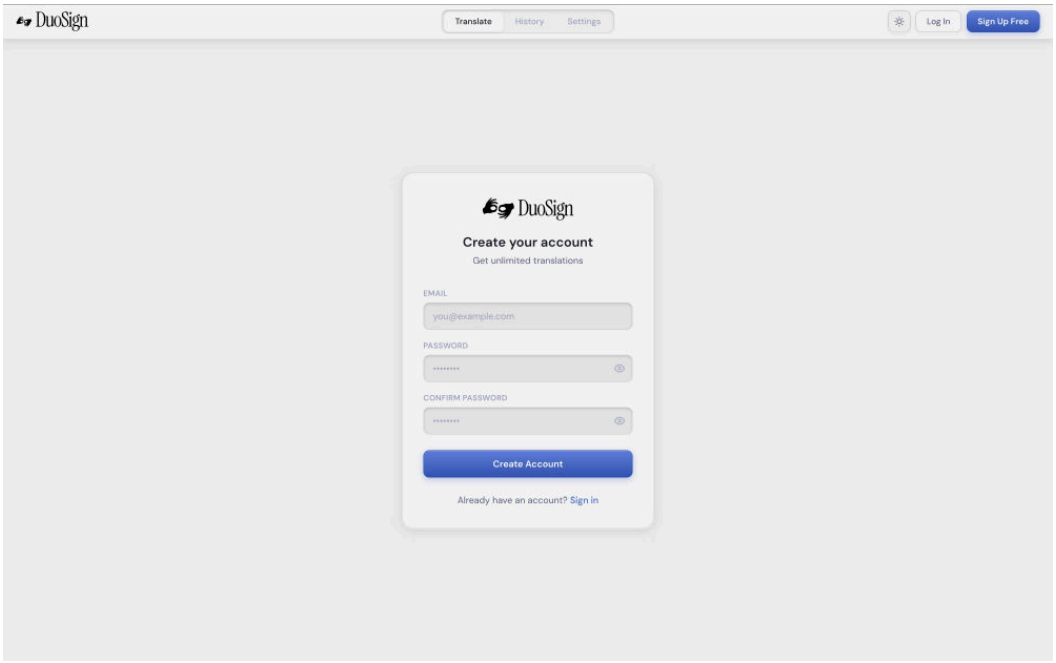


Figure 3: User sign-up page.

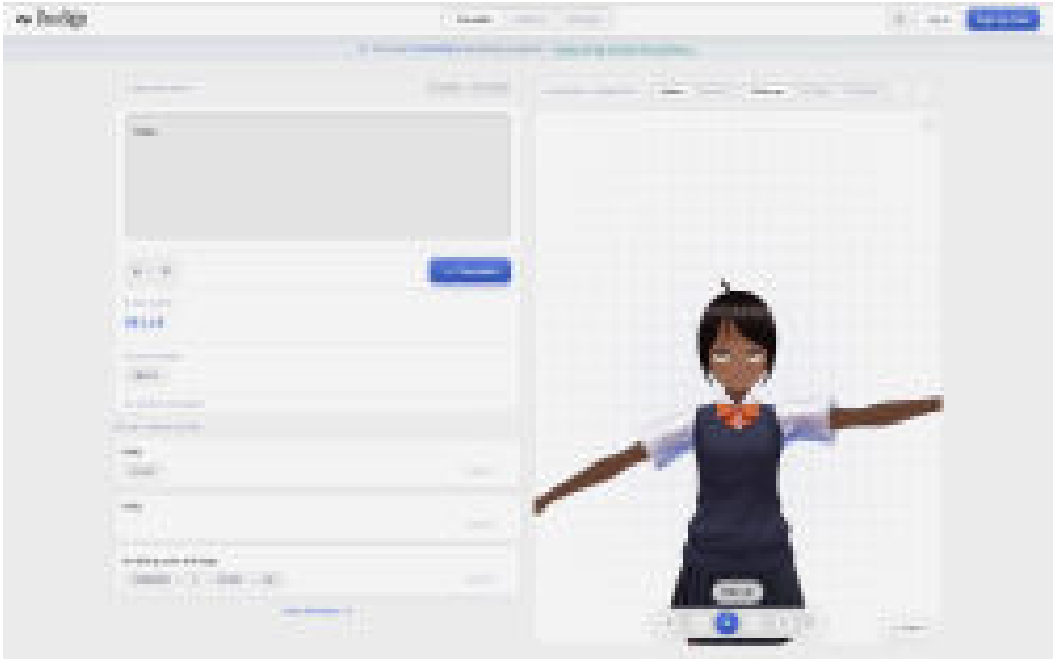


Figure 4: Main translation workspace with avatar output.

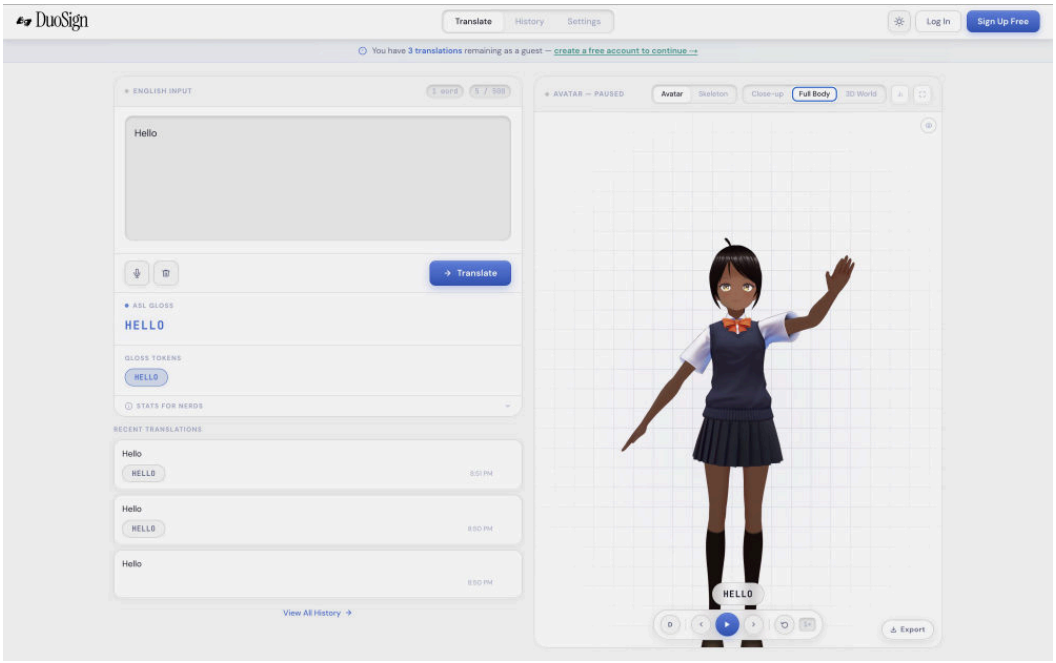


Figure 5: Full-body avatar display mode.

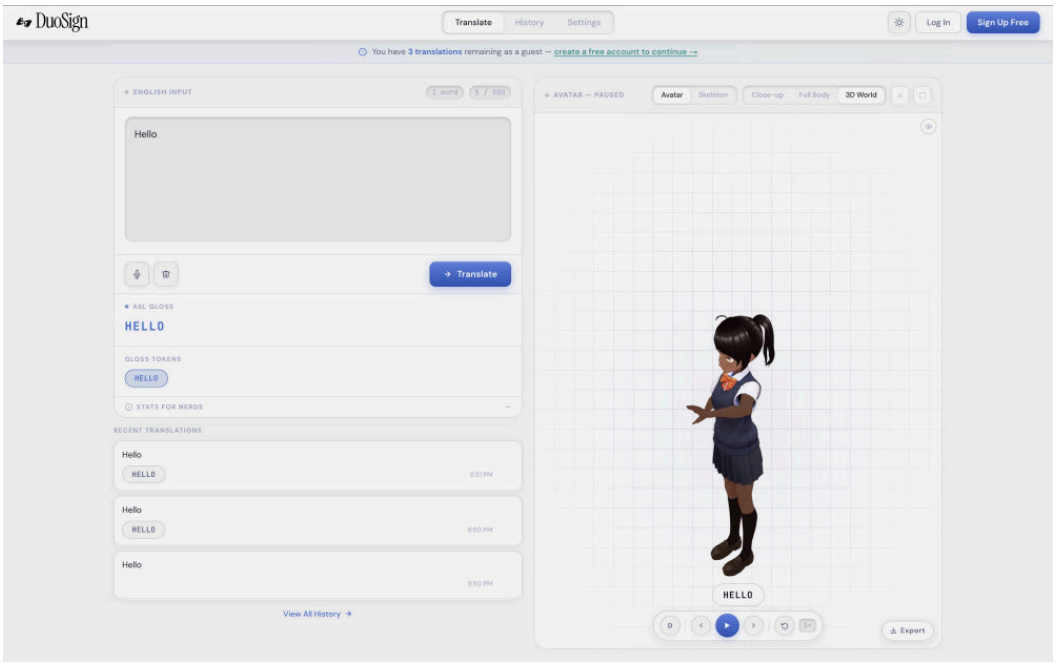


Figure 6: 3D world view of the avatar environment.

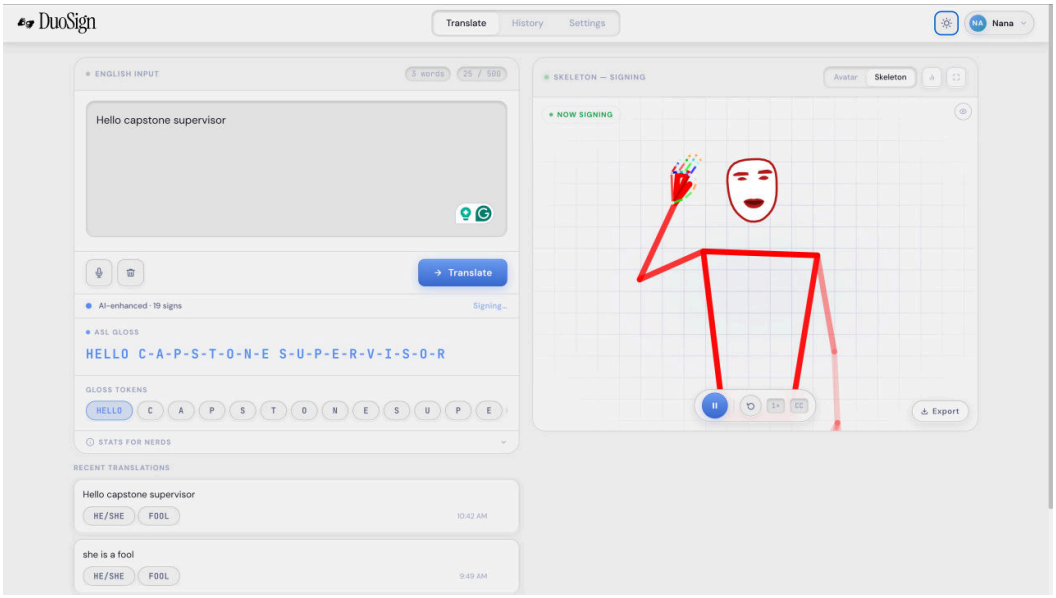


Figure 7: Skeleton view mode for pose playback.

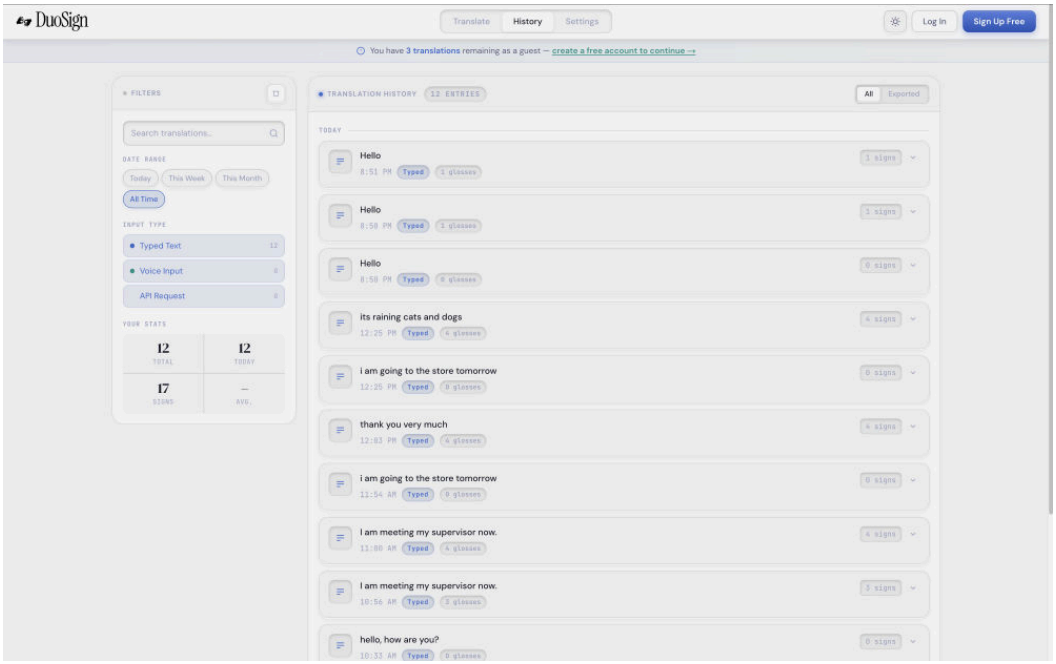


Figure 8: Translation history page.

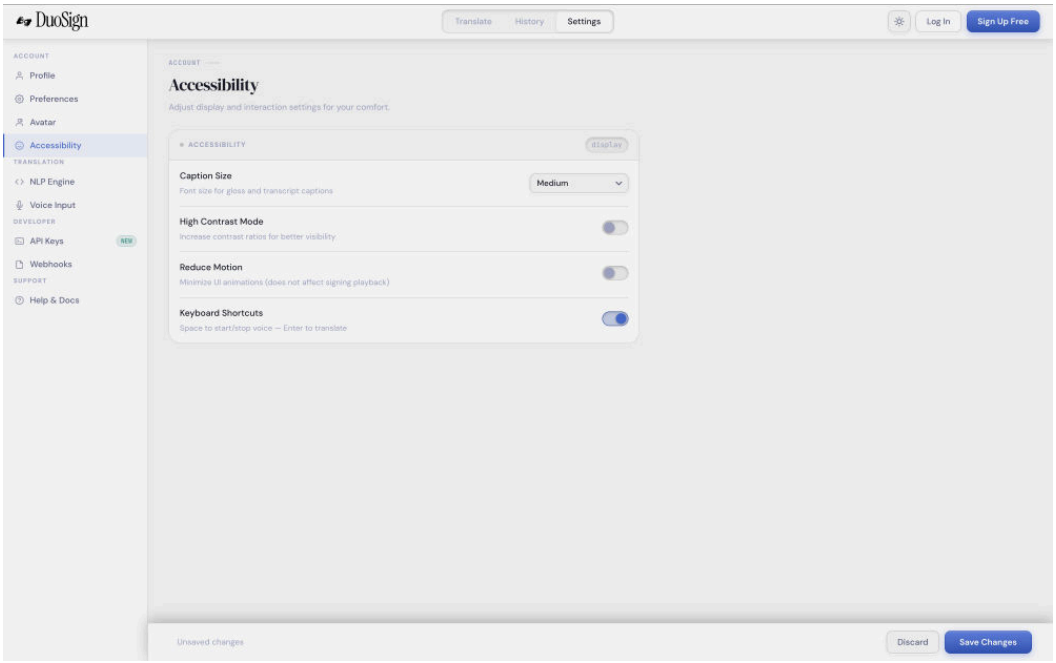


Figure 9: Application settings view.

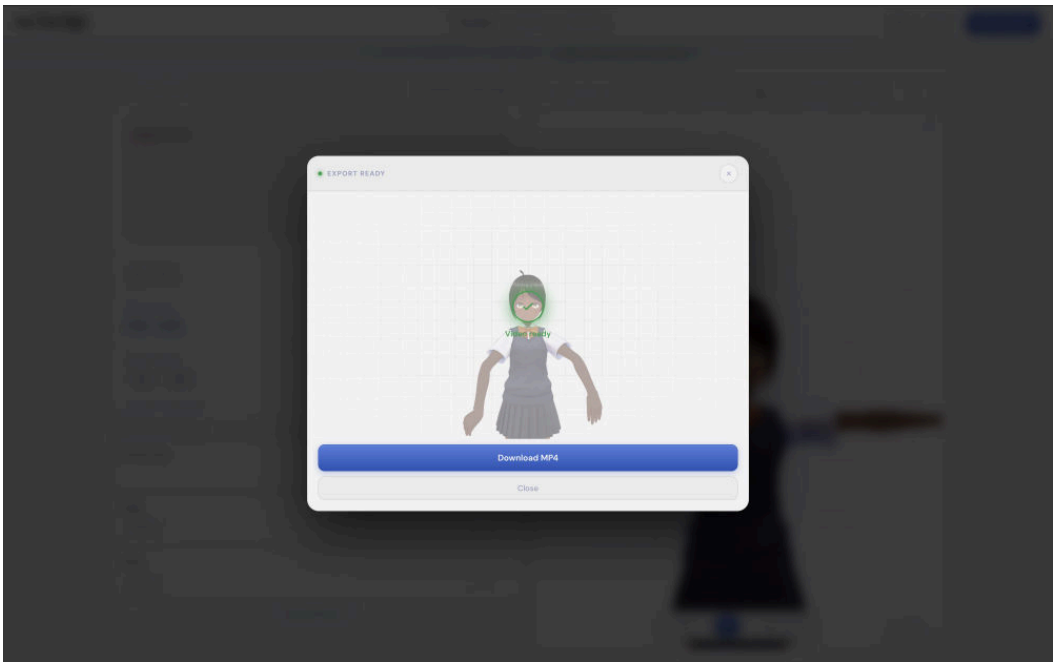


Figure 10: Export modal for saving generated output.



Figure 11: Mobile-responsive interface view.

G Kalidokit Retargeting Capability Matrix

Table 6: ASL parameter coverage under the current Kalidokit-VRM retargeting pipeline.

ASL Parameter	**Pipeline Capability**	**Notes**
Arm location (high/mid/low)	Reasonable	Main arm position is generally captured.
Arm movement path	Reasonable	Arc and direction are better in video-engine mode.
Simple handshapes (open/closed)	Partial	Gross handshape reproduced; fine distinctions degrade.
Complex handshapes	Poor	Hinge-dominant finger model cannot represent nuanced forms reliably.
Palm orientation	Partial	Front-facing configurations are stronger than oblique/occluded ones.
Contact precision (chin/temple/chest)	Partial	Contact location can drift in some clips.
Facial grammar (mouth, brows)	Poor	Not represented with required ASL grammatical fidelity.
Classifier handshape detail	Poor	Fine-grained finger states are underspecified.
Inter-sign natural rest posture	Partial	Neutral states may appear spread due to T-pose defaults.

H Kalidokit and VRM implementation findings

Table 7: Practical findings from the Kalidokit and VRM implementation used in DuoSign.

Component	**What looked promising**	**What worked in practice**	**Main issue / final decision**
Kalidokit hand and pose solving	Direct landmark-to-avatar retargeting in the browser without building a custom solver from scratch.	Good enough for basic motion and simpler signs, especially when source video quality was strong.	Complex handshapes, palm orientation, and contact-heavy signs were less reliable, so the system keeps this as a workable baseline rather than a final solution.
VRM avatar pipeline	A ready-made humanoid rig that could be animated inside the browser with standard web graphics tools.	The VRM setup allowed real-time signing playback, camera switching, and export from the same rendering pipeline.	The final implementation kept the VRM stack that behaved most consistently with Kalidokit, since not every VRM setup produced stable results.
Video-driven landmark path	Using real source video promised more natural timing and fewer retargeting artefacts than static pose playback alone.	This became the main animation path because it preserved the frame cadence Kalidokit handled best.	It still depends on source-video quality and does not fully solve facial grammar or fine contact precision.
Pose-file fallback path	Precomputed pose assets promised faster playback and less repeated inference at runtime.	Useful as a fallback and for skeleton inspection when direct video playback was unavailable or inconsistent.	Motion looked flatter and less natural than the live video path, so it remained a secondary mode rather than the default.

Table 8: Summary of DuoSign backend API endpoints.

Method	Route	Purpose
POST	/api/translate	Runs standard text-to-gloss translation and returns a single response payload.
POST	/api/translate/fast	Returns the rule-based translation path optimised for low latency.
POST	/api/translate/stream	Streams progressive translation updates over SSE for immediate UI feedback.
POST	/api/translate/audio	Accepts recorded audio and returns transcription/gloss output via Whisper integration.
GET	/api/vocabulary	Returns available gloss vocabulary metadata used by the frontend.
GET	/api/pose/{gloss}	Serves precomputed binary .pose landmark data for a given gloss token.
GET	/api/video/{gloss}	Serves the selected source sign video asset for a given gloss token.
POST	/api/export/video	Converts recorded client media to downloadable MP4 (FFmpeg pipeline).
GET	/api/health	Exposes service health status for uptime checks and deployment monitoring.

I Project Management

I.1 Development Methodology

DuoSign followed an iterative development approach aligned to the Ashesi capstone milestone schedule. Table 9 summarises the project phases.

Table 9: DuoSign development timeline.

Phase	**Period**	**Key Activities**
Research & Design	Sep–Dec 2025	Literature review, system architecture design, Figma UI/UX prototyping, IRB submission and approval, WLASL dataset analysis.
Core Backend	Jan 2026	Text-to-gloss NLP pipeline, fingerspelling module, vocabulary manager, LLM fallback integration, FastAPI endpoint scaffolding, WLASL pose extraction pipeline.
Frontend & Animation	Mid-Jan–Feb 2026	Next.js 14 UI implementation from Figma designs, MediaPipe + Kalidokit avatar rendering, skeleton fallback, voice input, playback controls, export modal, history and settings pages.
Integration & Polish	Feb–Mar 2026	End-to-end integration testing, deployment to Vercel and Render, Supabase storage migration, avatar switching, responsive design, accessibility improvements, thesis writing.
Testing & Submission	Mar–Apr 2026	User testing sessions (Deaf/HH and hearing participants), accessibility auditing, data analysis, thesis completion, and final submission.

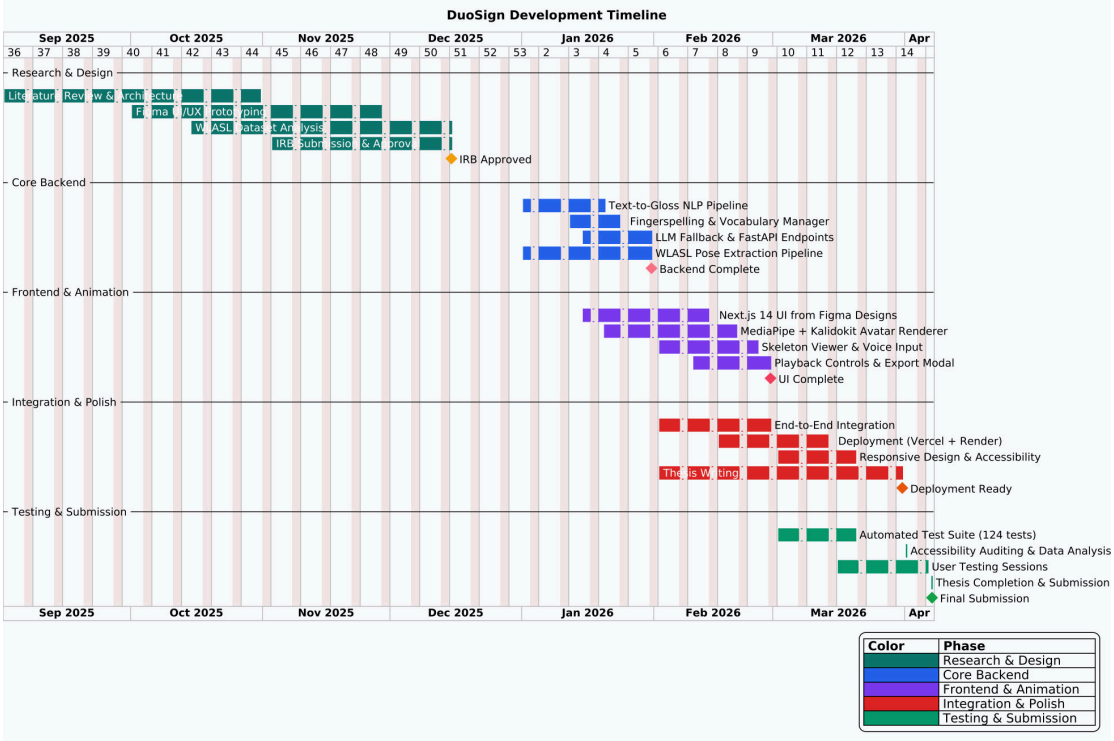


Figure 12: Development timeline Gantt chart.

J External Links and Resources

Table 10: DuoSign external resources.

Resource	URL
Live Application	`https://duosign.vercel.app/`
GitHub Repository	`https://github.com/nanadotam/duosign`
API Documentation	`https://duosign.vercel.app/api/docs`
WLASL Dataset (original)	`https://dxli94.github.io/WLASL/`